



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B
Session - 1.1

Course Anchor/s:

Dr. Chaitra N (RR Campus)
chaitran@pes.edu

Prof. Deepti C (EC Campus)
deeptic@pes.edu

Course Content/Slides:

Prof. V R Badri Prasad



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Introduction

Course Instructor
Prof. V R Badri Prasad

Microprocessor & Computer Architecture (μ pCA)

What we Know? [studied in 3 sem]

Digital Design!

- In DDCO course, *increase logic circuit speed, decrease logic resources required and decrease power consumed.*
- *Goal is to Optimize fundamental physical quantities of time, space and energy.*

Computer Organization !

- Hardware Abstraction above Digital Logic and below Operating System.
- Hardware details which are required by programmers, such as Control signals, Interfaces and Memory

Microprocessor & Computer Architecture (μpCA)

What we study - Syllabus

Unit 1: Architecture and Assembly Language Programming

Introduction, ISA Classification - RISC and CISC, Memory Addressing, Operands - Types and Size, Instruction Set - Operations, Control Flow, Instruction Encoding, Case Study - ARM/ MIPS/ x86 Processor. **16 Hours**

Unit 2: Pipelining

3- Stage Pipelining, 5 - Stage Pipelining, Pipeline Datapath and Control, Data Hazards – Forwarding vs. Stalling, Control Hazards, Branch Prediction Mechanisms and Exceptions, Performance Metrics. **12 Hours**

Unit 3 : Basics of Cache and Cache Optimization

Basics of Caches - Fully Associative, Direct Mapped and Set Associativity, Cache Performance, Basic Cache Optimization- Reduce in Miss Rate. Basic Cache Optimization- Reduce Miss Penalty, Reduce Hit Time. **14 Hours**

Unit 4: Advances in Architecture

Introduction to Parallel Computing, Parallel Computer Memory Architecture and Flynn's Taxonomy, Parallel Computing Concepts and Terminology, Design Issues & Constraints, performance in parallel computing : Amdahl's Law ; Gustafson's Law , Multicore Processors and OpenMP programming, Realization of ML operations on Modern Architecture, Introduction to Heterogenous Parallel Computing, Introduction to GPU Computing. **14 Hours**

Microprocessor & Computer Architecture (μpCA)

Evaluation Policy



	Conduction	Scaled (Marks)
ISA 1 (Units 1 and 2)	40 Marks	20
ISA 2 (Units 3 and 4)	40 Marks	20
Banana Problem	4 Marks Banana problem 1 conducted for 10 marks (Unit 1&2) Banana Problem 2 conducted for 10 marks (Unit 3&4) Each reduced to 2 marks (2*2=4 marks)	10
Orange Problem	6 Marks One Orange problem conducted for 10 marks, reduced to 6 marks	
Laboratory (1–credit) - FSA	● Mini Project component (Jackfruit problem) - 10 marks ● Quiz conducted for 20 marks, reduced to 4 marks ● Continuous evaluation of lab programs - 5 marks + Viva voce – 5 marks – Total 10 marks – reduced to 6 marks	20
ESA	100 Marks (25 marks from each unit, conducted for 100 marks, reduced to 50 marks)	50
Total		120*

*The total of 120 marks will further be scaled down to 100 for grading.

Microprocessor & Computer Architecture (μpCA)

Text and Reference Book(s)



Text Book(s):

- T1. "Computer Organization and Design", Patterson, Hennessey, 5th Edition, Morgan Kaufmann, 2014.
- T2. "Computer Organization and Design – ARM Edition", Patterson, Hennessey, 4th Edition, Morgan Kaufmann, 2010.
- T3. "ARM System Developer's Guide Designing and Optimizing System Software", Andrew N Sloss et al., Elsevier, 2004.
- T4. "Programming Massively Parallel Processors A Hands-on Approach", Third Edition David B. Kirk, Wen-mei W. Hwu, Morgan Kaufmann, 2016
- T5. "Multicore and GPU Programming An Integrated Approach", Gerassimos Barlas, second edition, Morgan Kaufmann, 2022

Reference Book(s):

- R1. "Computer Architecture: A Quantitative Approach", Hennessey, Patterson, 5th Edition, Morgan Kaufmann, 2011.
- R2. "ARM System-on-Chip Architecture", Steve Furber, 2nd Edition, Pearson India, 2015.
- R3. "The Definitive Guide to the ARM Cortex-M0 and Cortex M0+ processors", Joseph Yiu, 2nd Edition, Newnes, 2015



Microprocessor & Computer Architecture (μ pCA)

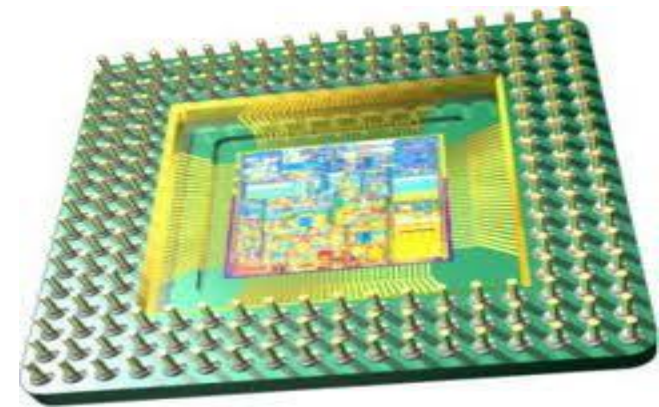
UE24CS251B

Introduction to
Microprocessors and Microcontrollers
RISC and CISC Architectures

Session - 1.11



Is **MICROPROCESSOR**
Brain or Heart
of a computer System
???



Statutory Warning: does not resemble anyone over here

What is a MicroProcessor?

- Brain of the Computer
- Basic Computing Unit on an Integrated Chip (IC)
- Three basic characteristics differentiate microprocessors:
Instruction set , Bandwidth, Clock speed



What is Computer Architecture ?

- The engineering decisions and tradeoffs that must be made in order to produce a ***“Good” Design***.

i.e Execution Time, Space and Resource

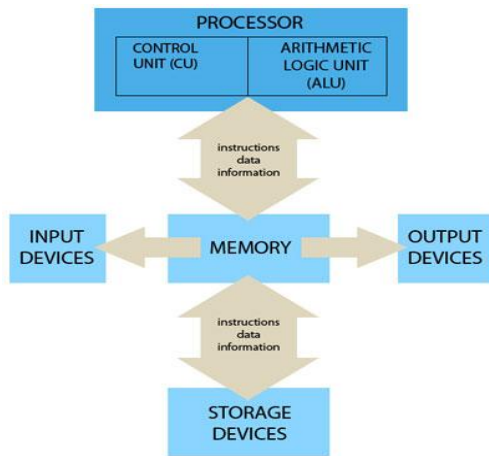
Why Study Microprocessor?

- Everywhere we have devices which are controlled by “Microprocessor” or “Microcontroller”
- Because its part of the Curriculum!!! 😊

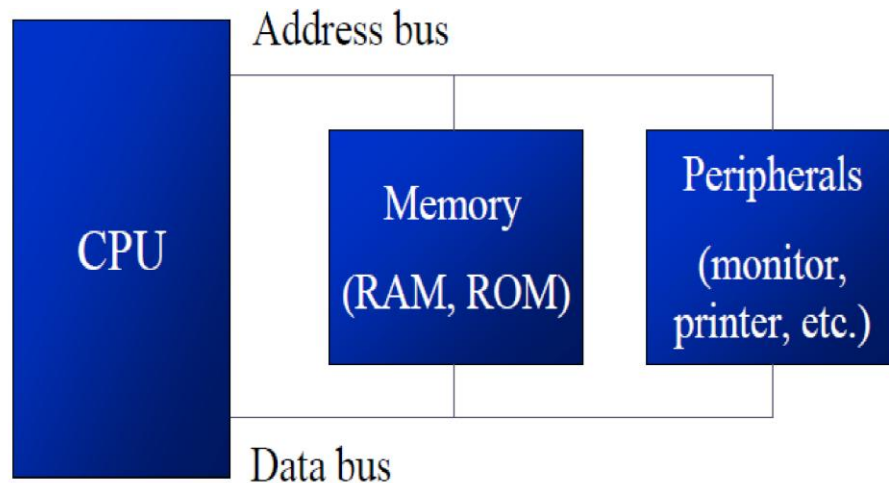


Microprocessor & Computer Architecture

Computer Organization & Computer Architecture

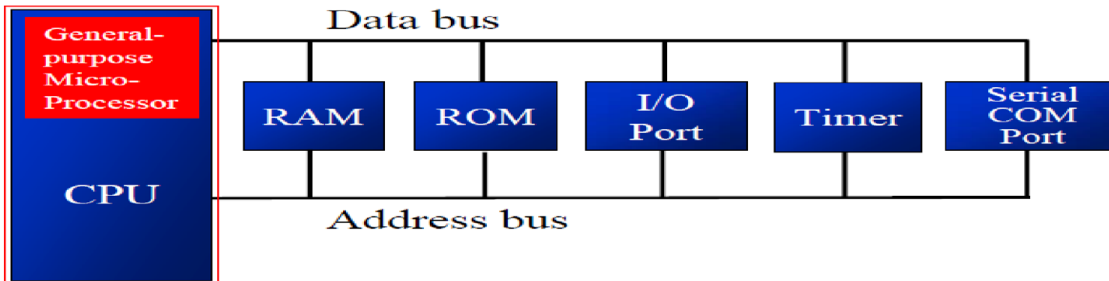


- Computer Organization refers to the **Operational Units and there interconnections** that realize or recognize the specifications of Computer Architecture.
- Organizational **attributes** includes **Hardware details transparent** to the programmer such as **control signals** , **Interface** between computers and peripherals and the memory technology used.



- **Computer Architecture** refers to those attributes of a system visible to a programmer, the other way those attributes that have direct impact on the logical execution of the program.
- Architectural attributes include the **instruction set**, **the no of bits used to represent the data types**, **Input Output mechanism and technique** for addressing memories.

Microprocessor



- Accepts [digital data](#) as input, processes it according to instructions stored in its memory, and provides results as output.
- Don't have RAM, ROM, and other peripheral on the chip.
- Memory , I/O and other components need to be connected externally.
- Cannot be used in compact systems.
- Power Consumption is High.

Microcontroller

Microcontroller



- Comprises of CPU, in addition with a fixed amount of RAM, ROM and other peripherals all embedded on a single chip.
- Also termed as a mini computer or a computer on a single chip.
- Designed to perform **specific tasks**.
- Requires **small resources** like RAM, ROM, I/O ports etc. and hence can be embedded on a single chip.
- Reduces the size and the cost.

Microprocessor & Computer Architecture (μ pCA)

Microprocessor, Where am I?



Microprocessor & Computer Architecture (μ pCA)

Microprocessor, Where am I?



Microprocessor & Computer Architecture (μpCA)

Microprocessor - Evolution

4004	Nov. 15,1971
8008	April 1972
8080	April 1974
8085	March 1976
8086	June 8, 1978
8088	June 1979
80286	Feb. 1982
i80386	1985 - 1990
I80486	1989 - 1992
Intel Pentium	1993 - 1999
Intel Pentium MMX	1996 - 1999
Intel Atom	2008 - 2009 (as Centrino Atom), 2008–present (as Atom)
Intel Celeron	1998–present
Intel Pentium Pro	1995 - 1998
Intel Pentium II	1997 - 1999
Intel Pentium III	1999 - 2003
Intel Xeon	1998–present
Pentium 4	2000 - 2008
Pentium 4	2000 - 2008
Pentium M	2003 - 2008
Pentium D/EE	2005 - 2008

Microprocessor & Computer Architecture (μpCA)

Microprocessor - Evolution

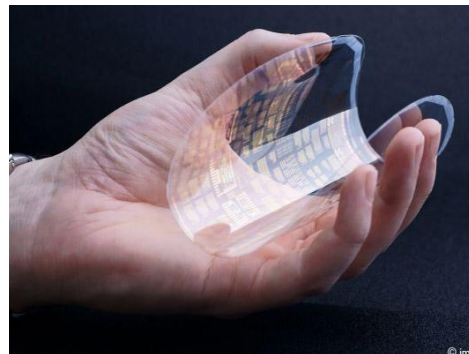


Intel Pentium Dual-Core	2006 - 2009
Intel Pentium (2009)	2009–present
Intel Core	2006 - 2008
Intel Core 2	2006 - 2011
Intel Core i3	2010–present
Intel Core i5	2009–present
Intel Core i7	2008–present
Intel Core i7 (Extreme Edition)	2011–present
Intel Core i9	2018–present
Intel Core i9 (Extreme Edition)	Q3 2017–present

Microprocessor & Computer Architecture (μ pCA)

Why Study Microprocessor ?

- Because its part of the Curriculum!!! 😊
- Everywhere we have devices which are controlled by “Microprocessor” or “Microcontroller”



Microprocessor & Computer Architecture (μpCA)

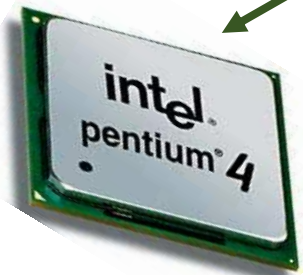
Classification of Microprocessors

CISC: Complex Instruction Set Computer

RISC: Reduced Instruction Set Computer



CISC



RISC



Special
Purpose



How to Classify Microprocessors?

Three basic characteristics differentiate microprocessors:

- Instruction set**: The set of instructions that the microprocessor can execute.
- Bandwidth** : The number of bits processed in a single instruction.
- Clock Speed** : How many instructions per second the processor can execute.

Given in megahertz (MHz),

- The complete collection of instructions that are understood by a CPU

Example: Data Movement, Data Processing, Branch Instruction.....etc

- Designers decide the ISA for respective Microprocessor.
- It is responsibility of Compiler to generate correct and Optimized code for respective Microprocessor.
- Main criteria to classify CISC & RISC Processor

Preamble

- Earliest machines were programmed in assembly language and memory was slow and expensive.
- The CISC philosophy made sense, and was commonly implemented in such large computers as the PDP-11 and the DEC system 10 and 20 machines.
- Most common microprocessor designs such as the Intel 80x86 and Motorola 68K series followed the CISC philosophy.
- But recent changes in software and hardware technology have forced a reexamination of CISC and many modern CISC processors are hybrids, implementing many RISC principles and vive versa.
- CISC was developed to make compiler development simpler. It shifts most of the burden of generating machine instructions to the processor.

- CISC processors were evolved in the 1970s.
- During this period, the computer memory used to be 'small' and 'very expensive'.
- Philosophy of CISC processors is to simplify the code and make it **shorter** in order to reduce the memory requirement.
- In a CISC processor, a single instruction has 'several low-level operations. This makes the CISC instructions short but 'complex'.

CISC: Complex Instruction Set Computer

Architecture

- CISC processors typically had **variable length** instruction sets with **many formats**.
- Typically allowed **values in memory** to be used as **operands** in data processing instructions.
- A **2-operand** instruction format
- **Register sets** were getting **larger**, but **none** was as large as RISC and most processors had different registers for different purposes.

CISC Organization

- Microprogrammed control instruction decode logic.
 - It was easier to implement and less expensive.
- CISC processors allowed **little**, if any, overlap between consecutive instructions.
 - The ease of microcoding new instructions allowed designers to make CISC machines upwardly compatible
- May take many clock cycles to complete a single instruction.
 - Microprogram instruction sets can be written to match the constructs of high-level languages, the compiler does not have to be complicated.
- Pentium is considered a modern CISC processor.

RISC – REDUCED INSTRUCTION SET COMPUTER

- **John Cocke - (May 30, 1925 – July 16, 2002)**, was an American [computer scientist](#) recognized for his large contribution on [computer architecture](#) and optimizing [compiler](#) design.
- Originated the **RISC** concept in **1974**.
- Considered by many to be "**the father of [RISC](#) architecture.**"
- Proved that about [20%](#) of the instructions in a computer did [80%](#) of the work.



The term **RISC** is **credited** to **David Patterson**, a teacher at the University of California in Berkeley. The concept was used in Sun Microsystems' [SPARC](#) microprocessors and led to the founding of what is now MIPS Technologies, part of Silicon Graphics. A number of current microchips now use the RISC concept.

- Idea of RISC processors was originated in the 1974 and Implemented in 1980's.
- Philosophy of RISC processors is to simplify operation of Individual Instruction.
- Separate Instruction for Load and Store (Data Movement)
- ***Number of lines of code*** in the program ***increases*** but Amount of work done by individual instruction is ***REDUCED***.

RISC – REDUCED INSTRUCTION SET COMPUTER

Architecture

- A **fixed** (32-bit).
- A **load-store architecture** where instructions that process data, operate only on registers and are separate from instructions that access memory.
- A large register bank (thirty-two 32-bit registers).
- Instruction size with few formats.

RISC Organization

- Hard-wired instruction decode logic (design of the control unit).
- Pipelined execution.
- Single-cycle execution.

RISC Advantages

- A smaller die size.
- A shorter development time.
- A higher performance.

RISC Drawbacks

- **Poor** code density.
- Do not execute x86 code.

RISC Vs CISC Machines Today

- With the rise of embedded systems and mobile computing, the terms RISC and CISC have lost their significance
- Both architectures almost seem to have **adopted** the strategies of the other.
- Some RISC systems provide more extravagant instruction sets than some CISC systems.
- CISC chips are now able to execute more than one instruction within a single clock, including Pipelining.
- The RISC Vs CISC debate started when chip-area and processor design complexity were issues – now energy and power are the issues.
- The two top competitors today are ARM and Intel – Intel focuses on performance and ARM focuses on efficiency (*British company* – **Acron** **RISC Machine**)

RISC – REDUCED INSTRUCTION SET COMPUTER

- **State of the art** processor technology has changed significantly since RISC chips were first introduced in the early '80s.
- A number of **advancements** are used by both **RISC and CISC processors**.
- The lines between the **two architectures** have begun to **blur**.
- In fact, the two architectures almost seem to have **adopted** the strategies of the other.
- Since, the processor speeds have **increased**, CISC chips are now able to execute more than one instruction within a single clock.
- This also allows CISC chips to make use of **pipelining**.
- With other technological improvements, it is now possible to fit many more transistors on a single chip.

RISC

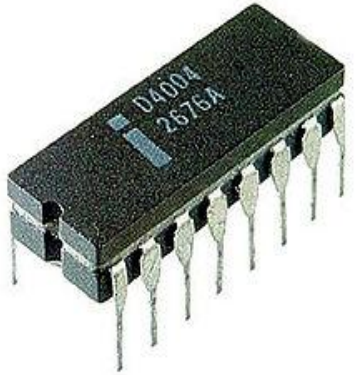
- Simple instructions, few in number.
- Fixed length instructions.
- Multiple register sets.
- Three operands per instruction.
- Parameter passing through register windows.
- Single-cycle instructions.
- Hardwired control.
- Highly pipelined.
- Complexity in compiler.
- Only **LOAD/STORE** instructions access memory.
- Few addressing modes.

CISC

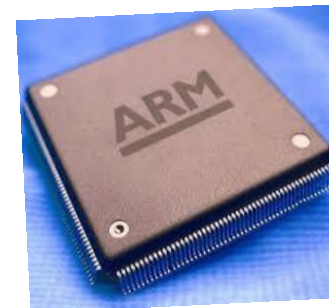
- Many complex instructions.
- Variable length instructions.
- Single register set.
- One or two register operands per instruction.
- Parameter passing through memory.
- Multiple cycle instructions.
- Microprogrammed control.
- Less pipelined.
- Many instructions can access memory.
- Many addressing modes.

Microprocessor & Computer Architecture (μ pCA)

Our Choice-> ARM



Mostly on-----?





THANK YOU



V R BADRI PRASAD

Department of Computer Science and Engineering



Course Instructor
Prof. V R Badri Prasad

Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 2



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Processor design trade-offs

What does the processor do?

Misconception - computers spend their time computing, i.e., carrying out arithmetic operations on user data.

- Although they do a fair amount of arithmetic, most of this is with addresses in order to locate the relevant data items and program routines.
 - Then, having found the user's data, most of the work is in moving it around rather than processing it in any transformational sense.

Consider a Scenario : The programmer generally wants to express his or her program in as abstract a way as possible, using a high-level language which supports ways of handling concepts that are appropriate to the problem.

- Modern trends towards functional and object-oriented languages move the level of abstraction higher.
 - The **semantic gap** between a high-level language construct and a machine instruction is bridged by a **compiler** (*A computer program that translates a high level language program into a sequence of machine instructions*).
 - Hence, the processor designer should define a good instruction set.

- A typical set of statistics is shown below were gathered [running a print preview](#) program on an ARM instruction emulator, but are broadly typical of what may be expected from other programs and instruction sets.

Instruction type	Dynamic usage
Data movement	43%
Control flow	23%
Arithmetic operations	15%
Comparisons	13%
Logical operations	5%
Other	1%

- Instructions to [optimize](#) are those concerned with [data movement](#), either between the [processor registers](#) and [memory](#) or from [register to register](#).
- Secondly, the [control flow instructions](#) such as [branches](#) and [procedure calls](#).
- [Arithmetic operations](#) are down at **15%**, as are [comparisons](#) at **13%**.

Now, that it is clear what processors spend their time doing, is there a way to make the processor work faster ?

- The most important of these is pipelining.
 - A very effective way of exploiting concurrency in a general-purpose processor.
- Another important technique is the use of a cache memory.
 - A CPU cache is a cache used by the CPU to reduce the average time to access data from the main memory.
- A third technique, super-scalar instruction execution.
 - A superscalar processor executes more than one instruction during a clock cycle by simultaneously dispatching multiple instructions to different execution units on the processor.

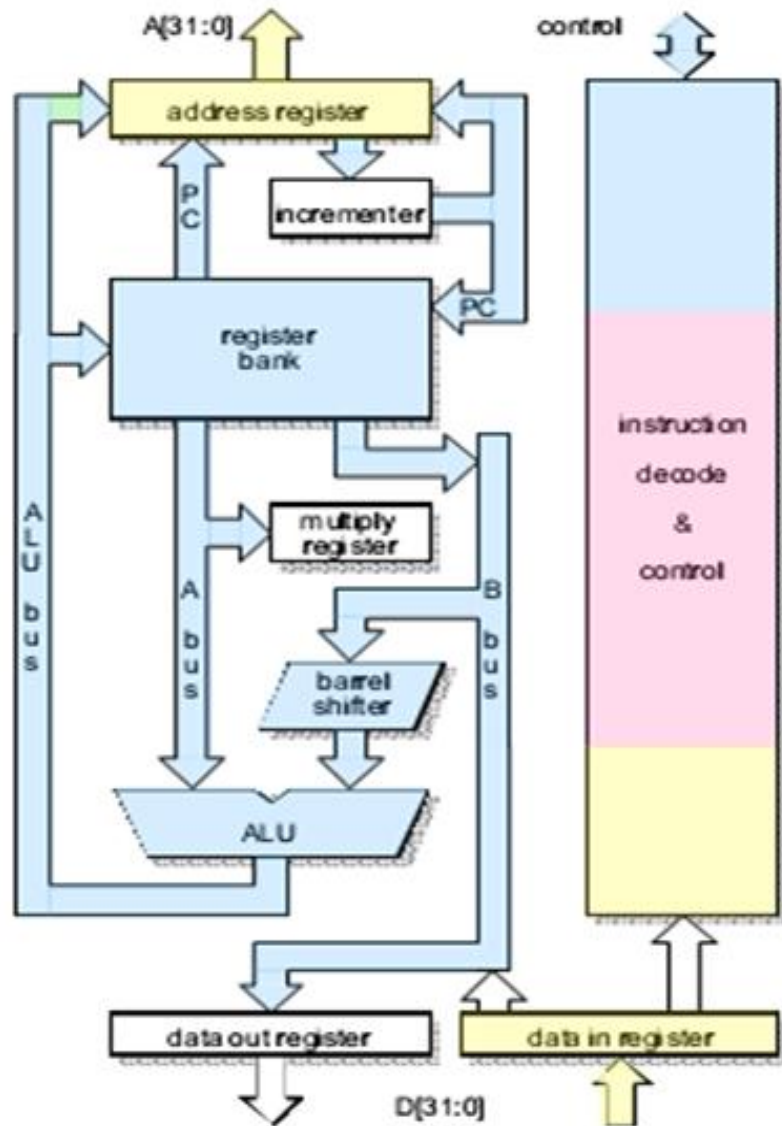


Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

ARM ISA:
Instruction Set Architecture

ARM Processor – 3 stage Pipeline Architecture



The ARM [A CORN RISC MACHINE] is a *RISC* processor

3 stages of Pipeline

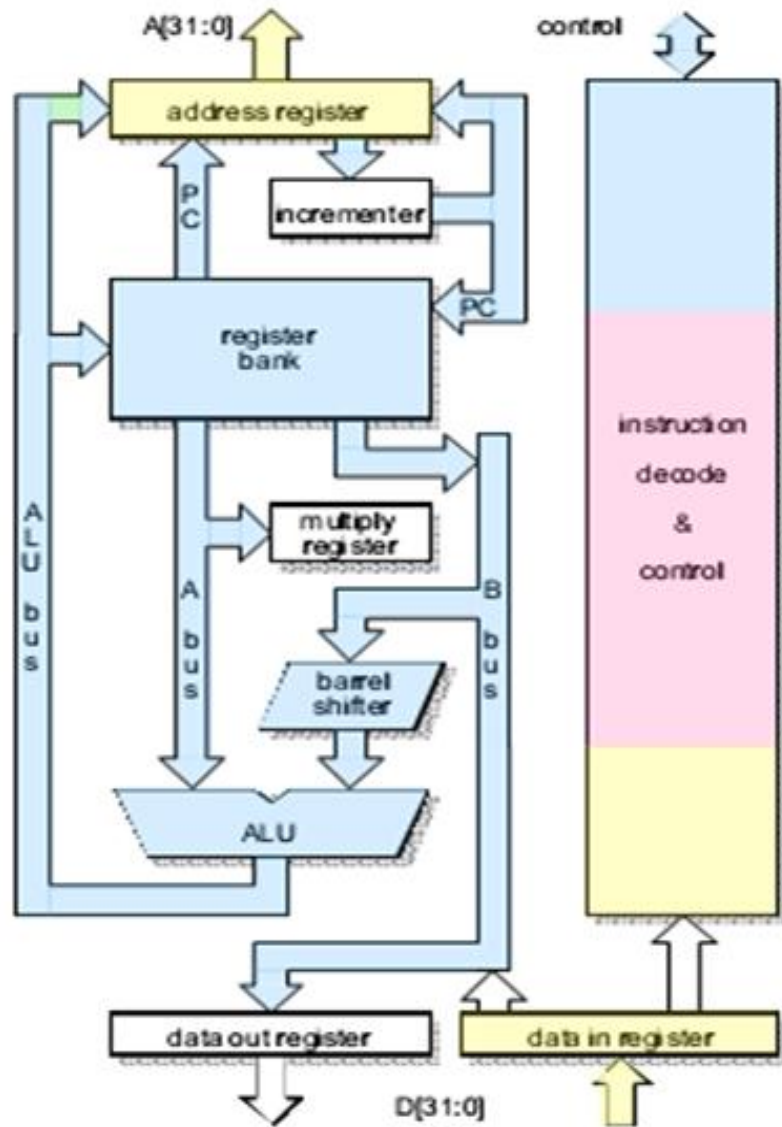
fetch

decode

execute

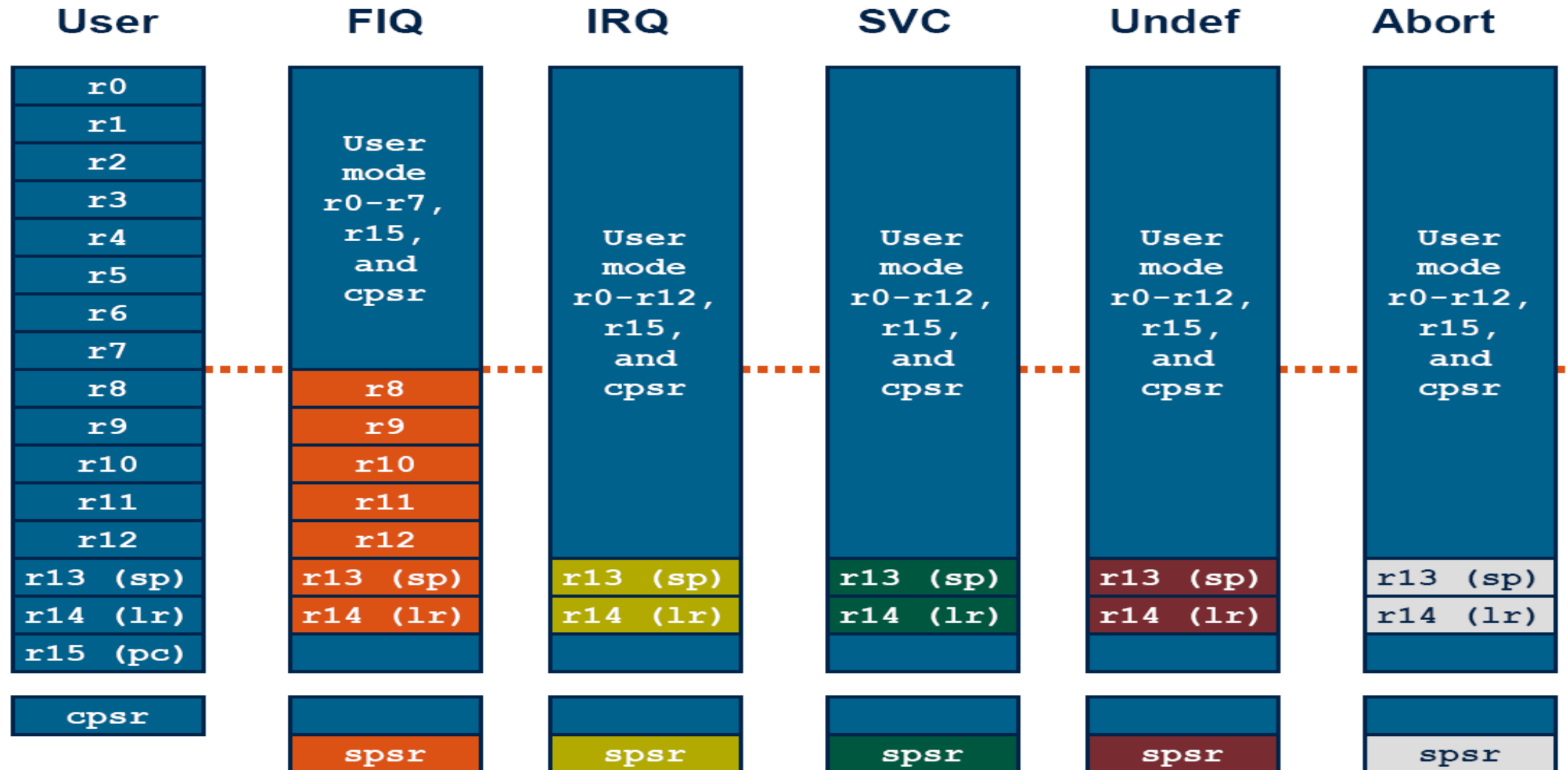
- The RISC features incorporated by ARM.
 - A load-store architecture.
 - Fixed-length 32-bit instructions.
 - 3-address instruction formats.

Organization of ARM – 3 stage PP



- The **register bank**, which stores the processor state.
 - Two read ports and One write port - to access any register
 - Plus an additional read port and an additional write port that give special access to r15
- The **barrel shifter**, which can shift or rotate one operand by any number of bits.
- The **ALU**, which performs the arithmetic and logic functions required by the instruction set.
- The **address register** and **incrementer**, which select and hold all memory addresses and generate sequential addresses when required.
- The **data registers**, which hold data passing to and from memory.
- The instruction decoder and associated control logic.

Register organization & Modes



The ARM Register Set – SVC Mode

Current Visible Registers

SVC Mode

r0
r1
r2
r3
r4
r5
r6
r7
r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)
r15 (pc)
cpsr
spsr

Banked out Registers

User

FIQ

IRQ

Undef

Abort

r13 (sp)
r14 (lr)

r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)

r13 (sp)
r14 (lr)

r13 (sp)
r14 (lr)

r13 (sp)
r14 (lr)

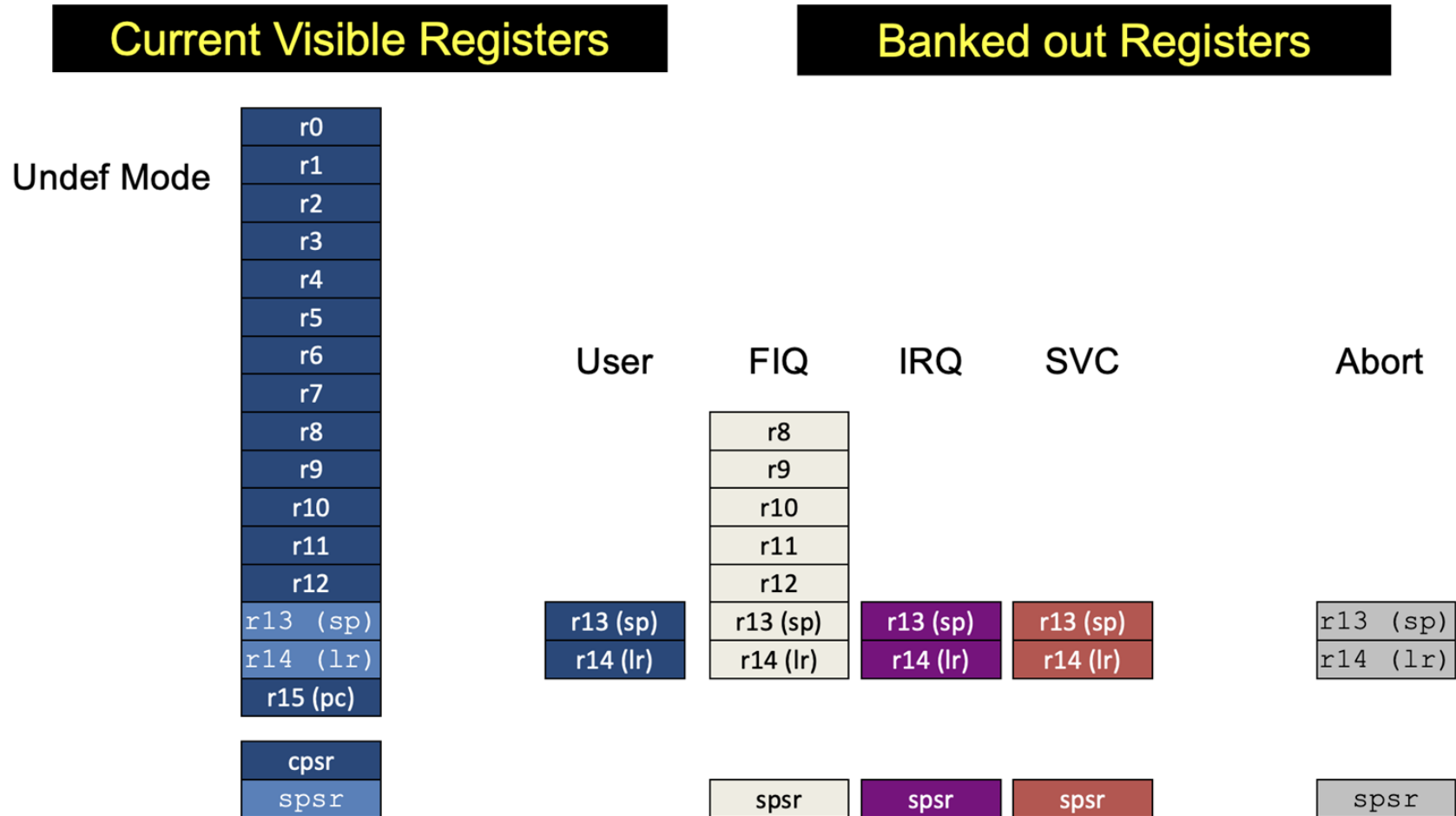
spsr

spsr

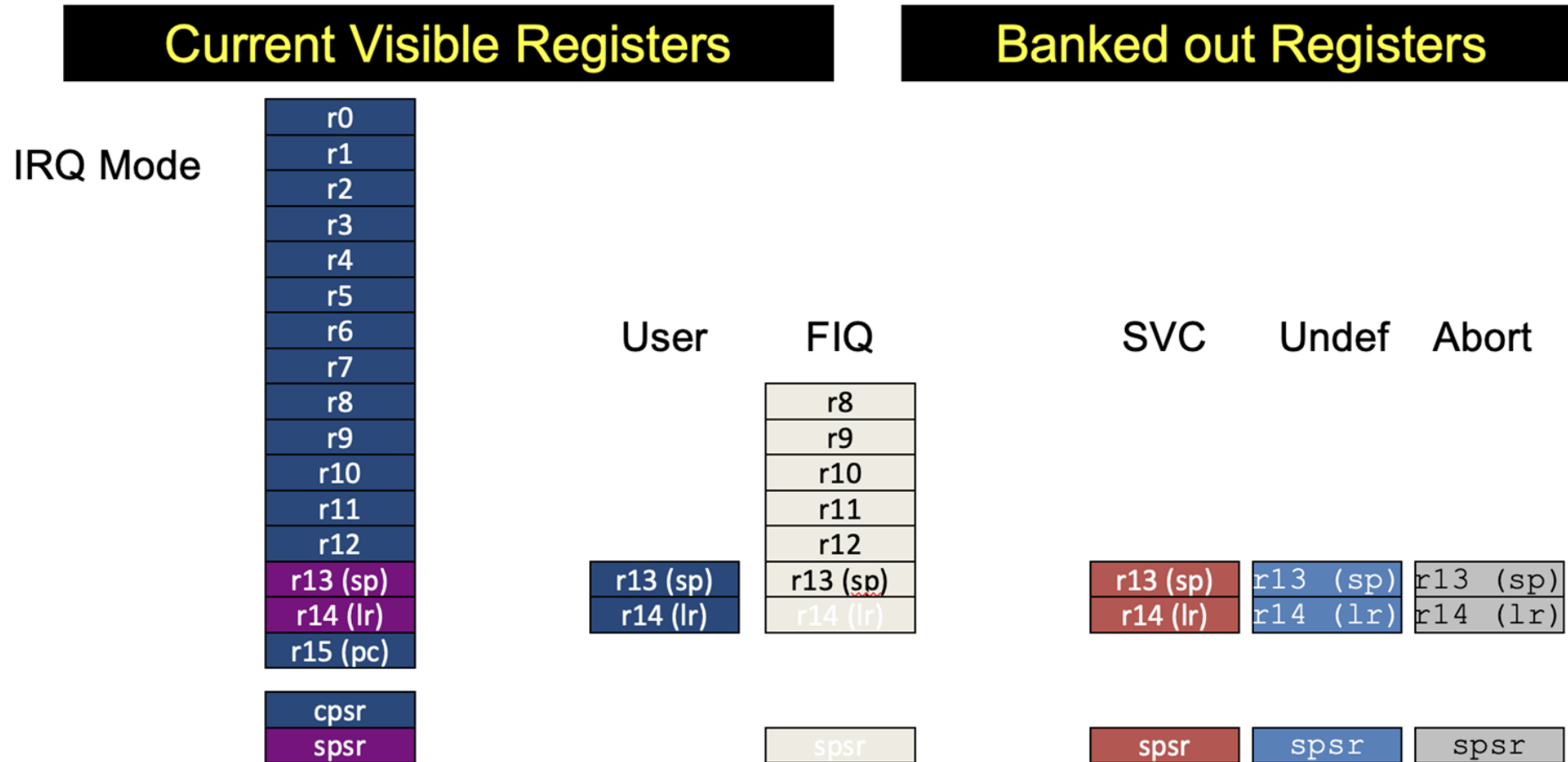
spsr

spsr

The ARM Register Set – Undef Mode



The ARM Register Set – IRQ Mode



The ARM Register Set – FIQ Mode

Current Visible Registers

FIQ Mode

r0
r1
r2
r3
r4
r5
r6
r7
r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)
r15 (pc)
cpsr
spsr

User

r8
r9
r10
r11
r12
r13 (sp)
r14 (lr)

Banked out Registers

IRQ

SVC

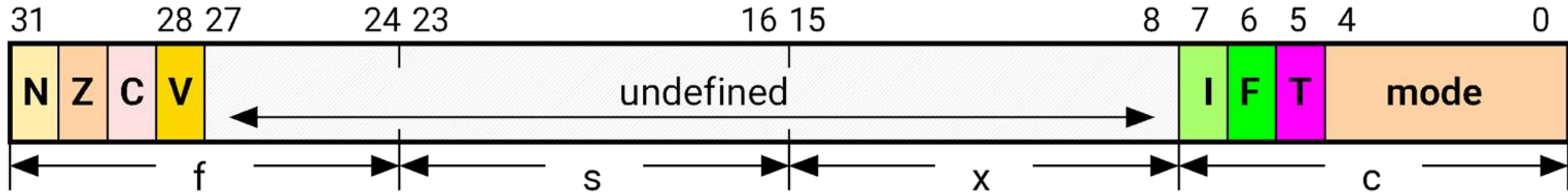
Undef

Abort

r13 (sp)	r13 (sp)	r13 (sp)	r13 (sp)
r14 (lr)	r14 (lr)	r14 (lr)	r14 (lr)

spsr	spsr	spsr	spsr
------	------	------	------

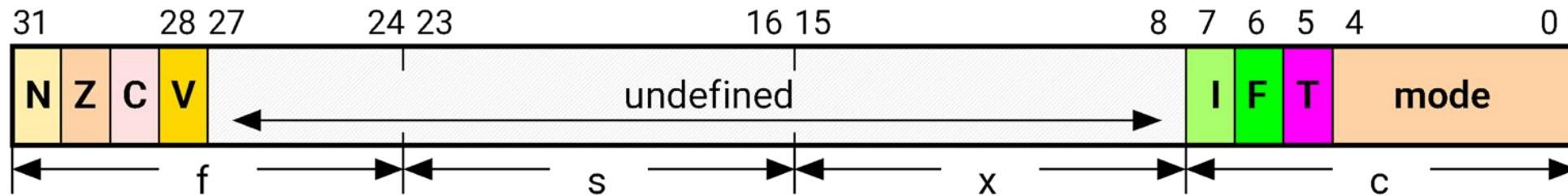
Status Register (SR)



SR fields are divided into four fields, each 8-bit wide:

- Flags (f)
- Status (s) : *Reserved for Future*
- Extension (x) : *Reserved for Future*
- Control (c).

Status Register (SR)



- **N: Negative:** Set to 1 if the Result of previous instruction is Negative.
- **Z: Zero:** Set to 1 if the Result of previous instruction is Zero.
- **C: Carry:** Set to 1, if result of either an Arithmetic or Shifter produce a carry-out,
- **V: oVerflow:** Set to 1, if previous instruction produce an overflow into the sign bit.

Mode bits

10000	User
10001	FIQ
10010	IRQ
10011	Supervisor
10111	Abort
11011	Undefined
11111	System

Interrupt Disable bits.

I = 1: Disables the IRQ.

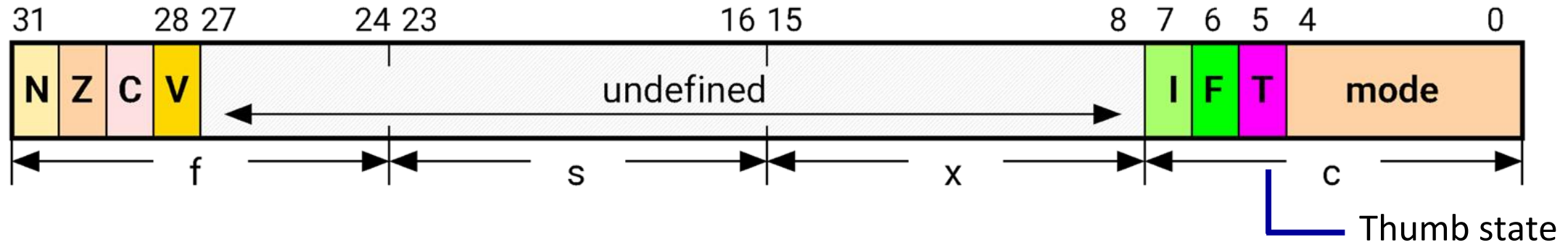
F = 1: Disables the FIQ.

T Bit (Arch. with Thumb mode only)

T = 0: Processor in ARM state

T = 1: Processor in Thumb state

Status Register (SR)



- Thumb is a 16-bit instruction set
 - Optimized for **code density** from C code (~65% of ARM code size)
 - Improved performance from memory with a **narrow data bus**
 - Subset of the **functionality** of the ARM instruction set
 - Only Low Registers **R0-R7** are used
 - Constants are of limited size.
 - Inline barrel shifter not used.

Status Register : CPSR & SPSR

Current Program Status Register: CPSR

- ARM core uses CPSR to monitor & control internal operations.
- The current status of the program under execution, such as, result of current execution instruction is zero/ -ve are captured here.

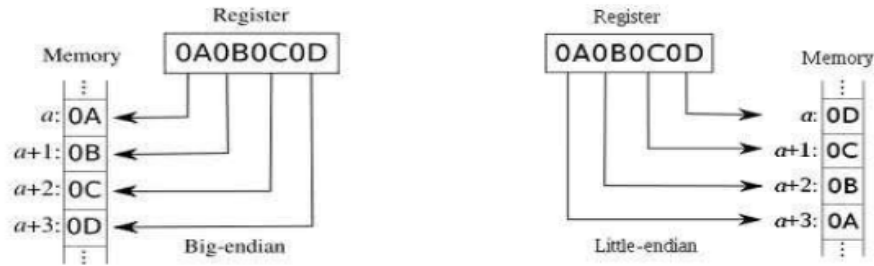
Saved Program Status Register: SPSR

- Processor, While Shifting one mode to another mode, CPSR will be copied to SPSR.
- SPSR will be copied back to CPSR when it return back to previous mode.

Example:

Suppose Processor is in USER mode of operation and if IRQ request arrives then processor has to switch itself to IRQ mode of operation. First CPSR is get copied to SPSR then processor will serve IRQ mode. After serving IRQ mode processor should return to USER mode and should resume its working. So SPSR is get copied again into CPSR to serve USER mode.

Big Endian vs. Little Endian



- ARM processor supports both **little endian & big endian** methods for storing data in memory.

ARM processors support **six data types**.

- 8-bit** signed and unsigned bytes.
- 16-bit** signed and unsigned half-words; these are aligned on 2-byte boundaries.
- 32-bit** signed and unsigned words; these are aligned on 4-byte boundaries.

ARM Instruction Set Architecture - Features

- All ARM instructions are 32 bits wide.
- The load-store architecture.
- 3-Address data processing instructions (that is, the two source operand registers and the result register are all independently specified).
- Conditional execution of every instruction.
- Inclusion of very powerful load and store multiple register instructions.
- Ability to perform a general shift operation and a general ALU operation in a single instruction that executes in a single clock cycle.

- The instruction set will only process [ADD, SUB] values which are in registers [Immediate data] and will always place the results of such processing into a register.
 - **LOAD** instructions operations which apply to memory state are ones which copy memory values into registers.
 - **STORE** instructions copy register values into memory.
- **ARM** does not support '**memory-to-memory**' operations.
- ARM instructions fall into one of the following **3 categories**.
 - **Data processing** instructions.
 - Ex: MOV, ADD, SUB, CMP, etc.,
 - **Data transfer** instructions.
 - Ex: LDR, STR
 - **Control flow** instructions
 - Ex: B, BGT, BEQ, BL, etc.,

- **Register windows.**

- RISC processors incorporated a **large** number of registers.
- Procedure entry and exit instructions moved the visible 'window' to give each procedure access to new registers.
- Thereby **reducing the data traffic** between the processor and memory resulting from register saving and restoring.
- The principal problem with register windows is the large chip area occupied by the large number of registers.

This feature was therefore rejected on cost grounds.

The RISC Features rejected by ARM.

- **Delayed branches.**

- Branches cause pipelines problems since they **interrupt** the smooth flow of instructions.
- RISC processors **ameliorate** the problem by using delayed branches where the branch takes effect *after* the following instruction has executed.
- Work well on single issue pipelined processors
 - But, **do not scale well** to super-scalar implementations & can interact **badly with branch prediction** mechanisms.

- **Single-cycle execution of all instructions.**

- ARM **executes** most data processing instructions in a single clock cycle, **many** other instructions take **multiple clock cycles**.
- A simple load or store instruction requires at least two memory accesses.
- Single cycle operation of all instructions is only possible with **separate** data and instruction memories.

1. Which category of instructions typically accounts for the largest proportion of execution time in ARM-based processors?

- A. Arithmetic and logical instructions
- B. Data movement instructions
- C. Comparison instructions
- D. Floating-point instructions

2. Which sequence correctly represents the stages of the ARM 3-stage pipeline?

- A. Fetch → Decode → Execute
- B. Fetch → Execute → Write Back
- C. Decode → Execute → Memory Access
- D. Fetch → Memory Access → Execute

3. Why does the ARM architecture not support memory-to-memory arithmetic operations?

- A. To simplify compiler design
- B. To reduce instruction length
- C. To enforce a load–store architecture
- D. To improve branch prediction accuracy

4. What is the primary purpose of the Saved Program Status Register (SPSR) in the ARM architecture?

- A. To store arithmetic and logical results
- B. To hold the return address of subroutines
- C. To preserve processor state during mode switching
- D. To control memory access permissions

1. Which category of instructions typically accounts for the largest proportion of execution time in ARM-based processors?

- A. Arithmetic and logical instructions
- B. Data movement instructions
- C. Comparison instructions
- D. Floating-point instructions

Answer - Data movement instructions

2. Which sequence correctly represents the stages of the ARM 3-stage pipeline?

- A. Fetch → Decode → Execute
- B. Fetch → Execute → Write Back
- C. Decode → Execute → Memory Access
- D. Fetch → Memory Access → Execute

Answer - Fetch → Decode → Execute

3. Why does the ARM architecture not support memory-to-memory arithmetic operations?

- A. To simplify compiler design
- B. To reduce instruction length
- C. To enforce a load–store architecture
- D. To improve branch prediction accuracy

Answer - To enforce a load-store architecture

4. What is the primary purpose of the Saved Program Status Register (SPSR) in the ARM architecture?

- A. To store arithmetic and logical results
- B. To hold the return address of subroutines
- C. To preserve processor state during mode switching
- D. To control memory access permissions

Answer - To preserve processor state mode switching



THANK YOU



V R BADRI PRASAD

badriprasad@pes.edu

Department of Computer Science and Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 3

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

ARM program structure
and
Introduction to Arm Simulator

ARM Program Structure

Address	Instruction & Data
	.text
Address of Instruction 1	ARM Instruction_1
Address of Instruction 2	ARM Instruction_2
.....	
.....	
Address of Instruction n	ARM Instruction_n
	.data
Address of Data1	Declaration of variable 1
Address of Data2	Declaration of variable 2
.....	
.....	
Address of Data n	Declaration of variable n

Note: Blue color depict the code written by the programmer
 Orange color depict the address assigned during execution

General Format of Instruction

MNEMONIC{condition}{S} {Rd},Operand1,Operand2

MNEMONIC - Short name of the instruction. *Eg: ADD,SUB....*

{condition} - Condition that is needed to be met in order for the instruction to be executed *EG: EQ, MI,GT,LT,LE,AL,NE*

{S} - An optional suffix. If S is specified, the condition flags are updated on the result of the operation . *Eg: To set N,O, C, V of CPSR*

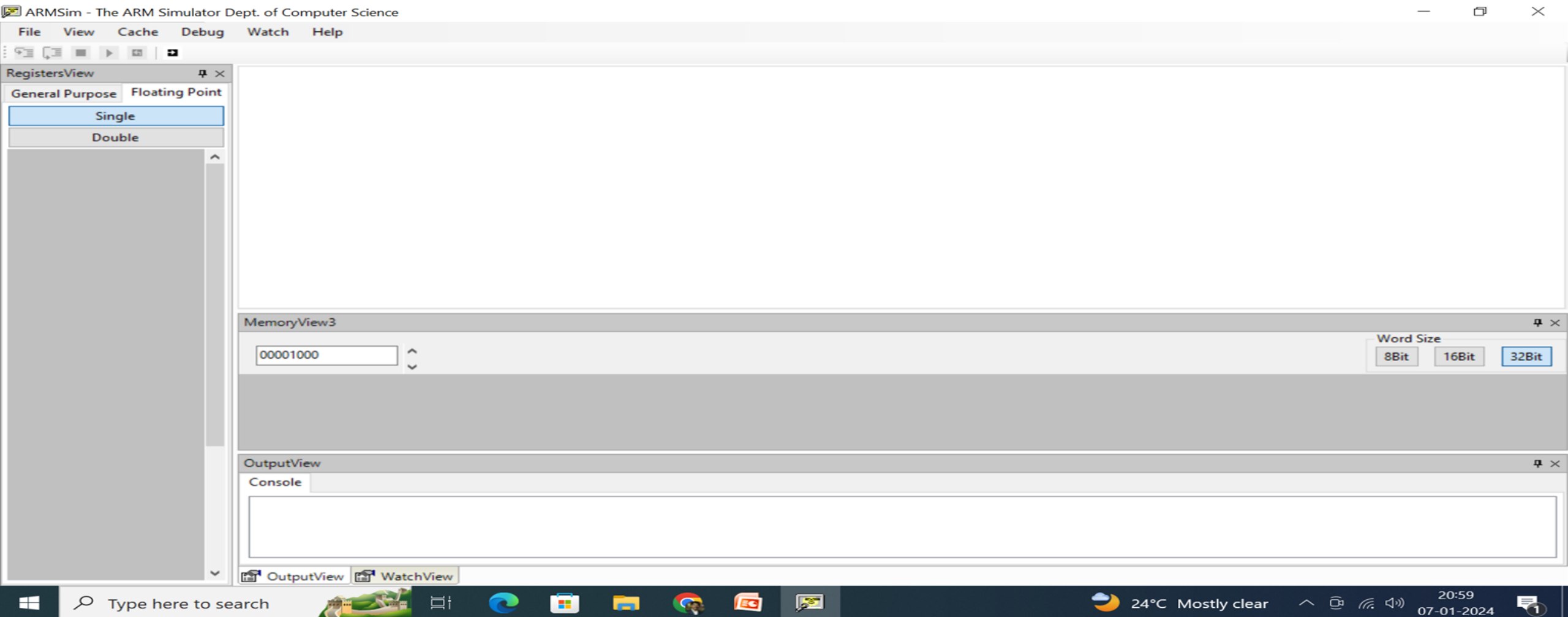
{Rd} - Register (destination) for storing the result of the instruction

Operand1 - First operand. Either a register or an immediate value

Operand2 - Second (flexible) operand. Can be an immediate value (number) or a register with an optional shift

Microprocessor & Computer Architecture (μ pCA)

ARMSIMULTOR



NOTE: Recommended to use and demo on ARM simulator directly.

Loading Sample Program on ARMSIMULATOR

ARMSim - The ARM Simulator Dept. of Computer Science

File View Cache Debug Watch Help



RegistersView

General Purpose Floating Point

Hexadecimal
Unsigned Decimal
Signed Decimal

R0 : 00000000
R1 : 00000000
R2 : 00000000
R3 : 00000000
R4 : 00000000
R5 : 00000000
R6 : 00000000
R7 : 00000000
R8 : 00000000
R9 : 00000000
R10 (s1) : 00000000
R11 (fp) : 00000000
R12 (ip) : 00000000
R13 (sp) : 00005400
R14 (lr) : 00000000
R15 (pc) : 00001000

CPSR Register
Negative (N) : 0
Zero (Z) : 0
Carry (C) : 0
Overflow (V) : 0
IRQ Disable : 1
FIQ Disable : 1
Thumb (T) : 0
CPU Mode : System
0x000000df

addition.s

```
; Simple Code for Addition of 2 numbers.
00001000:E3A01003 MOV R1, #3
00001004:E3A02004 MOV R2, #4
00001008:E0813002 ADD R3, R1, R2
```

MemoryView3

00001000

Word Size: 8Bit 16Bit 32Bit

00001000	E3A01003	E3A02004	E0813002	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181
0000102C	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181
00001058	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181
00001084	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181

OutputView

Console Stdin/Stdout/Stderr

Loading assembly language file F:\academics\subjects\MPCA\MPCA 2024\Slides\Sample Programs\addition.s

OutputView WatchView

Microprocessor & Computer Architecture (μpCA)

Register View (ARMSIM Simulator)

ARMSim - The ARM Simulator Dept. of Computer Science

File View Cache Debug Watch Help

RegistersView

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0 : 00000000
R1 : 00000000
R2 : 00000000
R3 : 00000000
R4 : 00000000
R5 : 00000000
R6 : 00000000
R7 : 00000000
R8 : 00000000
R9 : 00000000
R10 (s1) : 00000000
R11 (fp) : 00000000
R12 (ip) : 00000000
R13 (sp) : 00005400
R14 (lr) : 00000000
R15 (pc) : 00001000

CPSR Register
Negative (N) : 0
Zero (Z) : 0
Carry (C) : 0
Overflow (V) : 0
IRQ Disable : 1
FIQ Disable : 1
Thumb (T) : 0
CPU Mode : System

0x000000df

a1.s

```
00001000:E3A00003  mov r0, #3
00001004:E3A02004  Mov r2, #4
00001008:E0801002  ADD R1, R0, R2
```

MemoryView3

Word Size
8Bit 16Bit 32Bit

00001000	E3A00003	E3A02004	E0801002	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181
0000102C	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181
00001058	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181
00001084	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181

OutputView

Console Stdin/Stdout/Stderr

Loading assembly language file F:\academics\subjects\MPCA\MPCA 2024\a1.s

OutputView WatchView

General Format of Instruction Example

ARMSim - The ARM Simulator Dept. of Computer Science

FileViewCacheDebugWatchHelp

RegistersView

General PurposeFloating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0 : 00000000

R1 : 00000003

R2 : 00000004

R3 : 00000007

R4 : 00000000

R5 : 00000000

R6 : 00000000

R7 : 00000000

R8 : 00000000

R9 : 00000000

R10 (s1) : 00000000

R11 (fp) : 00000000

R12 (ip) : 00000000

R13 (sp) : 00005400

R14 (lr) : 00000000

R15 (pc) : 0000100c

CPSR Register

Negative (N) : 0

Zero (Z) : 0

Carry (C) : 0

Overflow (V) : 0

IRQ Disable : 1

FIQ Disable : 1

Thumb (T) : 0

CPU Mode : System

0x000000df

addition.s

; Simple Code for Addition of 2 numbers.

00001000:E3A01003MOV R1, #3

00001004:E3A02004MOV R2, #4

00001008:E0813002ADD R3, R1, R2

MemoryView3

00001000

Word Size

8Bit16Bit32Bit

00001000	E3A01003	E3A02004	E0813002	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181
0000102C	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181
00001058	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181
00001084	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181	81818181

OutputView

ConsoleStdin/Stdout/Stderr

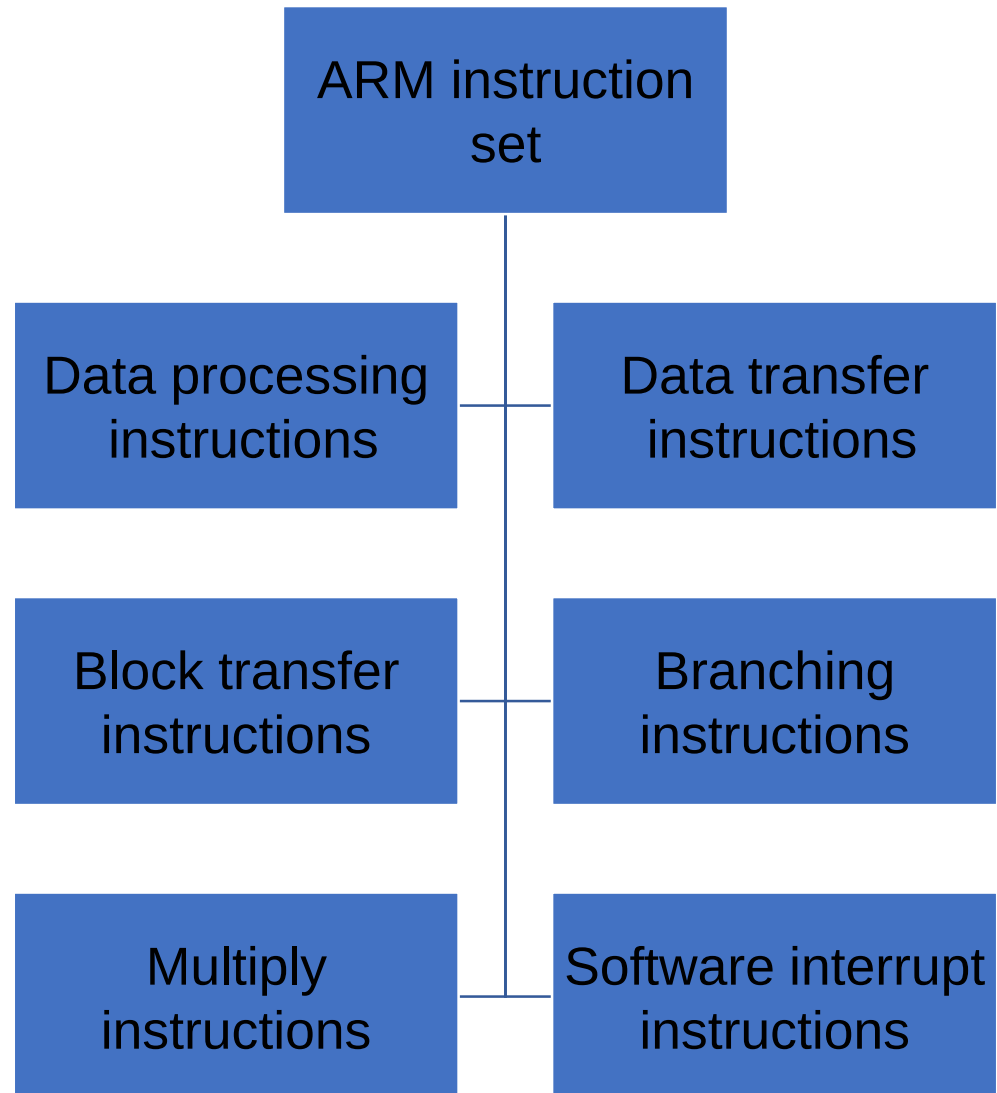
Loading assembly language file F:\academics\subjects\MPCA\MPCA 2024\Slides\Sample Programs\addition.s

Samples of following types of Numerical Data on Simulator

Demonstration of the following data is shown in the simulator.

- Decimal Numbers
- Hexadecimal Numbers
- Octal Numbers
- Usage of Bytes, Decimal and Hexadecimal data in the code and in the Simulator
- Read and observe the contents of the following in the simulator.
 - Memory
 - Stack
 - Registers
 - Flag Register
 - Console

ARM Instruction Set





THANK YOU



V R BADRI PRASAD

badriprasad@pes.edu

Department of Computer Science and Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 4

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Data Processing Instructions

ARM instructions are grouped as follows.

1. Data Processing Instructions.

Ex: MOV, ADD, SUB, CMP, CMN, etc.,

2. Branch Instructions.

Ex: B, BL, BGT, BLT, BGE, BLE, etc.,

3. Data Transfer Instructions.

Ex: LOAD – STORE Instructions- LDR, STR, LDM, STM, etc.,

4. Software Interrupt Instructions.

Ex: SWI 0x11, etc.,

5. Program Status Register Instructions.

Ex: MSR, MRS, etc.,

1. Data Processing Instructions.

MOVE instruction

Syntax: <mnemonic><cond><s> <dest Rd>, <src op1 Rn>

Ex1: **MOV** **R0 , R1**
 R0 = R1

*Moves the contents of the source operand registers **R1** and copies to the destination register **R0**.*

Ex2: **MVN** **R0, R1**
 R0 = negate(R1)

*Moves the complemented value of the source operand register **R1** and stores to the destination register **R0**.*

Arithmetic Instruction:

Syntax: **<mnemonic><cond><s> <dest Rd>, <src op1 Rn >, <src op2 Rm>**

Ex3: **ADD R3, R1, R2**
R3 = R1 + R2

Adds the contents of the source operand registers R1 & R2 and stores the result in the destination register R3.

Compare Instruction:

Syntax:

<mnemonic><cond><s> <dest Rd>, <src op1 Rn > ,
Ex4: **CMP R3, R1**

Neither source nor destination fields are affected.

Ex4: **CMN R3, R1**

Performs the **operation R3 + R1** and **sets** the condition codes / flags.

Neither source nor destination fields are affected.

- Any data processing instruction can set the condition codes if the programmers wish it to

64-bit addition

ADD_S R2, R2, R0

ADC R3, R3, R1

Comparison

compare

- **CMP** R1, R2 @ set cc on R1-R2

compare negated

- **CMN** R1, R2 @ set cc on R1+R2

bit test

- **TST** R1, R2 @ set cc on R1 and R2

test equal

- **TEQ** R1, R2 @ set cc on R1 xor R2

Arithmetic Operations

- MOV R4, #10
- MVN R5, R4
- MOV R1, #5
- MOV R2, #2

- ADD r0, r1, r2 ; r0:=r1+r2
- ADC r0, r1, r2 ; r0:=r1+r2+C
- SUB r0, r1, r2 ; r0:=r1-r2
- SBC r0, r1, r2 ; r0:=r1-r2+C-1
- RSB r0, r1, r2 ; r0:=r2-r1, **Reverse Subtraction**
- RSC r0, r1, r2 ; r0:=r2-r1+C-1

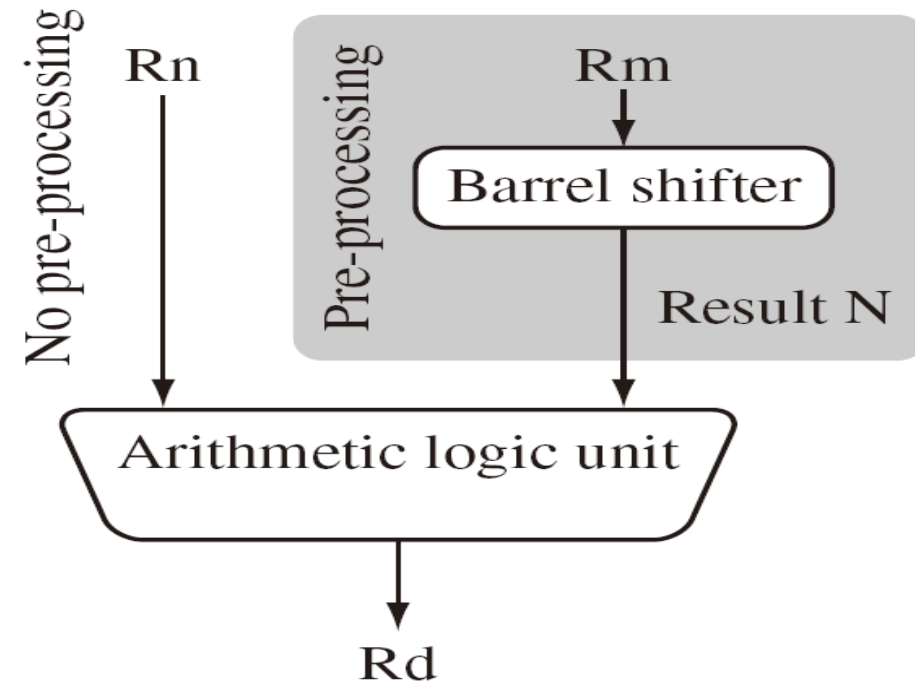
By default data processing operations do not affect the condition flags

Bit-wise Logical Operations

- **AND** r0, r1, r2 ;r0 := r1 **AND** r2
- **ORR** r0, r1, r2 ;r0 := r1 **OR** r2
- **EOR** r0, r1, r2 ;r0 := r1 **XOR** r2
- **BIC** r0, r1, r2 ;r0 := r1 **AND** (**NOT** r2), bit clear

Shifted register operands

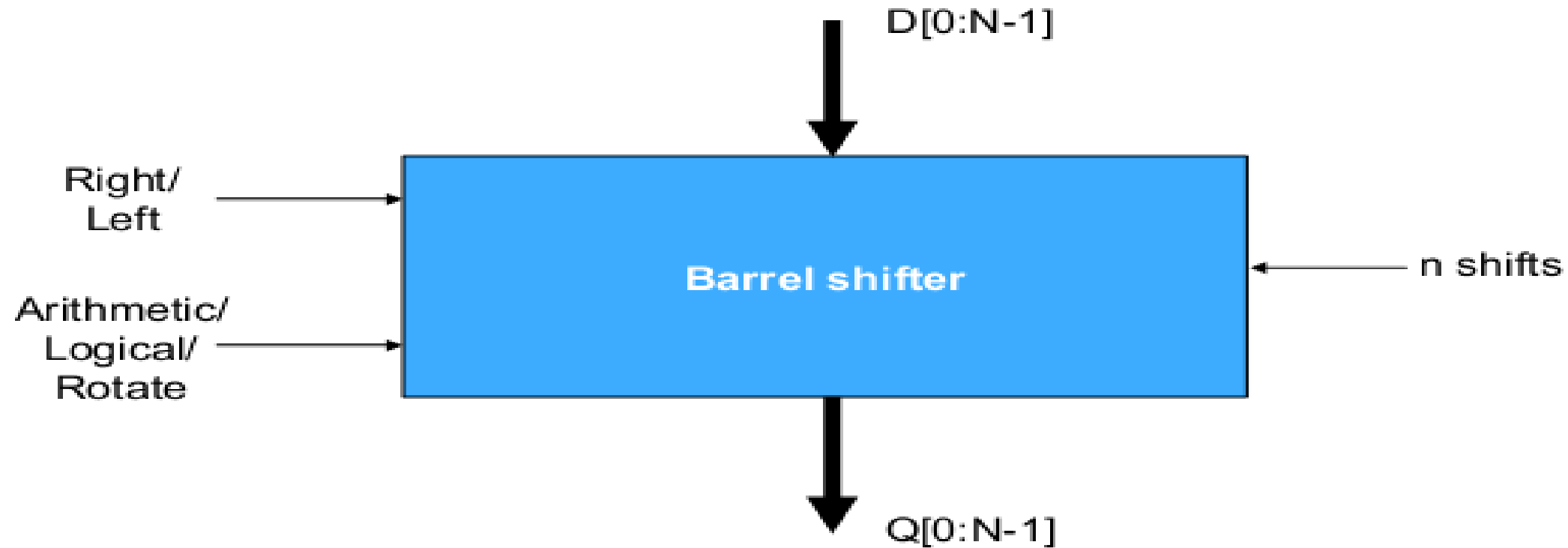
- One operand to ALU is routed through the **Barrel shifter**. Thus, the operand can be modified before it is used. Useful for fast multiplication and dealing with lists, table and other complex data structure. (similar to the displacement addressing mode in CISC.)



- Some instructions (e.g. **MUL**, **CLZ**) do not read barrel shifter.

BARREL SHIFTER - BLOCK DIAGRAM

Barrel shifter



BARREL SHIFTER

- ARM architecture supports instructions which performs a **shift with ALU** operation.
- The **shifter performance** is therefore **critical** as it contributes directly to the **datapath cycle time**.
- To **minimize** the delay through the **shifter**, A **cross-bar switch matrix** is used to steer each input to appropriate output.
- Example shown is **4 x 4 matrix cross bar** switch.
- ARM uses a **32 x 32 matrix cross bar** switch.
- Each input is connected to each output through a switch.

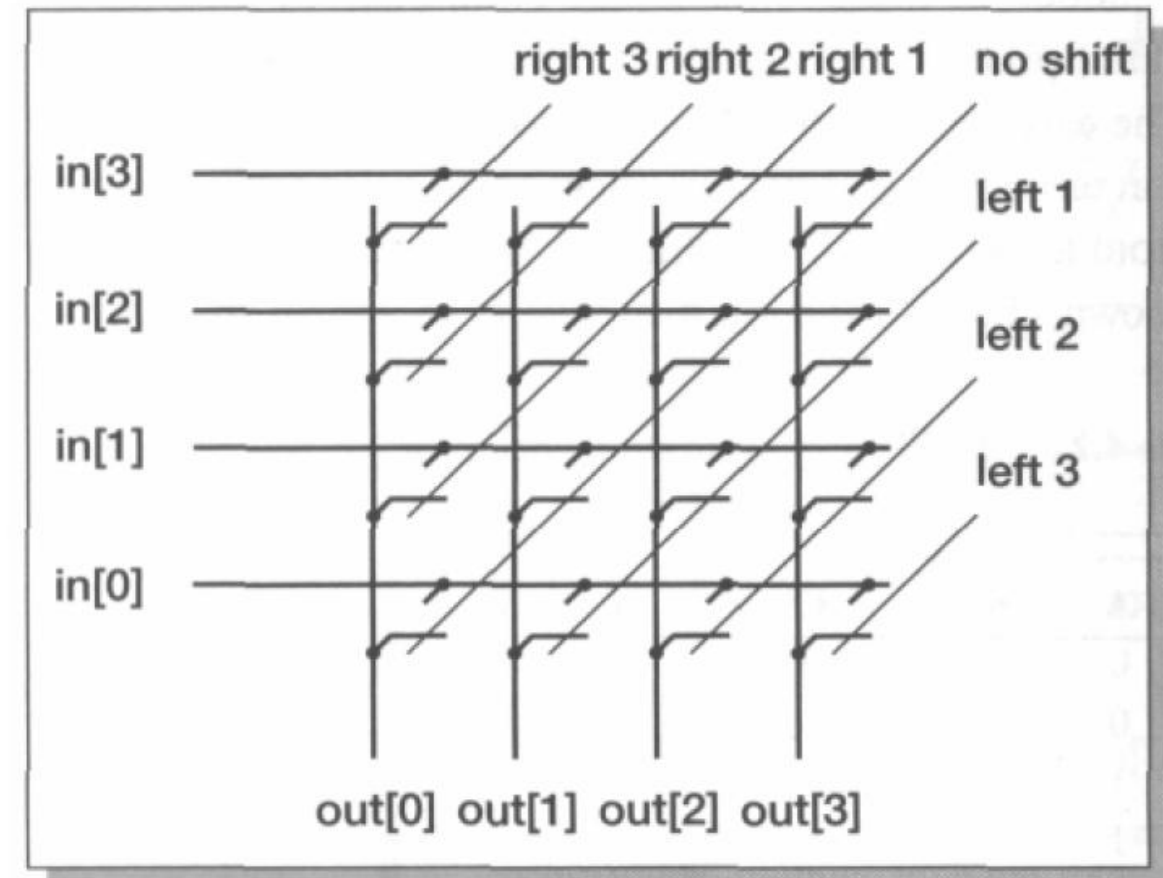


Figure 4.15 The cross-bar switch barrel shifter principle.

BARREL SHIFTER - FUNCTION

- Shift Operation – Left
 - One left shift diagonal line is turned on.
- Shift Operation – Right
 - One right shift diagonal line is turned on.
- Arithmetic Shift right
 - Uses sign extension rather than zero filling for unconnected output.
- Rotate operation
 - The right shift diagonal is enabled together with complementary left shift diagonal.

Mnemonic	Description	Shift	Result
LSL	logical shift left	$x\text{LSL } y$	$x \ll y$
LSR	logical shift right	$x\text{LSR } y$	$(\text{unsigned})x \gg y$
ASR	arithmetic right shift	$x\text{ASR } y$	$(\text{signed})x \gg y$
ROR	rotate right	$x\text{ROR } y$	$((\text{unsigned})x \gg y) (x \ll (32 - y))$
RRX	rotate right extended	$x\text{RRX}$	$(c \text{ flag} \ll 31) ((\text{unsigned})x \gg 1)$

MULTIPLICATION BY CONSTANT USING THE BARREL SHIFTER

- **Multiplication by 2^n (1,2,4,8,16,32..)**

MOV Ra, Rb, LSL #n

- **Multiplication by 2^{n+1} (3,5,9,17..)**

ADD Ra,Ra,Ra,LSL #n

- **Multiplication by 2^{n-1} (3,7,15..)**

RSB Ra,Ra,Ra,LSL #n

- **Multiplication by 6**

ADD Ra,Ra,Ra,LSL #1 ; multiply by 3

MOV Ra,Ra,LSL#1 ; and then by 2

- **Multiply by 10 and add in extra number**

ADD Ra,Ra,Ra,LSL#2 ; multiply by 5

ADD Ra,Rc,Ra,LSL#1 ; multiply by 2 and add in next digit

BARREL SHIFTER - EXAMPLES

- Shift Operation – Left

- One left shift diagonal line is turned on.

Example 1: ADD R0, R1, R2, LSL #3

- Since LSL is used, it selects left diagonal #3.

- Rotate operation

- The right shift diagonal is enabled together with complementary left shift diagonal.

Example 2: ADD R0, R1, R2, ROR #3

- Selects Right diagonal #3 and left diagonal # 29.

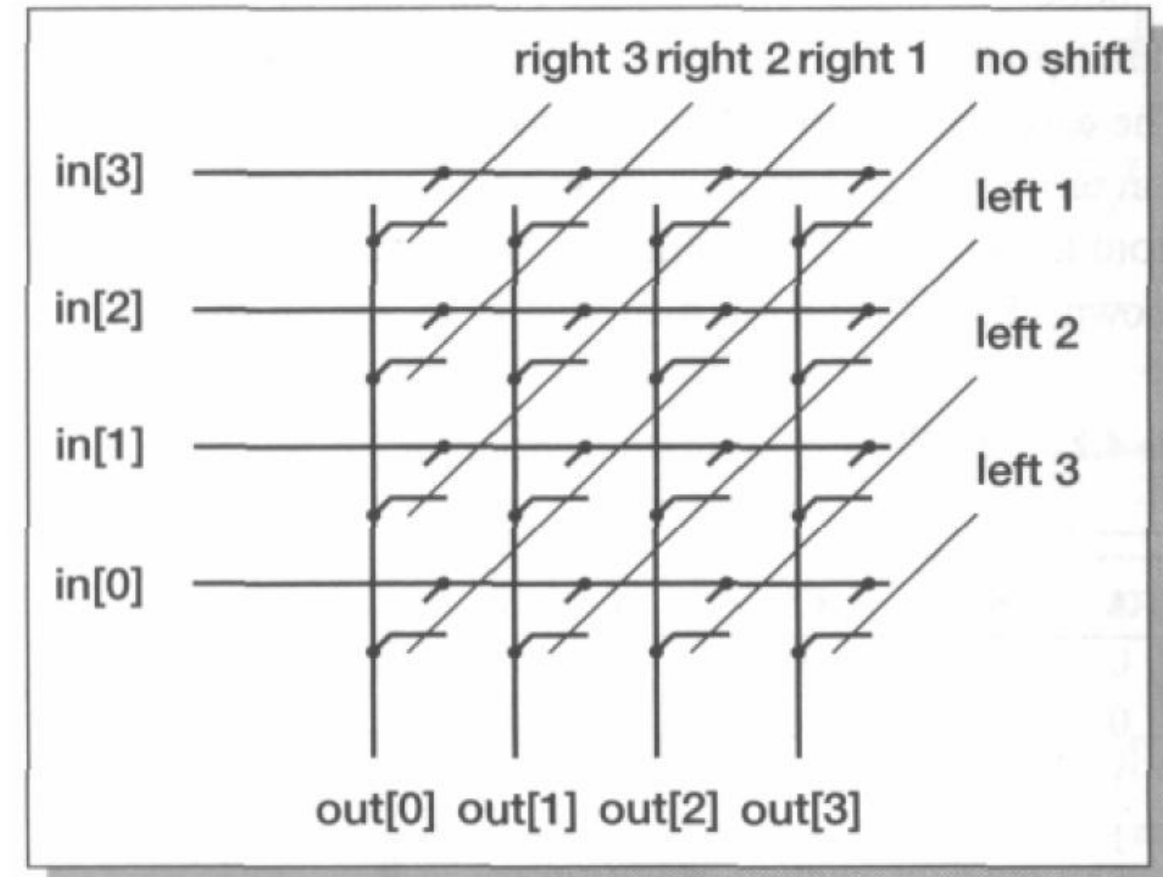


Figure 4.15 The cross-bar switch barrel shifter principle.

Logical shift left



```
MOV  R0, R2, LSL #2 @ R0 := R2 << 2
                        @ R2 unchanged
```

Example: 0...0 0011 0000

Before R2=0x00000030

After R0=0x000000C0

R2=0x00000030

Logical shift right



```
MOV  R0, R2, LSR #2 @ R0 := R2 >> 2
                        @ R2 unchanged
```

Example: 0...0 0011 0000

Before R2=0x00000030

After R0=0x0000000C

R2=0x00000030

Arithmetic shift right



MOV R0, R2, **ASR #2** @ R0 := R2 >> 2

@ R2 unchanged

Example: **1010 0...0 0011 0000**

Before R2 = 0xA0000030

After R0 = 0xE800000C

R2 = 0xA0000030

Rotate right



```
MOV  R0, R2, ROR #2 @ R0:=R2 rotate  
                        @ R2 unchanged
```

Example: 0...0 0011 0001

Before R2=0x00000031

After R0=0x4000000C

R2=0x00000031

Rotate right extended



`MOV R0, R2, RRX` @ R0:=R2 rotate
 @ R2 unchanged

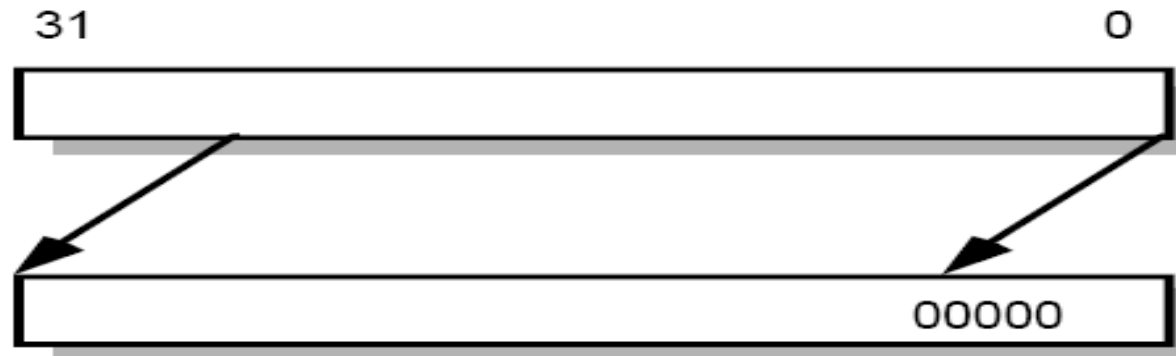
Example: **0...0 0011 0001**

Before R2=0x00000031, C=1

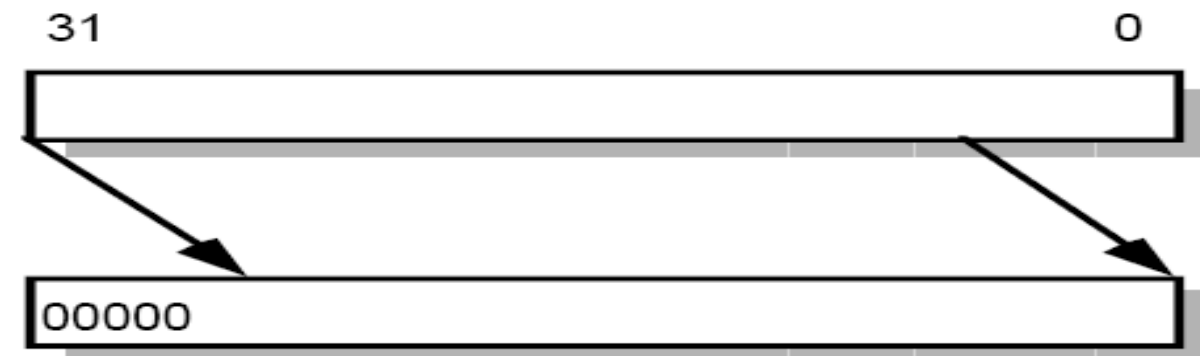
After R0=0x80000018, C=1

R2=0x00000031

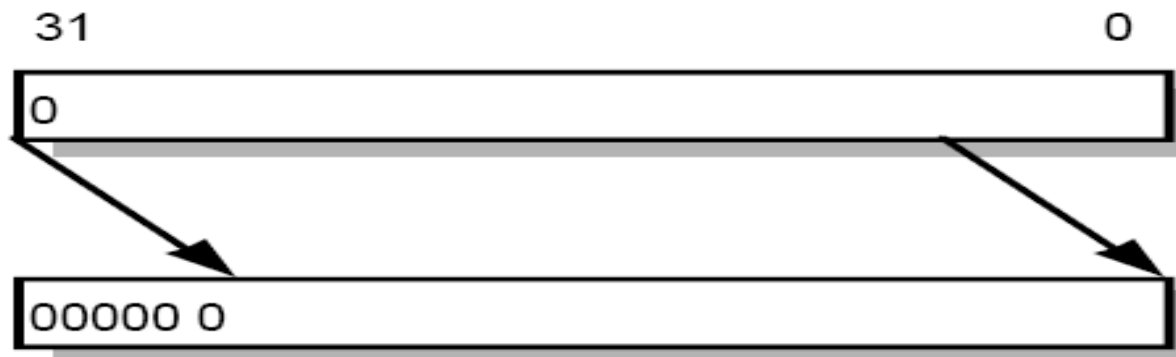
Shifted register operands



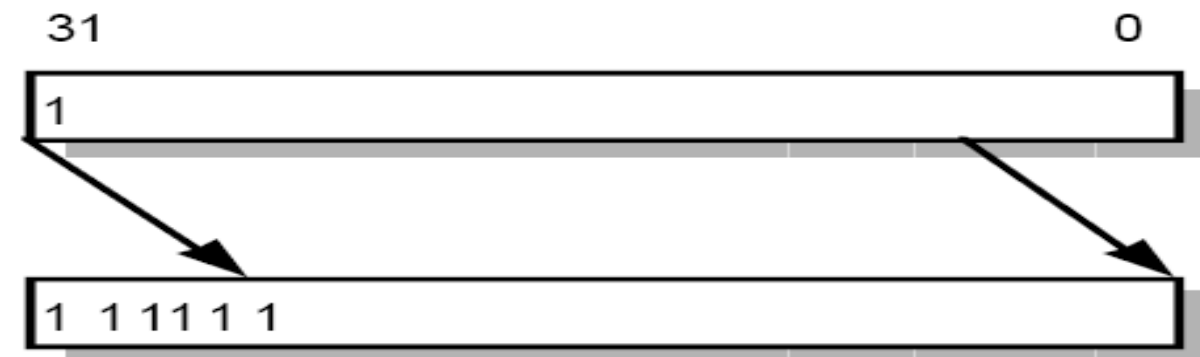
LSL #5



LSR #5

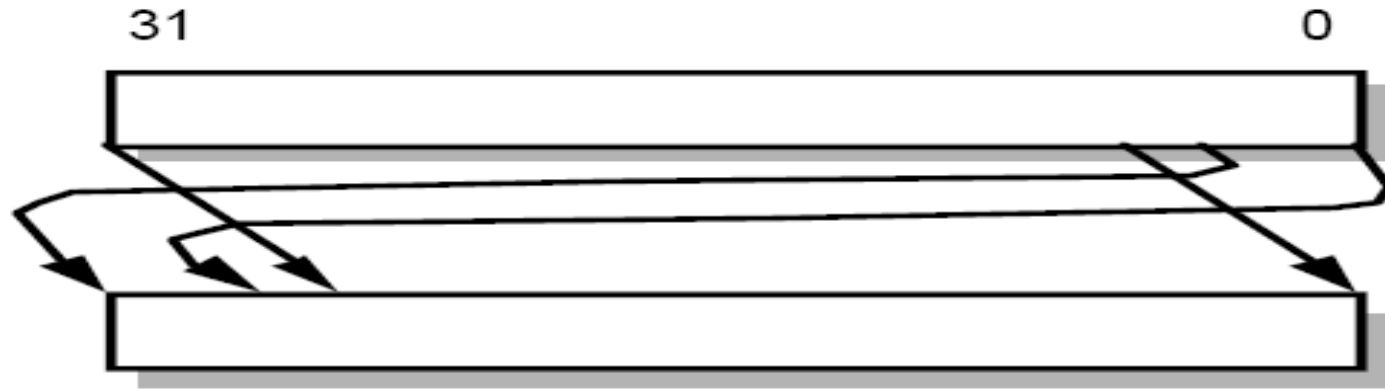


ASR #5 , positive operand

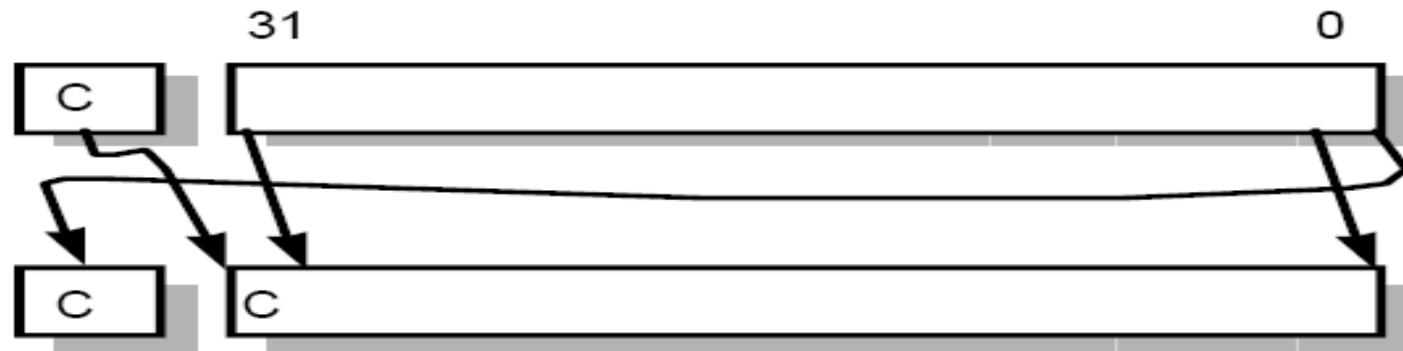


ASR #5 , negative operand

Shifted register operands



ROR #5



RRX

Shifted register operands

- It is possible to use a register to specify the number of bits to be shifted; only the bottom 8 bits of the register are significant.

@ array index calculation

ADD R0, R1, R2, LSL R3 @ $R0 := R1 + R2 * 2^{R3}$

@ fast multiply $R2 = 35 \times R0$

ADD R0, R0, R0, LSL #2 @ $R0' = 5 \times R0$

RSB R2, R0, R0, LSL #3 @ $R2 = 7 \times R0'$

Shifted register operands

<i>N</i> shift operations	Syntax
Immediate	<code>#immediate</code>
Register	<code>Rm</code>
Logical shift left by immediate	<code>Rm, LSL #shift_imm</code>
Logical shift left by register	<code>Rm, LSL Rs</code>
Logical shift right by immediate	<code>Rm, LSR #shift_imm</code>
Logical shift right with register	<code>Rm, LSR Rs</code>
Arithmetic shift right by immediate	<code>Rm, ASR #shift_imm</code>
Arithmetic shift right by register	<code>Rm, ASR Rs</code>
Rotate right by immediate	<code>Rm, ROR #shift_imm</code>
Rotate right by register	<code>Rm, ROR Rs</code>
Rotate right with extend	<code>Rm, RRX</code>

Multiplication as Addition & Subtraction

MOV R1, #35

MUL R2, R0, R1

or

ADD R0, R0, R0, LSL #2 @ $R0' = 5 \times R0$

RSB R2, R0, R0, LSL #3 @ $R2 = 7 \times R0'$

1. Which of the following best describes the purpose of data processing instructions in the ARM instruction set?

- A. They control program flow using branch operations
- B. They perform arithmetic, logical, and comparison operations on register data
- C. They transfer data directly between memory locations
- D. They manage processor modes and interrupts

2. Which of the following statements about comparison instructions in ARM is correct?

- A. They store the result of comparison in a destination register
- B. They modify only the source registers
- C. They update condition flags without storing the result
- D. They perform memory-to-memory comparison

3. Which instruction uses the barrel shifter to multiply a register value by 2^n in a single ARM instruction?

- A. ADD Ra, Ra, #n
- B. MOV Ra, Rb, LSL #n
- C. MUL Ra, Rb, #n
- D. RSB Ra, Rb, #n

4. Which shift operation in ARM preserves the sign bit while shifting bits to the right?

- A. Logical Shift Left (LSL)
- B. Logical Shift Right (LSR)
- C. Arithmetic Shift Right (ASR)
- D. Rotate Right (ROR)

1. Which of the following best describes the purpose of data processing instructions in the ARM instruction set?

- A. They control program flow using branch operations
- B. They perform arithmetic, logical, and comparison operations on register data
- C. They transfer data directly between memory locations
- D. They manage processor modes and interrupts

Answer - They perform arithmetic, logical, and comparison operations on register data

2. Which of the following statements about comparison instructions in ARM is correct?

- A. They store the result of comparison in a destination register
- B. They modify only the source registers
- C. They update condition flags without storing the result
- D. They perform memory-to-memory comparison

Answer - They update condition flags without storing the result

3. Which instruction uses the barrel shifter to multiply a register value by 2^n in a single ARM instruction?

- A. ADD Ra, Ra, #n
- B. MOV Ra, Rb, LSL #n
- C. MUL Ra, Rb, #n
- D. RSB Ra, Rb, #n

Answer - MOV Ra, Rb, LSL #n

4. Which shift operation in ARM preserves the sign bit while shifting bits to the right?

- A. Logical Shift Left (LSL)
- B. Logical Shift Right (LSR)
- C. Arithmetic Shift Right (ASR)
- D. Rotate Right (ROR)

Answer - Arithmetic Shift Right (ASR)



Arm Instruction Set – data processing group

Using the ARM instruction set, write Assembly level language instructions / sequence / programs to perform the following as directed.

1. Compile the following C assignment statements into ARM.
 - i. $A = B + C;$
 - ii. $D = A - C;$
 - iii. Perform $D = A - C$ using RSB instruction.
2. Convert the following assignment into ARM.
 - i. $F = (G + H) - (I + J).$
use the register **R0 to R4** as operands **F to J** respectively.
3. Convert the statement into ARM where the operand is in memory.
 - i. $G = H + A[10].$

Assignment.

Note: Use different addressing modes both at source & destination operand positions.

THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 5A

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Additional Programs

Program-1 : Convert a given 2 digit hexadecimal number to ASCII CODE and store the result in the memory location X and X+1 using ARM7TDMI – ISA.

Example: $(54)_{10}$ to ASCII code :

1. Mask the LS byte.
2. 54 and 0xF0 gives 50.
3. shift 50 right by 4 times
4. 50 will be 05.
5. Now, 05 && 0x30
6. Results in 0x35.
7. Repeat the same for the LS byte.
8. Mask the MS byte.
9. 54 and 0x0F gives 04.
10. Now, 04 && 0x30
11. Results in 0x34.

;ARM7TDMI – ISA code:

.TEXT

LDR R3, =X

MOV R1, #0x54

AND R0, R1, #0xF0

MOV R0, R0, LSR #4

ORR R0, R0, #0x30

STR R0, [R3]

AND R2, R1, #0x0F

ORR R2, R2, #0x30

ADD R3, R3, #4

STR R2, [R3]

.DATA

X: .WORD 0,0

Program - 2: Write a program using ARM7TDMI – ISA to find the largest 32 bit element stored in an array A.

.DATA

A: .WORD 10,20,30,40,50

.TEXT

LDR R1,=A

LDR R0, [R1] ; Initially , set first element as the largest element (stored in register R0).

MOV R3, #0 ; Count Register

LOOP: LDR R2, [R1, #4]! ; load the next element

CMP R0, R2 ; compare

BLT Less ; if the previous element is less than next element then store the largest element in register R0.

L1: ADD R3, R3, #1

CMP R3, #4

BNE LOOP

B EXIT

Less: MOV R0, R2

B L1

EXIT: SWI 0X011

Program - 3: Convert the following C code to ALP using ARM7TDMI – ISA

C statement: **while(save[i] == k)**
 i+=1;

.TEXT

LDR R1, =K ; ADDRESS K
LDR R0, [R1] ; VALUE of K
LDR R2, =SAVE ; SAVE is the starting address of the array
MOV R3, #0 ; VALUE of I

LOOP: ADD R5, R2, R3, LSL #2 ; R5 = SAVE+I*4
LDR R6, [R5,#0] ; R6 = [R5]

CMP R0, R6
BNE EXIT ; REPEAT if save[I]==K

ADD R3, R3, #1
CMP R3, #5
BNE LOOP

EXIT: SWI 0X011

.DATA

K: .WORD 5

I: .WORD 0

SAVE: .WORD 5, 5, 5, 50, 35, 40

Program – 4: Consider the following c code snippet. Write the compiled ARM assembly code. Use ARM7TDMI – ISA.

```
int example( int g, int h, int l, int j)
{ int f;
  f= (g+h)-(i+j);
  return f;
}
```

```
.TEXT
LDR R4,=F
MOV R0, #5 ;G
MOV R1, #10 ;H
MOV R2, #1 ;l
MOV R3, #4 ;J
```

```
BL example
STR R0, [R4]
```

```
SWI 0X011
```

example:

```
SUB R13, R13, #12
STR R6, [R13,#8]
STR R5, [R13,#4]
STR R4, [R13,#0]
```

```
ADD R5, R0, R1 ; G+H
ADD R6, R2, R3 ; l+j
SUB R4, R5, R6 ;F= (G+H)-(l+J)
```

```
MOV R0, R4 ;RETURNS F THROUGH R0.
```

```
LDR R4,[R13,#0]
LDR R5,[R13,#4]
LDR R6,[R13,#8]
```

```
ADD R13, R13, #12
MOV PC, LR
```

```
.DATA
F: .WORD 0
```

Program – 5: Consider the following c code snippet. Write the compiled ARM assembly code. Use ARM7TDMI – ISA. ; Note: Pass parameters to function on stack.

```
int example( int g, int h, int l, int j)
{ int f;
  f= (g+h)-(i+j);
  return f;
}
```

```
.TEXT
LDR R4,=F
MOV R0, #5 ;G
MOV R1, #10 ;H
MOV R2, #1 ;I
MOV R3, #4 ;J
; PASS PARAMETRS TO STACK
SUB R13, R13, #16
STR R0, [R13,#12]
STR R1, [R13,#8]
STR R2, [R13,#4]
STR R3, [R13,#0]
BL example
STR R0, [R4]
SWI 0X011
```

example:

```
LDR R5,[R13,#0] ; J
LDR R6,[R13,#4] ; I
LDR R7,[R13,#8] ; H
LDR R8,[R13,#12] ; G
```

```
ADD R9, R7, R8 ; G+H
ADD R10, R5, R6 ; I+j
SUB R11, R9, R10 ;F= (G+H)-(I+J)
```

```
MOV R0, R11 ;RETURNS F THROUGH R0.
```

```
ADD R13, R13, #16
MOV PC, LR
```

```
.DATA
F: .WORD 0
```

1. Which instruction is used to mask the higher nibble of a byte in ARM?
 - A) ORR
 - B) EOR
 - C) AND
 - D) BIC

2. Which instruction converts a 4-bit hex digit into ASCII in Program-1?
 - A) AND
 - B) ADD
 - C) ORR
 - D) EOR

1. Which instruction is used to mask the higher nibble of a byte in ARM?
 - A) ORR
 - B) EOR
 - C) AND**
 - D) BIC

2. Which instruction converts a 4-bit hex digit into ASCII in Program-1?
 - A) AND
 - B) ADD**
 - C) ORR
 - D) EOR

3. What is the purpose of the instruction MOV PC, LR at the end of the example function?

- A) To copy the Program Counter to the Link Register.
- B) To restart the function from the beginning.
- C) To return to the calling instruction by restoring the return address into the PC.
- D) To exit the program and trigger a software interrupt.

4. How many bytes does .WORD reserve?

- A) 1
- B) 2
- C) 4
- D) 8

3. What is the purpose of the instruction MOV PC, LR at the end of the example function?

A) To copy the Program Counter to the Link Register.

B) To restart the function from the beginning.

C) To return to the calling instruction by restoring the return address into the PC.

D) To exit the program and trigger a software interrupt.

4. How many bytes does .WORD reserve?

A) 1

B) 2

C) 4

D) 8

THANK YOU



Team MPCA

Department of Computer Science and Engineering

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Session - 5



Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

**Branch Instructions &
Addressing Modes**

Branch / Flow Control Instructions

Syntax: B{<cond>} Label
BL{<cond>} Label
BX{<cond>} Rm
BLX{<cond>} Rm

B	Branch	Program Counter = Label
BL	Branch & Link	Step1: PC will be copied to R14 the Link Register (LR) before branch is taken. Step2: Program Counter = Label
BX	Branch Exchange	Used for changing ARM to Thumb mode or from Thumb mode to ARM mode.
BLX	Branch Exchange with link	

- Branch Instruction- (Unconditional)

```
B    label
```

```
...
```

```
label:  ...
```

- Conditional Branch Instruction

```
MOV    R0, #0
```

```
loop:  ...
```

```
ADD    R0, R0, #1
```

```
CMP    R0, #10
```

```
BNE   loop
```

- **Exercise-1:** Program to find the sum of N NUMBERS (use conditional branch).
- Store the result in memory location.

Program:

```
.DATA
    A: .WORD 10,20,30,40,50,60,70,80,90,100
    SUM: .WORD 0
.TEXT
LDR R1,=A
LDR R2,=SUM
MOV R4,#0    ; INITIALISATION
MOV R5,#1    ; COUNT register

L1:  LDR R3, [R1]
     ADD R4,R4,R3    ; Add next element in the array.
     ADD R1, R1, #4  ; address to the next data
     ADD R5, R5, #1  ; increment the count register
     CMP R5, #11     ; Check whether all numbers are added
     BNE L1          ; Else branch to L1 location
     STR R4,[R2]      ; store the result in location SUM.
     SWI 0X011        ; logical end of the program.
```

CONDITIONAL BRANCH OPTIONS

Branch	Interpretation	Normal uses
B BAL	Unconditional Always	Always take this branch Always take this branch
BEQ	Equal	Comparison equal or zero result
BNE	Not equal	Comparison not equal or non-zero result
BPL	Plus	Result positive or zero
BMI	Minus	Result minus or negative
BCC BLO	Carry clear Lower	Arithmetic operation did not give carry-out Unsigned comparison gave lower
BCS BHS	Carry set Higher or same	Arithmetic operation gave carry-out Unsigned comparison gave higher or same
BVC	Overflow clear	Signed integer operation; no overflow occurred
BVS	Overflow set	Signed integer operation; overflow occurred
BGT	Greater than	Signed integer comparison gave greater than
BGE	Greater or equal	Signed integer comparison gave greater or equal
BLT	Less than	Signed integer comparison gave less than
BLE	Less or equal	Signed integer comparison gave less than or equal
BHI	Higher	Unsigned comparison gave higher
BLS	Lower or same	Unsigned comparison gave lower or same

ADDRESSING HALF WORDS

Program to find the sum of N numbers using **half word**

.DATA

A: **.HWORD** 0x10,0x20,0x30,0x40,0x50,0x60,0x70,
0x80,0x90, 0x0100

SUM: .WORD 00

.TEXT

LDR R1,=A

LDR R2,=SUM

MOV R4,#0 ;INITIALISATION

MOV R5,#1 ;COUNT

L1: **LDRH R3,[R1]**

ADD R4,R4,R3

ADD R1,R1,#2

ADD R5,R5,#1

CMP R5, #11

BNE L1

STRH R4, [R2]

SWI 0X011

Addressing Byte Data

Program to find the sum of N numbers using **Byte Data**

; SUM OF N NUMBERS

; DATA GIVEN

.DATA

A: **.BYTE** 1,2,3,4,5,6,7,8,9,10

SUM: .word 0

.TEXT

LDR R1,=A

LDR R2,=SUM

MOV R4,#0 ;INITIALISATION

MOV R5,#1 ;COUNT

L1: **LDRB R3,[R1]**

ADD R4,R4,R3

ADD R1,R1,#1

ADD R5,R5,#1

CMP R5, #11

BNE L1

STRB R4,[R2]

SWI 0X011

Addressing memory locations

- Memory is addressed by a **register** and an **offset**.

```
LDR R0, [R1] @ mem[R1]
```

- Three ways to specify offsets:

- Immediate

```
LDR R0, [R1, #4] @ mem[R1+4]
```

- Register

```
LDR R0, [R1, R2] @ mem[R1+R2]
```

- Scaled Register

```
LDR R0, [R1, R2, LSL #2] @ mem[R1+4*R2]
```

Addressing Modes of ARM Architecture

- 1. **Preindexing or Preindexing without writeback**
- 1. **Preindexing with Writeback or Autoindexing**
- 1. **Post indexing**

Syntax of Addressing Modes used in ARM Architecture

- Pre Indexing Without Write Back

Syntax: **LDR Rd, [Rn, OFFSET]**

- Pre Indexing With Write Back

Syntax: **LDR Rd, [Rn, OFFSET]!**

- Post Indexing

Syntax: **LDR Rd, [Rn] ,OFFSET**

Operand specified in register

- Pre-indexed addressing

```
LDR  R0, [R1, R2]  @ R0=mem[R1+R2]  
                        @ R1 unchanged
```

- Auto-indexing addressing

```
LDR  R0, [R1, R2]! @ R0=mem[R1+R2]  
                        @ R1=R1+R2
```

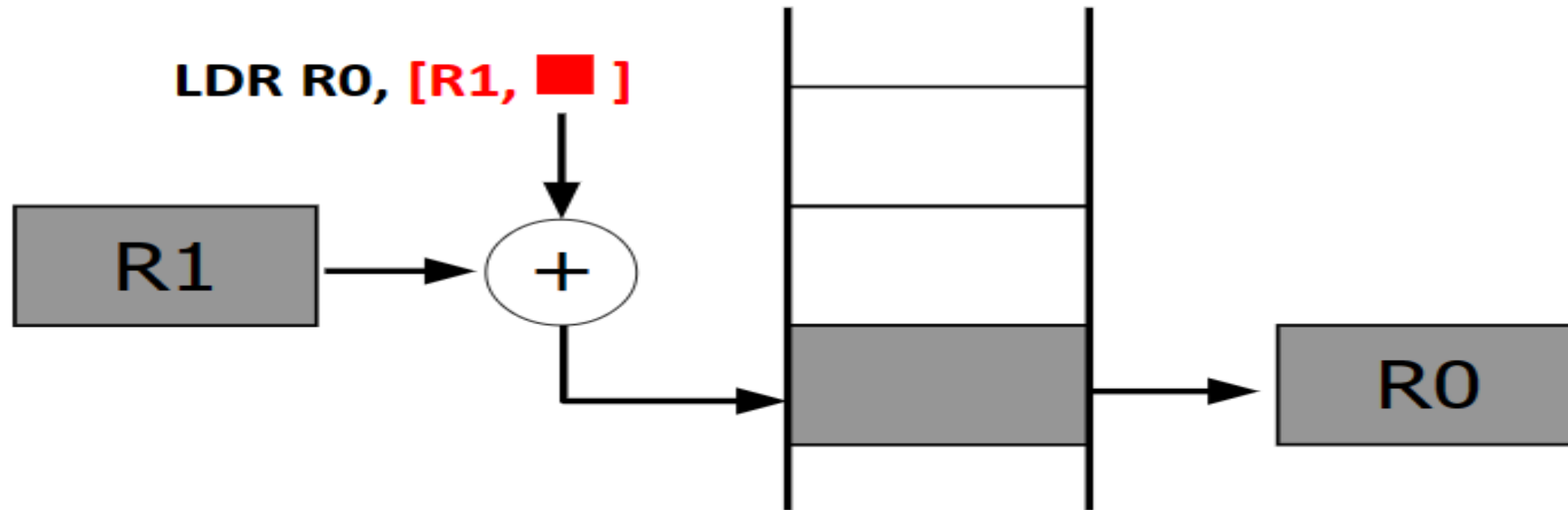
- Post-indexed addressing

```
LDR  R0, [R1], R2  @ R0=mem[R1]  
                        @ R1=R1+R2
```

Will improve the coding style.

Pre-index addressing

LDR R0, [R1, #4] @ R0=mem[R1+4]
@ R1 unchanged



Example – Using preindexing addressing mode

- Program to find the sum of N NUMBERS
- Store the result in memory location.

Program:

.DATA

A: .WORD 10,20,30,40,50,60,70,80,90,100

SUM: .WORD 0

.TEXT

LDR R1,=A

LDR R2,=SUM

MOV R4,#0 ; INITIALISATION

MOV R5,#1 ; COUNT register

Has no impact on the preindexing mode....

LDR R3,[R1]

ADD R4,R4,R3

L1: **LDR R3, [R1, #4]**

ADD R4,R4,R3 ; Add next element in the array.

ADD R1,R1,#4 ; useless

ADD R5, R5, #1 ; increment the count register

CMP R5, #10 ; Check whether all numbers are added

BNE L1 ; Else branch to L1 location

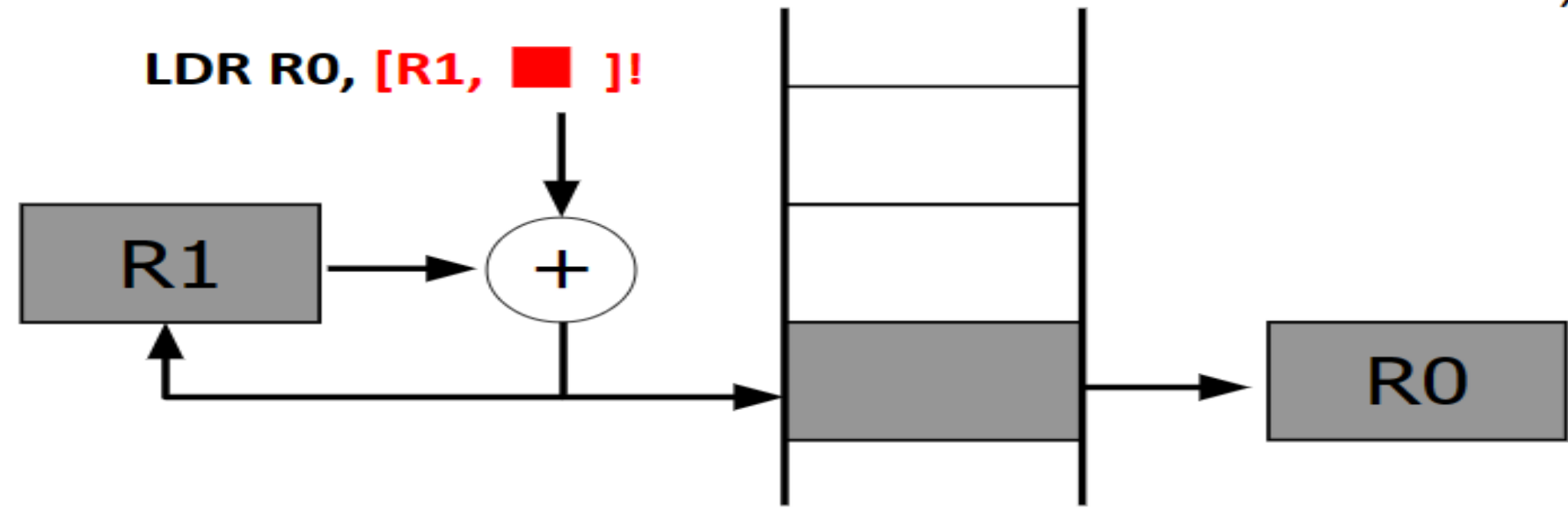
STR R4,[R2] ; store the result in location SUM.

SWI 0X011 ; logical end of the program.

Auto-indexing addressing or Preindexing with writeback

LDR R0, [R1, #4]! @ R0=mem[R1+4]
 @ R1=R1+4

No extra time; Fast;



Example – Using preindexing with writeback or autoindexing addressing mode

- Program to find the sum of N NUMBERS
- Store the result in memory location.

Program:

.DATA

A: .WORD 10,20,30,40,50,60,70,80,90,100

SUM: .WORD 0

.TEXT

LDR R1,=A

LDR R2,=SUM

MOV R4,#0 ; INITIALISATION

MOV R5,#1 ; COUNT register

LDR R3,[R1]

ADD R4,R4,R3

L1: **LDR R3, [R1,#4]!**

ADD R4,R4,R3 ; Add next element in the array.

ADD R5, R5, #1 ; increment the count register

CMP R5, #10 ; Check whether all numbers are added

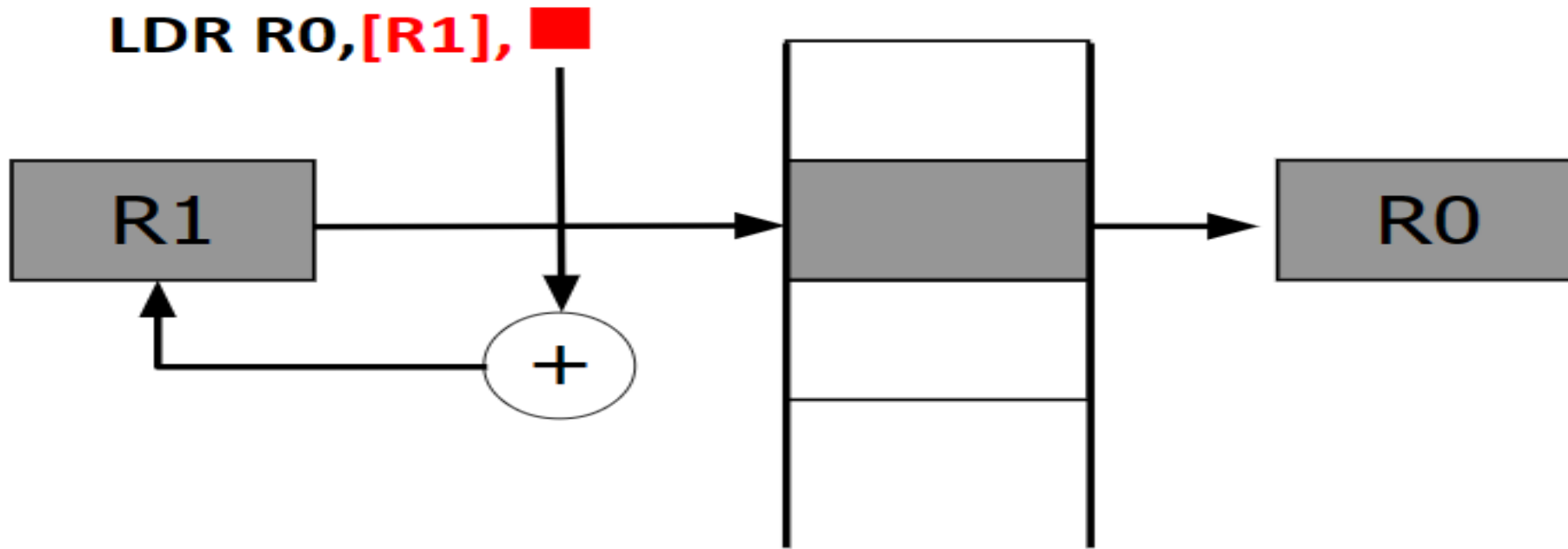
BNE L1 ; Else branch to L1 location

STR R4,[R2] ; store the result in location SUM.

SWI 0X011 ; logical end of the program.

Post-index addressing

LDR R0, [R1], #4 @ R0=mem[R1]
 @ R1=R1+4



Example – Using postindexing Addressing Mode

- Program to find the sum of N NUMBERS and Store the result in memory location.

Program:

.DATA

A: .WORD 10,20,30,40,50,60,70,80,90,100

SUM: .WORD 0

.TEXT

LDR R1,=A

LDR R2,=SUM

MOV R4,#0 ; INITIALISATION

MOV R5,#1 ; COUNT register

L1: **LDR R3, [R1], #4**

ADD R4,R4,R3

ADD R5,R5,#1

CMP R5, #11

BNE L1

STR R4,[R2]

SWI 0X011

1. Which ARM instruction is specifically used to branch to a subroutine and save the return address?

- A. B
- B. BX
- C. BL
- D. BLX

2. In the instruction `LDR R0, [R1, R2, LSL #2]`, what does the effective memory address correspond to?

- A. $R1 + R2$
- B. $R1 + (R2 \ll 2)$
- C. $R2 + (R1 \ll 2)$
- D. $R1 \ll (R2 + 2)$

1. Which ARM instruction is specifically used to branch to a subroutine and save the return address?

- A. B
- B. BX
- C. BL
- D. BLX

Answer - BL

2. In the instruction LDR R0, [R1, R2, LSL #2], what does the effective memory address correspond to?

- A. $R1 + R2$
- B. $R1 + (R2 \ll 2)$
- C. $R2 + (R1 \ll 2)$
- D. $R1 \ll (R2 + 2)$

Answer - $R1 + (R2 \ll 2)$

3. Which addressing mode updates the base register after the memory access is completed?

- A. Pre-indexing without writeback
- B. Pre-indexing with writeback (auto-indexing)
- C. Post-indexing
- D. Register-offset addressing

4. Which instruction is most appropriate for loading byte-sized data from memory in ARM?

- A. LDR
- B. LDRH
- C. LDRB
- D. STRB

3. Which addressing mode updates the base register after the memory access is completed?

- A. Pre-indexing without writeback
- B. Pre-indexing with writeback (auto-indexing)
- C. Post-indexing
- D. Register-offset addressing

Answer - Post-indexing

4. Which instruction is most appropriate for loading byte-sized data from memory in ARM?

- A. LDR
- B. LDRH
- C. LDRB
- D. STRB

Answer - LDRB

THANK YOU



Team MPCA

Department of Computer Science and Engineering





Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 6.1

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Multiplication Instruction

Multiplication

Syntax: $\text{MLA}\{\text{<cond>}\}\{\text{S}\} \text{ Rd}, \text{ Rm}, \text{ Rs}, \text{ Rn}$
 $\text{MUL}\{\text{<cond>}\}\{\text{S}\} \text{ Rd}, \text{ Rm}, \text{ Rs}$

MLA	multiply and accumulate	$Rd = (Rm * Rs) + Rn$
MUL	multiply	$Rd = Rm * Rs$

Syntax: $\text{<instruction>}\{\text{<cond>}\}\{\text{S}\} \text{ RdLo}, \text{ RdHi}, \text{ Rm}, \text{ Rs}$

SMLAL	signed multiply accumulate long	$[RdHi, RdLo] = [RdHi, RdLo] + (Rm * Rs)$
SMULL	signed multiply long	$[RdHi, RdLo] = Rm * Rs$
UMLAL	unsigned multiply accumulate long	$[RdHi, RdLo] = [RdHi, RdLo] + (Rm * Rs)$
UMULL	unsigned multiply long	$[RdHi, RdLo] = Rm * Rs$

- **MUL R0, R1, R2**
 - @ $R0 = (R1 \times R2)_{[31:0]}$
- Features:
 - Second operand can't be immediate
 - The result register must be different from the first operand
 - Cycles depends on core type
 - If S bit is set, C flag is meaningless

Multiplication Instruction

- There are 2 classes of multiply - producing 32-bit and 64-bit results
- **32-bit versions on an ARM7TDMI will execute in 2 - 5 cycles**

- `MUL r0, r1, r2 ; r0 = r1 * r2`
- `MLA r0, r1, r2, r3 ; r0 = (r1 * r2) + r3`

- **64-bit multiply instructions offer both signed and unsigned versions**

- For these instruction there are 2 destination registers

- `[U|S]MULL r4, r5, r2, r3 ; r5:r4 = r2 * r3`
- `[U|S]MLAL r4, r5, r2, r3 ; r5:r4 = (r2 * r3) + r5:r4`

Multiplication of Corresponding elements in an array

.DATA	Loop:	
A: .word 1,3,5		LDR R1, [R5], #4
B: .word 2,4,6		LDR R2, [R6], #4
C: .word		MUL R0, R1, R2 ;multiply R1 And R2 And Store Result In R0
.TEXT		ADD R4, R4, R0
LDR R5,=A		STR R4, [R7], #4
LDR R6,=B		SUB R3, R3, #1
LDR R7,=C		TEQ R3, #0 ; Test For Equality
MOV R3, #3 ; number of elements in array		BNE Loop
MOV R4, #0		SWI 0x11 ; Interrupt For Termination Of Program

$$C = C + A[i] * B[i]$$

```

        .TEXT
00001000:E59F5030    LDR R5,=A
00001004:E59F6030    LDR R6,=B
00001008:E59F7030    LDR R7,=C
0000100C:E3A03003    Mov R3,#3        ;count Of Numbers
00001010:E3A04000    MOV R4,#0
00001014:           Loop:
00001014:E4951004           Ldr R1,[R5],#4
00001018:E4962004           Ldr R2,[R6],#4
0000101C:E0000291           Mul R0,R1,R2      ;multiple R1 And R2 And Store Result In R0
00001020:E0844000           Add R4,R4,R0
00001024:E4874004           STR R4,[R7],#4
00001028:E2433001           Sub R3,R3,#1
0000102C:E3330000           Teq R3,#0         ; Test For Equality
00001030:1AFFFFFF7           Bne Loop
00001034:EF000011           Swi 0x11          ; Interrupt For Termination Of Program

        .DATA
00001044:           A: .word 1,3,5
00001050:           B: .word 2,4,6
0000105C:           C: .word

```

MLA EXAMPLE – Matrix Representation in memory

0	00000000
1	00000004
2	
3	
4	
5	
6	
7	
8	00000020
9	

$$\begin{array}{ccc}
 1 & 2 & 3 \\
 4 & 5 & 6 \\
 7 & 8 & 9
 \end{array}
 \Rightarrow
 \begin{array}{ccc}
 A(0,0) & A(0,1) & A(0,2) \\
 A(1,0) & A(1,1) & A(1,2) \\
 A(2,0) & A(2,1) & A(2,2)
 \end{array}$$

Row Major Order for a 3 x 3 matrix :

$$A [2, 2] = \text{Row Num.} \times \text{No. of elements per row} + \text{Column Num.}$$

$$2 \times 3 + 2 = 8^{\text{th}} \text{ location.}$$

- Since each location is of 4 bytes, the address of the 3rd element in the 2nd row is $8 \times 4 = 32$.
- This is achieved in ARM processor, using MLA instruction as follows.

MLA	R1,	R2,	R3,	R4
Mnemonic	Dst	Row #	# of elements per row	col #

- To get consecutive data elements in a @D Array.

Multiply-Accumulate $c=c+a[i]*b[i]$

<MLA> Rd, Rf, Rn, Rm @ $Rd = Rf \times Rn + Rm$

- **MLA** R4, R1, R2, R4 @ $R4 = R1 \times R2 + R4$

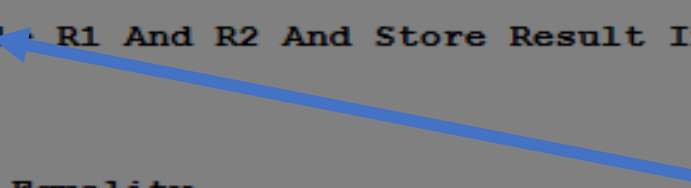
```

TEXT
00001000:E3A05D41  LDR R5,=A
00001004:E59F6028  LDR R6,=B
00001008:E59F7028  LDR R7,=C
0000100C:E3A03003  Mov R3,#3      ;count Of Numbers
00001010:E3A04000  MOV R4,#0
00001014:          Loop:
00001014:E4951004          Ldr R1,[R5],#4
00001018:E4962004          Ldr R2,[R6],#4
0000101C:E0244291  MLA R4,R1,R2,R4      ;multiply R1 And R2 And Store Result In R4

00001020:E4874004          STR R4,[R7],#4
00001024:E2433001  Sub R3,R3,#1
00001028:E3330000  Teq R3,#0          ; Test For Equality
0000102C:1AFFFFFF8  Bne Loop
00001030:EF000011  Swi 0x11          ; Interrupt For Termination Of Program

.DATA
00001040:          A: .word 1,3,5
0000104C:          B: .word 2,4,6
00001058:          C: .word

```





Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 6.2

Conditional Execution

Conditional Execution

- Almost all ARM instructions have a condition field which allows it to be executed conditionally.

MOVCS R0, R1

Mnemonic	Condition	Mnemonic	Condition
CS	<i>Carry Set</i>	CC	<i>Carry Clear</i>
EQ	<i>Equal (Zero Set)</i>	NE	<i>Not Equal (Zero Clear)</i>
VS	<i>Overflow Set</i>	VC	<i>Overflow Clear</i>
GT	<i>Greater Than</i>	LT	<i>Less Than</i>
GE	<i>Greater Than or Equal</i>	LE	<i>Less Than or Equal</i>
PL	<i>Plus (Positive)</i>	MI	<i>Minus (Negative)</i>
HI	<i>Higher Than</i>	LO	<i>Lower Than (aka CC)</i>
HS	<i>Higher or Same (aka CS)</i>	LS	<i>Lower or Same</i>

Conditional Execution - Example 01

; Convert the C statement to ARM instructions.

```
if (R0!=5) {R1=R1+R0-R2}
```

```
    CMP    R0, #5
```

```
    BEQ    bypass
```

```
    ADD    R1, R1, R0
```

```
    SUB    R1, R1, R2
```

```
bypass: ...
```

```
    CMP    R0, #5
```

```
    ADDNE  R1, R1, R0
```

```
    SUBNE  R1, R1, R2
```

; smaller and faster

Rule of thumb: if the conditional sequence is three instructions or less, it is better to use conditional execution than a branch.

Conditional Execution - Example 02

```
if ( (R0==R1) && (R2==R3) ) R4++
```

```
CMP    R0, R1
BNE    skip
CMP    R2, R3
BNE    skip
ADD    R4, R4, #1
```

skip:...

```
CMP    R0, R1
CMPEQ  R2, R3
ADDEQ  R4, R4, #1
```

MCQs

1. How many destination registers are used by UMULL?

- A) 1
- B) 2
- C) 3
- D) 4

2. Which register pair stores the 64-bit result in UMULL R4, R5, R2, R3 ?

- A) R4 only
- B) R5 only
- C) R4:R5
- D) R5:R4

MCQs

1. How many destination registers are used by UMULL?

- A) 1
- B) 2**
- C) 3
- D) 4

2. Which register pair stores the 64-bit result in UMULL R4, R5, R2, R3 ?

- A) R4 only
- B) R5 only
- C) R4:R5
- D) R5:R4**

MCQs

3. For a 3×3 matrix in row-major order, address of $A[2][2]$ is:

- A) 20 bytes
- B) 24 bytes
- C) 28 bytes
- D) 32 bytes

4. Why is conditional execution preferred for ≤ 3 instructions?

- A) Easier to write
- B) Reduces memory
- C) Smaller and faster
- D) Improves accuracy

MCQs

3. For a 3×3 matrix in row-major order, address of $A[2][2]$ is:

- A) 20 bytes
- B) 24 bytes
- C) 28 bytes
- D) 32 bytes**

4. Why is conditional execution preferred for ≤ 3 instructions?

- A) Easier to write
- B) Reduces memory
- C) Smaller and faster**
- D) Improves accuracy

MCQs

5. Conditional execution mainly reduces:

- A) Power consumption
- B) Clock cycles
- C) Branch penalty
- D) Register usage

MCQs

5. Conditional execution mainly reduces:

A) Power consumption

B) Clock cycles

C) Branch penalty

D) Register usage

THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 7

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Multiple Load / Store Instructions or Block Transfer

MULTIPLE REGISTER LOAD / STORE

- Transfer a block of data more efficiently.
- Used for procedure entry and exit for saving and restoring workspace registers and the return address
- For ARM7, $2+Nt$ cycles
(N : #words, t : time for a word for sequential access).
- Increase interrupt latency since it can't be interrupted.
- Registers are considered in increasing order always.

```
LDMIA  R1, {R0, R2, R5}    @ R0 = mem[R1]
                                @ R2 = mem[R1+4]
                                @ R5 = mem[R1+8]
```

MULTIPLE REGISTER LOAD / STORE

Syntax: <LDM|STM>{<cond>}<addressing mode> $Rn\{!\}$,<registers>{^}

Addressing mode	Description	Start address	End address	$Rn!$
IA	increment after	Rn	$Rn + 4*N - 4$	$Rn + 4*N$
IB	increment before	$Rn + 4$	$Rn + 4*N$	$Rn + 4*N$
DA	decrement after	$Rn - 4*N + 4$	Rn	$Rn - 4*N$
DB	decrement before	$Rn - 4*N$	$Rn - 4$	$Rn - 4*N$

MULTIPLE REGISTER LOAD / STORE

LDM<mode> Rn, {<registers>}

IA: **addr:=Rn**

IB: addr:=Rn+4

DA: addr:=Rn-#<registers>*4+4

DB: addr:=Rn-#<registers>*4

For each Ri in <registers>

IB: addr:=addr+4

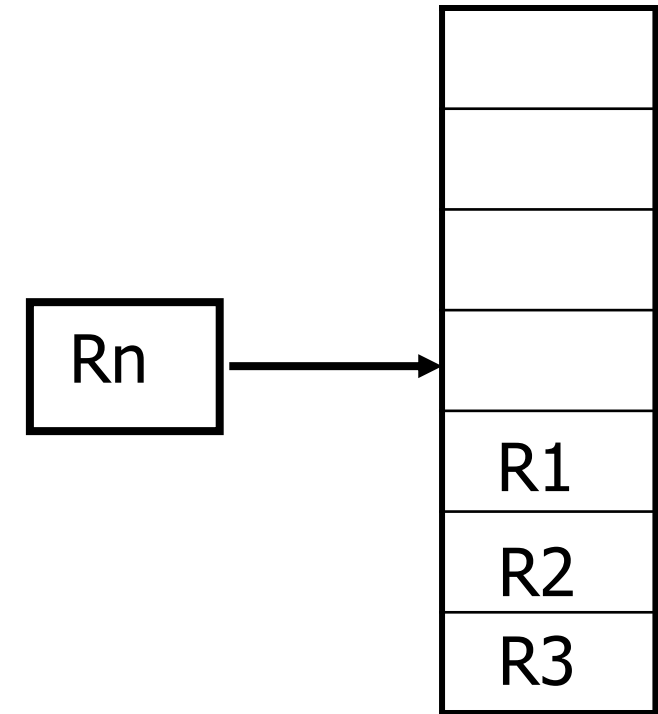
DB: addr:=addr-4

Ri:=M[addr]

IA: **addr:=addr+4**

DA: addr:=addr-4

<!:>: Rn:=addr



MULTIPLE REGISTER LOAD / STORE

LDMIA R0, {R1,R2,R3}

or

LDMIA R0, {R1-R3}

R1: 10

R2: 20

R3: 30

R0: 0x10



addr	data
0x010	10
0x014	20
0x018	30
0x01C	40
0x020	50
0x024	60

MULTIPLE REGISTER LOAD EXAMPLE-01

- Program to transfer data from memory locations to registers using single **Load Multiple Instruction using IA condition**

.DATA

A: .WORD 100,200,300,400,500,600,700,800

B: .WORD 0,0,0,0,0,0,0,0

.TEXT

LDR R0,=A

LDR R7,=B

LDMIA R0!,{R1,R8,R9,R11}

.END

MULTIPLE REGISTER STORE EXAMPLE-01 CONTD..

- Program to transfer data from memory locations to registers using single **STORE** Multiple Instruction using IA condition

.DATA

A: .WORD 100,200,300,400,500,600,700,800

B: .WORD 0,0,0,0,0,0,0,0

.TEXT

LDR R0,=A

LDR R7,=B

LDMIA R0!,{R1,R8,R9,R11}

STMIA R7!,{R1,R8,R9,R11}

.END

MULTIPLE REGISTER LOAD/STORE

LDM<mode> Rn, {<registers>}

IA: addr:=Rn

IB: addr:=Rn+4

DA: addr:=Rn-#<registers>*4+4

DB: addr:=Rn-#<registers>*4

For each Ri in <registers>

IB: addr:=addr+4

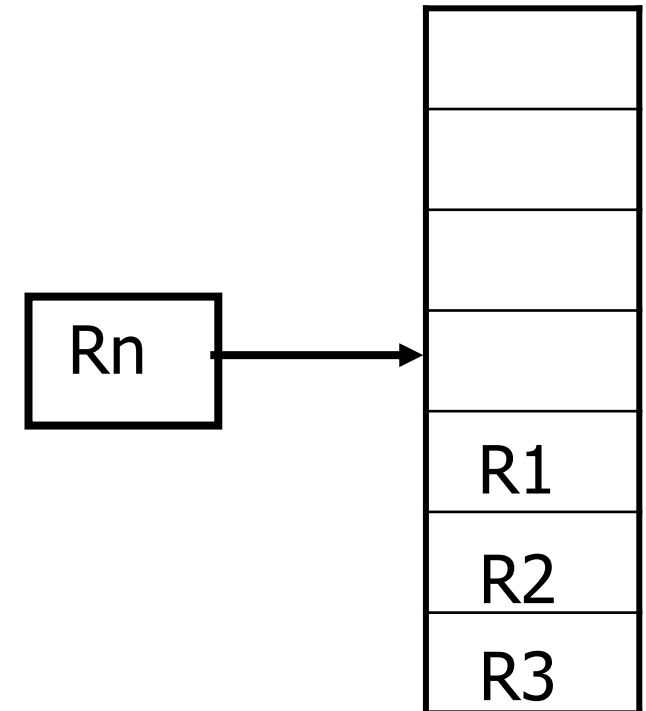
DB: addr:=addr-4

Ri:=M[addr]

IA: addr:=addr+4

DA: addr:=addr-4

<!:>: Rn:=addr



MULTIPLE REGISTER LOAD/STORE

LDMIB R0!, {R1,R2,R3}

R1: 20

R2: 30

R3: 40

R0: 0x01C

R0

addr	data
0x010	10
0x014	20
0x018	30
0x01C	40
0x020	50
0x024	60

MULTIPLE REGISTER LOAD/STORE EXAMPLE-02

- Program to transfer data from memory locations to registers using single **STORE Multiple Instruction using IB condition**

.DATA

A: .WORD 100,200,300,400,500,600,700,800,900,1000

B: .WORD 0,0,0,0,0,0,0,0,0,0

.TEXT

LDR R0,=A

LDR R11,=B

ADD R0, R0, #-4

ADD R11, R11, #-4

LDMIB R0!,{R1-R10}

STMIB R11!,{R1-R10}

.END

MULTIPLE REGISTER LOAD/STORE

LDM<mode> Rn, {<registers>}

IA: $\text{addr} := \text{Rn}$

IB: $\text{addr} := \text{Rn} + 4$

DA: $\text{addr} := \text{Rn} - \#<\text{registers}> * 4 + 4$

DB: $\text{addr} := \text{Rn} - \#<\text{registers}> * 4$

For each Ri in <registers>

IB: $\text{addr} := \text{addr} + 4$

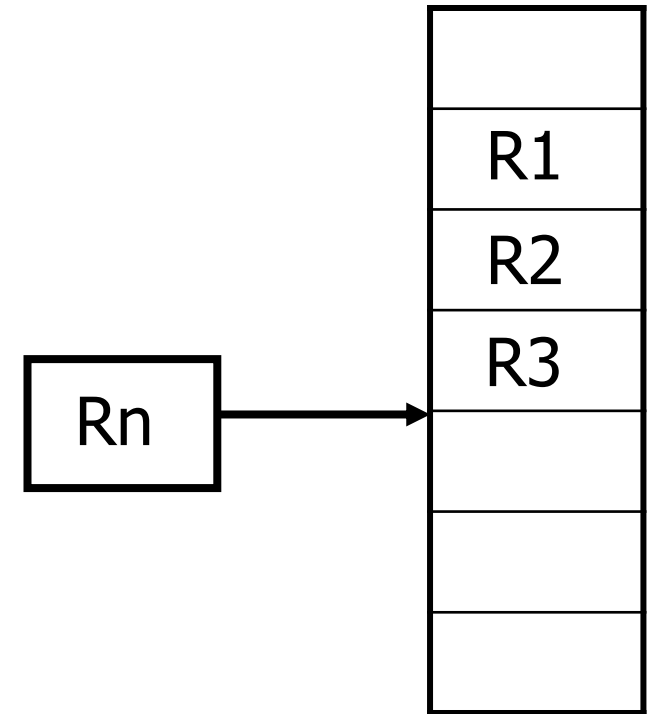
DB: $\text{addr} := \text{addr} - 4$

$\text{Ri} := \text{M}[\text{addr}]$

IA: $\text{addr} := \text{addr} + 4$

DA: $\text{addr} := \text{addr} - 4$

<!:> $\text{Rn} := \text{addr}$



MULTIPLE LOAD/STORE REGISTER

LDMDA R0!, {R1,R2,R3}

R1: 40

R2: 50

R3: 60

R0: 0x018



addr	data
0x010	10
0x014	20
0x018	30
0x01C	40
0x020	50
0x024	60

MULTIPLE REGISTER LOAD/STORE EXAMPLE-02

- Program to transfer data from memory locations to registers using single **STORE Multiple Instruction using DA condition**

.DATA

A: .WORD 100,200,300,400,500,600,700,800,900,1000

B: .WORD 0,0,0,0,0,0,0,0,0,0

.TEXT

LDR R0,=A

ADD R0,R0,#36

LDR R11,=B

ADD R11,R11,#36

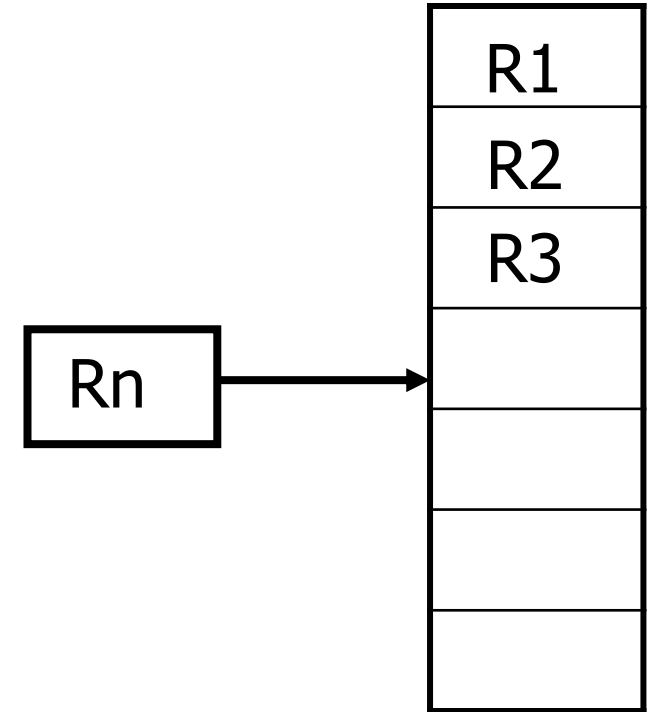
LDMDA R0!,{R1-R10}

STMDA R11!,{R1-R10}

.END

MULTIPLE LOAD/STORE REGISTER

```
LDM<mode> Rn, {<registers>}  
IA: addr:=Rn  
IB: addr:=Rn+4  
DA: addr:=Rn-#<registers>*4+4  
DB: addr:=Rn-#<registers>*4  
For each Ri in <registers>  
    IB: addr:=addr+4  
    DB: addr:=addr-4  
    Ri:=M[addr]  
    IA: addr:=addr+4  
    DA: addr:=addr-4  
<!:>: Rn:=addr
```



MULTIPLE LOAD/STORE REGISTER

LDMDB R0!, {R1,R2,R3}

R1: 30

R2: 40

R3: 50

R0: 0x018



addr	data
0x010	10
0x014	20
0x018	30
0x01C	40
0x020	50
0x024	60

MULTIPLE REGISTER LOAD/STORE EXAMPLE-03

- Program to transfer data from memory locations to registers using single **STORE Multiple Instruction using DB condition**

.DATA

A: .WORD 100,200,300,400,500,600,700,800,900,1000

B: .WORD 0,0,0,0,0,0,0,0,0,0

.TEXT

LDR R0,=A

ADD R0,R0,#40

LDR R11,=B

ADD R11,R11,#40

LDMDB R0!,{R1-R10}

STMDB R11!,{R1-R10}

.END

MULTIPLE REGISTER LOAD/STORE EXAMPLE-04

PRE mem32[0x80018] = 0x03
 mem32[0x80014] = 0x02
 mem32[0x80010] = 0x01
 r0 = 0x00080010
 r1 = 0x00000000
 r2 = 0x00000000
 r3 = 0x00000000

 LDMIA r0!, {r1-r3}

POST r0 = 0x0008001c
 r1 = 0x00000001
 r2 = 0x00000002
 r3 = 0x00000003

MULTIPLE REGISTER LOAD/STORE EXAMPLE-04

Address pointer	Memory address	Data	
	0x80020	0x00000005	
	0x8001c	0x00000004	
	0x80018	0x00000003	$r3 = 0x00000000$
	0x80014	0x00000002	$r2 = 0x00000000$
$r0 = 0x80010 \rightarrow$	0x80010	0x00000001	$r1 = 0x00000000$
	0x8000c	0x00000000	

LDMIA $r0!$, {r1-r3}

Address pointer	Memory address	Data	
	0x80020	0x00000005	
$r0 = 0x8001c \rightarrow$	0x8001c	0x00000004	
	0x80018	0x00000003	$r3 = 0x00000003$
	0x80014	0x00000002	$r2 = 0x00000002$
	0x80010	0x00000001	$r1 = 0x00000001$
	0x8000c	0x00000000	

MULTIPLE REGISTER LOAD/STORE EXAMPLE-05

Address pointer	Memory address	Data	
	0x80020	0x00000005	
	0x8001c	0x00000004	
	0x80018	0x00000003	$r3 = 0x00000000$
	0x80014	0x00000002	$r2 = 0x00000000$
$r0 = 0x80010 \rightarrow$	0x80010	0x00000001	$r1 = 0x00000000$
	0x8000c	0x00000000	

LDMIB $r0!, \{r1-r3\}$

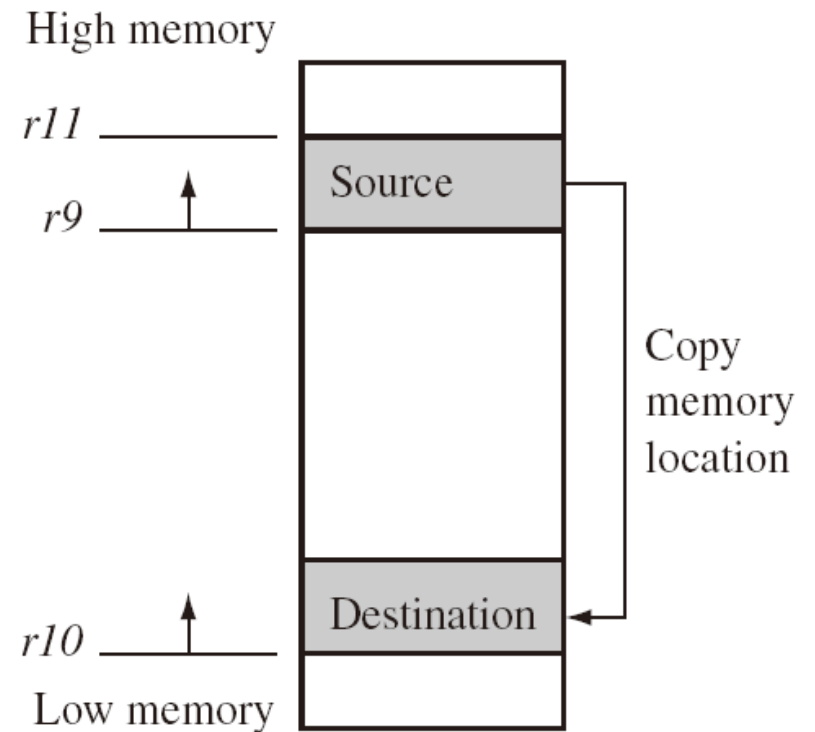
Address pointer	Memory address	Data	
	0x80020	0x00000005	
$r0 = 0x8001c \rightarrow$	0x8001c	0x00000004	$r3 = 0x00000004$
	0x80018	0x00000003	$r2 = 0x00000003$
	0x80014	0x00000002	$r1 = 0x00000002$
	0x80010	0x00000001	
	0x8000c	0x00000000	

MULTIPLE REGISTER LOAD / STORE- APPLICATION

Write a program using ARM7TDMI - ISA to copy a block 512 bytes of data from location A to location B if the rate of data transfer rate is 32 bytes.

- R9: address of the source A
- R10: address of the destination B
- R11: count : 16

```
loop: ADD R11,R11,#1
      LDMIA R9!, {R0-R7}
      STMIA R10!, {R0-R7}
      CMP R11,#16
      BNE loop
```



Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Block Transfer Instructions: Stack Operations

Block Transfer Instructions(STACK)

The Memory access can be in LIFO order.

i.e Can be treated like STACK.

R13 is a stack pointer which will keep the address of TOP of the STACK.

Mainly used in Procedural Call.

Stack can grow upward or downward direction based on the MODE used by the user in the program.

Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	20
0x1020	30
0x1024	40
0x1028	50
0x102C	60
0x1030	
0x1034	
0x1038	

Block Transfer Instructions(STACK)

10

20

10

140

15

60

27

R13=0x1018

SP

Addr	data
0x1010	
0x1014	
0x1018	
0x101C	
0x1020	
0x1024	
0x1028	
0x102C	
0x1030	
0x1034	
0x1038	

Block Transfer Instructions(STACK)

10

20

75

140

15

60

27

R13=0x101C

SP



Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	
0x1020	
0x1024	
0x1028	
0x102C	
0x1030	
0x1034	
0x1038	

Block Transfer Instructions(STACK)

10 20 75 140 15 60 27

R13=0x1020

SP



Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	20
0x1020	
0x1024	
0x1028	
0x102C	
0x1030	
0x1034	
0x1038	

Block Transfer Instructions(STACK)

10

20

75

140

15

60

27

R13=0x1024

SP



Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	20
0x1020	75
0x1024	
0x1028	
0x102C	
0x1030	
0x1034	
0x1038	

Block Transfer Instructions(STACK)

10 20 75 140 15 60 27

R13=0x1028

Addr	data
0x1010	
0x1014	
0x1018	10
0x101C	20
0x1020	75
0x1024	140
0x1028	
0x102C	
0x1030	
0x1034	
0x1038	

SP



Block Transfer Instructions(STACK)

Syntax:

<LDM/STM> <Addressing Mode> R13 {!}, Registers

Addressing Mode	=LDM	=STM
Full ascending (FA)	LDMDA	STMIB
Full descending (FD)	LDMIA	STMDB
Empty ascending (EA)	LDMDB	STMIA
Empty descending (ED)	LDMIB	STMDA

- STM is used to PUSH on to the STACK
- LDM is used to POP from the STACK

Block Transfer Instructions(STACK)

Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000000
R1	: 00000000
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (sl)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001000

CPSR Register	
Negative (N)	: 0
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

0x000000df	

```
.text
MOV R0,#4
MOV R1,#5
STMEA R13!, {R0,R1}
.end
```

Initially Stack is Empty

StackView	
000053C4	: 81818181
000053C8	: 81818181
000053CC	: 81818181
000053D0	: 81818181
000053D4	: 81818181
000053D8	: 81818181
000053DC	: 81818181
000053E0	: 81818181
000053E4	: 81818181
000053E8	: 81818181
000053EC	: 81818181
000053F0	: 81818181
000053F4	: 81818181
000053F8	: 81818181
000053FC	: 81818181
00005400	: 81818181
00005404	: 81818181
00005408	: 81818181
0000540C	: 81818181
00005410	: 81818181
00005414	: 81818181
00005418	: 81818181
0000541C	: 81818181
00005420	: 81818181
00005424	: 81818181
00005428	: 81818181
0000542C	: 81818181
00005430	: 81818181
00005434	: 81818181
00005438	: 81818181
0000543C	: 81818181

Block Transfer Instructions(STACK)

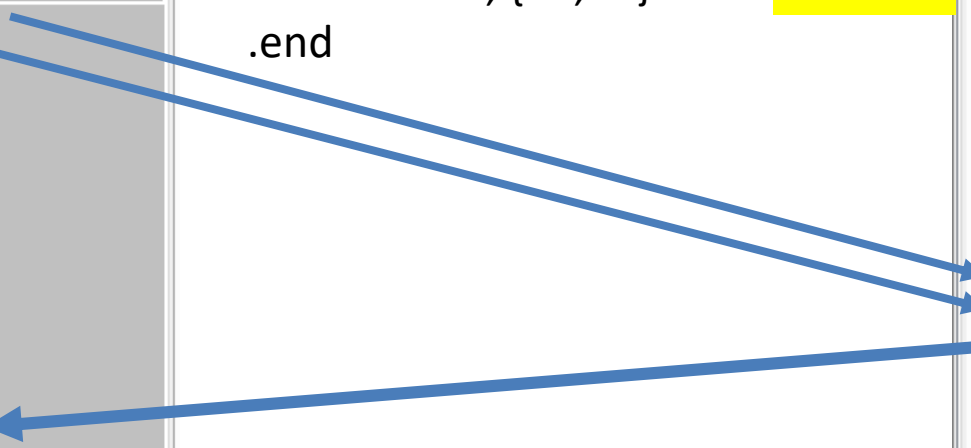
RegistersView	
General Purpose Floating Point	
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000004
R1	: 00000005
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (sl)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005408
R14 (lr)	: 00000000
R15 (pc)	: 00011400

CPSR Register	
Negative (N)	: 0
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

```
.text
MOV R0,#4
MOV R1,#5
STMEA R13!, {R0,R1}
.end
```

PUSH

StackView	
000053CC	: 81818181
000053D0	: 81818181
000053D4	: 81818181
000053D8	: 81818181
000053DC	: 81818181
000053E0	: 81818181
000053E4	: 81818181
000053E8	: 81818181
000053EC	: 81818181
000053F0	: 81818181
000053F4	: 81818181
000053F8	: 81818181
000053FC	: 81818181
00005400	: 00000004
00005404	: 00000005
00005408	: 81818181
0000540C	: 81818181
00005410	: 81818181
00005414	: 81818181
00005418	: 81818181
0000541C	: 81818181
00005420	: 81818181
00005424	: 81818181
00005428	: 81818181
0000542C	: 81818181
00005430	: 81818181
00005434	: 81818181
00005438	: 81818181
0000543C	: 81818181
00005440	: 81818181
00005444	: 81818181



Block Transfer Instructions(STACK)

RegistersView	
General Purpose Floating Point	
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000004
R1	: 00000005
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000004
R6	: 00000005
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001010

CPSR Register	
Negative (N)	: 0
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

0x000000df	

```
.text
MOV R0,#4
MOV R1,#5
STMEA R13!, {R0,R1}
LDMEA R13!,{R5,R6}
.end
```

POP

Again the Stack is Empty

StackView	
000053C4	: 81818181
000053C8	: 81818181
000053CC	: 81818181
000053D0	: 81818181
000053D4	: 81818181
000053D8	: 81818181
000053DC	: 81818181
000053E0	: 81818181
000053E4	: 81818181
000053E8	: 81818181
000053EC	: 81818181
000053F0	: 81818181
000053F4	: 81818181
000053F8	: 81818181
000053FC	: 81818181
00005400	: 00000004
00005404	: 00000005
00005408	: 81818181
0000540C	: 81818181
00005410	: 81818181
00005414	: 81818181
00005418	: 81818181
0000541C	: 81818181
00005420	: 81818181
00005424	: 81818181
00005428	: 81818181
0000542C	: 81818181
00005430	: 81818181
00005434	: 81818181
00005438	: 81818181
0000543C	: 81818181

1. In a "Full Descending" (FD) stack, which Load/Store Multiple instructions are used for POP and PUSH operations?

- A) POP = LDMDA, PUSH = STMIB
- B) POP = LDMIA, PUSH = STMDB
- C) POP = LDMDB, PUSH = STMIA
- D) POP = LDMIB, PUSH = STMDA

2. If R0 = 0x1000 and instruction is LDMDB R0!, {R1,R2,R3}, final value of R0 becomes:

- A) 0x1000
- B) 0x1004
- C) 0x0FF4
- D) 0x0FF0

1. In a "Full Descending" (FD) stack, which Load/Store Multiple instructions are used for POP and PUSH operations?

- A) POP = LDMDA, PUSH = STMIB
- B) POP = LDMIA, PUSH = STMDB**
- C) POP = LDMDB, PUSH = STMIA
- D) POP = LDMIB, PUSH = STMDA

2. If R0 = 0x1000 and instruction is LDMDB R0!, {R1,R2,R3}, final value of R0 becomes:

- A) 0x1000
- B) 0x1004
- C) 0x0FF4**
- D) 0x0FF0

3. If 512 bytes are copied using {R0-R7} in each iteration, number of loop iterations is:

- A) 8
- B) 16
- C) 32
- D) 64

4. Which addressing mode gives highest memory address first?

- A) IA
- B) IB
- C) DA
- D) DB

3. If 512 bytes are copied using {R0-R7} in each iteration, number of loop iterations is:

- A) 8
- B) 16**
- C) 32
- D) 64

4. Which addressing mode gives highest memory address first?

- A) IA
- B) IB
- C) DA**
- D) DB

5. Why does block transfer increase interrupt latency?

- A) Too many instructions
- B) Uses stack
- C) Cannot be interrupted mid-execution
- D) Uses pipeline

6. Which addressing mode accesses lowest memory address first?

- A) IA
- B) IB
- C) DA
- D) DB

5. Why does block transfer increase interrupt latency?

- A) Too many instructions
- B) Uses stack
- C) Cannot be interrupted mid-execution**
- D) Uses pipeline

6. Which addressing mode accesses lowest memory address first?

- A) IA**
- B) IB
- C) DA
- D) DB



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 8

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Procedure Call or Subroutine Call or Function Call

Procedure Call or Subroutine Call or Function Call

- The single variant on the **branch** theme is the option to perform a link operation before the branch is executed. (**BL**)
- This simply means storing the current value of **R15 (PC)** in **R14** **before** the **branch** is taken, so that the program can resume its operation.
 - Procedure call is invoked using a BL instruction .
 - Branch with link, **BL** instruction i.e.,
 - **BL subroutine**

Procedure Call or Subroutine Call or Function Call

- ARM has no dedicated stack.
 - It copies R15 into R14
- If the called Routine needs to use R14 for something, it can save it on the stack explicitly.
- The ARM method has the advantage that subroutines which don't need to save R14 can be called return very quickly.
- The disadvantage is that all other routines have the overhead of explicitly saving R14.
- To return from a subroutine, the program simply has to move R14 back into register R15.
 - MOV R15, R14 or MOV PC, LR
 - MOVS R15, R14 or MOV PC, LR

PROCEDURE CALL

R0-R7	
R8-R12	
R13 (SP)	
R14 (LR)	
R15 (PC)	0x0010
CPSR	

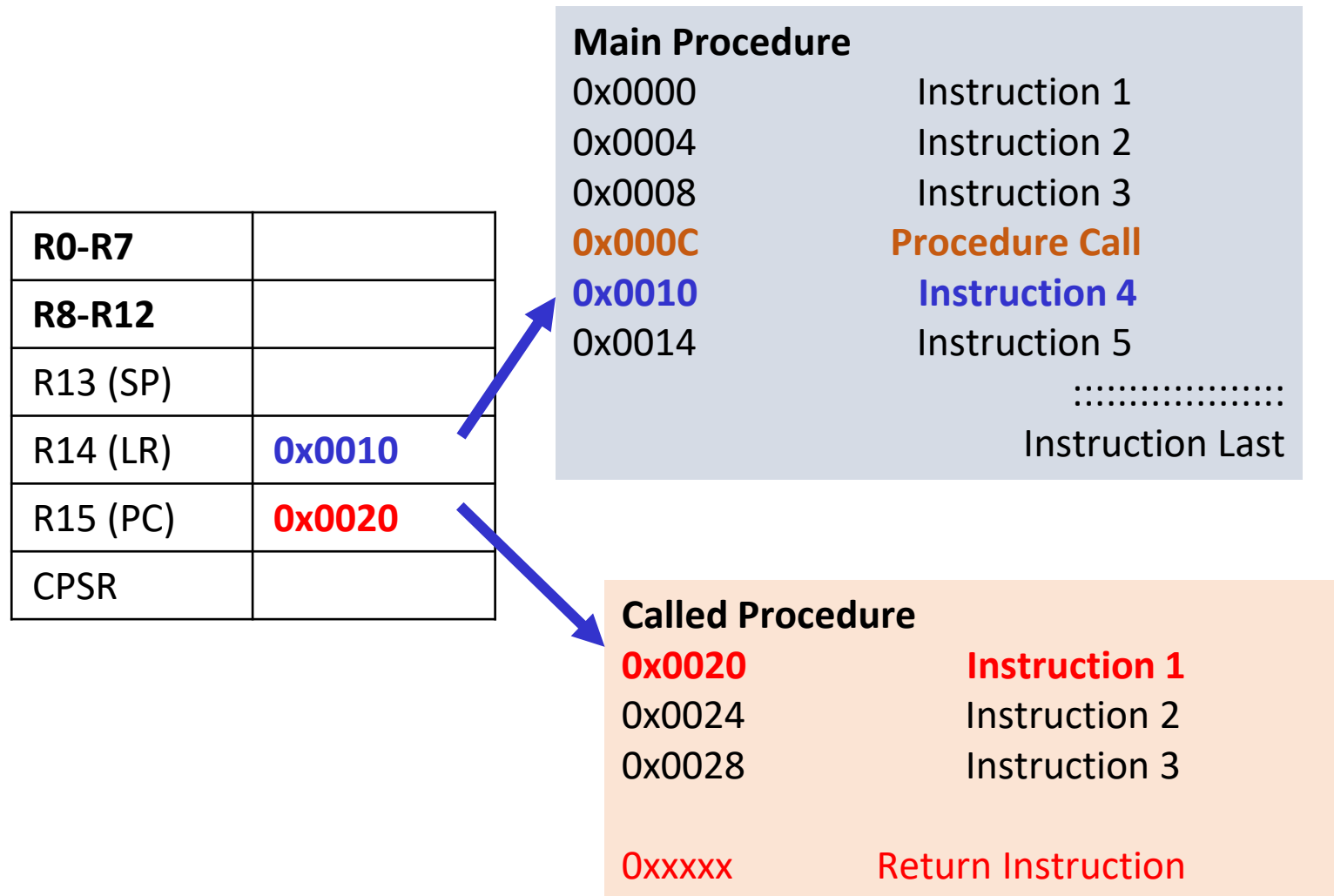
Main Procedure

0x0000	Instruction 1
0x0004	Instruction 2
0x0008	Instruction 3
0x000C	Procedure Call
0x0010	Instruction 4
0x0014	Instruction 5
	
	Instruction Last

Called Procedure

0x0020	Instruction 1
0x0024	Instruction 2
0x0028	Instruction 3
0xxxxx	Return Instruction

General Structure of Procedure Call:

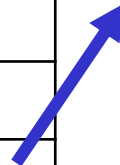


LR=PC

PC= Address of the 1st Instruction

General Structure of Procedure return:

R0-R7	
R8-R12	
R13 (SP)	
R14 (LR)	
R15 (PC)	0x0010
CPSR	



Main Procedure

0x0000 Instruction 1
 0x0004 Instruction 2
 0x0008 Instruction 3
 0x000C **Procedure Call**
 0x0010 **Instruction 4**
 0x0014 Instruction 5

 Instruction Last

Called Procedure

0x0020 **Instruction 1**
 0x0024 Instruction 2
 0x0028 Instruction 3

 0xxxxx **Return Instruction**

PC=LR

General Structure of Procedure call and return:

Main Procedure

0x0000	Instruction 1
0x0004	Instruction 2
0x0008	Instruction 3
0x000C	BL Procedure
0x0010	Instruction 4
0x0014	Instruction 5
	
	Instruction Last

Called Procedure

0x0020	Procedure: Instruction 1
0x0024	Instruction 2
0x0028	Instruction 3
0xxxxx	MOV PC LR
	or
	BX LR

PROCEDURE CALL EXAMPLE:01

```
MOV R1, #3  
BL FOOO  
ADD R2, R0, R1  
SWI 0x11
```

FOOO:

```
MOV R0, #2  
BX LR
```

```
MOV R1, #3  
BL FOOO  
ADD R2, R0, R1  
SWI 0x11
```

FOOO:

```
MOV R0, #2  
MOV PC, LR
```

PROCEDURE CALL EXAMPLE:01

RegistersView 🔍 ✕

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0	:	00000000
R1	:	00000000
R2	:	00000000
R3	:	00000000
R4	:	00000000
R5	:	00000000
R6	:	00000000
R7	:	00000000
R8	:	00000000
R9	:	00000000
R10 (s1)	:	00000000
R11 (fp)	:	00000000
R12 (ip)	:	00000000
R13 (sp)	:	00005400
R14 (lr)	:	00000000
R15 (pc)	:	00001000

even.s

```

00001000:E3A01003    main:    mov r1, #3
00001004:EB000001                bl foo
00001008:E0802001                add r2, r0, r1
0000100C:EF000011                swi 0x11
00001010:                                foo:
00001010:E3A00002                mov r0, #2
00001014:E1A0F00E                mov pc, lr

```

PROCEDURE CALL EXAMPLE:01

RegistersView

General Purpose

Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0 : 00000000
R1 : 00000003
R2 : 00000000
R3 : 00000000
R4 : 00000000
R5 : 00000000
R6 : 00000000
R7 : 00000000
R8 : 00000000
R9 : 00000000
R10 (s1) : 00000000
R11 (fp) : 00000000
R12 (ip) : 00000000
R13 (sp) : 00005400
R14 (lr) : 00001008
R15 (pc) : 00001010

jeven.s

```

00001000:E3A01003    main:  mov r1, #3
00001004:EB000001          bl foo
00001008:E0802001          add r2, r0, r1
0000100C:EF000011          swi 0x11
00001010:                foo:
00001010:E3A00002          mov r0, #2
00001014:E1A0F00E          mov pc, lr

```

PROCEDURE CALL EXAMPLE:01

RegistersView		✕
General Purpose		Floating Point
Hexadecimal		
Unsigned Decimal		
Signed Decimal		
R0	: 00000002	
R1	: 00000003	
R2	: 00000000	
R3	: 00000000	
R4	: 00000000	
R5	: 00000000	
R6	: 00000000	
R7	: 00000000	
R8	: 00000000	
R9	: 00000000	
R10 (s1)	: 00000000	
R11 (fp)	: 00000000	
R12 (ip)	: 00000000	
R13 (sp)	: 00005400	
R14 (lr)	: 00001008	
R15 (pc)	: 00001008	

jeven.s	
00001000:E3A01003	main: mov r1, #3
00001004:EB000001	bl foo
00001008:E0802001	add r2, r0, r1
0000100C:EF000011	swi 0x11
00001010:	foo:
00001010:E3A00002	mov r0, #2
00001014:E1A0F00E	mov pc, lr

PARAMETER PASSING TO PROCEDURES USING STACK

```
MOV R1, #25           ; parameter1
MOV R2, #25           ; parameter2
STMFD R13!, { R1, R2} ; parameters are PUSHed on stack.
BL LINK
LDR R4, =A
STR R0, [R4]           ; return value in Reg. R0.
SWI 0x11
```

```
LINK: LDMFD R13!, {R4, R5} ; parameters are POPed from the stack
      ADD R0, R4, R5       ; Result is in register R0.
      MOV PC, LR
```

```
A: .WORD 0
```

```
.END
```

PARAMETER PASSING TO PROCEDURES USING STACK

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0 : 00000000
R1 : 00000000
R2 : 00000000
R3 : 00000000
R4 : 00000000
R5 : 00000000
R6 : 00000000
R7 : 00000000
R8 : 00000000
R9 : 00000000
R10 (s1) : 00000000
R11 (fp) : 00000000
R12 (ip) : 00000000
R13 (sp) : 00005400
R14 (lr) : 00000000
R15 (pc) : 00001000

CPSR Register

Negative (N) : 0

Zero (Z) : 0

Carry (C) : 0

Overflow (V) : 0

IRQ Disable: 1

FIQ Disable: 1

Thumb (T) : 0

CPU Mode : System

.text

```
00001000:E59F4024    LDR    R4, =A
00001004:E3A01019    MOV    R1, #25
00001008:E3A02019    MOV    R2, #25
0000100C:E92D0006    STMFD  R13!, { R1, R2}
00001010:EB000001    BL     LINK
00001014:E5840000    STR    R0, [R4]
00001018:EF000011    SWI    0x11

0000101C:E8BD0030    LINK:  LDMFD R13!, { R4, R5}
00001020:E0840005    ADD    R0, R4, R5
00001024:E1A0F00E    MOV    PC, LR

00001028:00000000    A:     .WORD 0

0000102C:00001028    .END
```

StackView

000053E0:81818181
000053E4:81818181
000053E8:81818181
000053EC:81818181
000053F0:81818181
000053F4:81818181
000053F8:81818181
000053FC:81818181
00005400:81818181
00005404:81818181
00005408:81818181
0000540C:81818181
00005410:81818181
00005414:81818181
00005418:81818181
0000541C:81818181

OutputView

Console Stdin/Stdout/S

PARAMETER PASSING TO PROCEDURES USING STACK

RegistersView

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0	:00000000
R1	:00000019
R2	:00000019
R3	:00000000
R4	:00001028
R5	:00000000
R6	:00000000
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:000053f8
R14 (lr)	:00000000
R15 (pc)	:00001010

CPSR Register

Negative (N) : 0

Zero (Z) : 0

Carry (C) : 0

Overflow (V) : 0

IRQ Disable: 1

FIQ Disable: 1

Thumb (T) : 0

jeven.s

.text

00001000:E59F4024 LDR R4, =A

00001004:E3A01019 MOV R1, #25

00001008:E3A02019 MOV R2, #25

0000100C:E92D0006 STMFD R13!, { R1, R2}

00001010:EB000001 BL LINK

00001014:E5840000 STR R0, [R4]

00001018:EF000011 SWI 0x11

0000101C:E8BD0030 LINK: LDMFD R13!, { R4, R5}

00001020:E0840005 ADD R0, R4, R5

00001024:E1A0F00E MOV PC, LR

00001028:00000000 A: .WORD 0

0000102C:00001028 .END

StackView

000053D8:81818181

000053DC:81818181

000053E0:81818181

000053E4:81818181

000053E8:81818181

000053EC:81818181

000053F0:81818181

000053F4:81818181

000053F8:00000019

000053FC:00000019

00005400:81818181

00005404:81818181

00005408:81818181

0000540C:81818181

00005410:81818181

OutputView

PARAMETER PASSING TO PROCEDURES USING STACK

RegistersView

General Purpose Floating Point

Hexadecimal

Unsigned Decimal

Signed Decimal

R0	: 00000000
R1	: 00000019
R2	: 00000019
R3	: 00000000
R4	: 00001028
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 000053f8
R14 (lr)	: 00001014
R15 (pc)	: 0000101c

CPSR Register

Negative (N) : 0

Zero (Z) : 0

Carry (C) : 0

Overflow (V) : 0

IRQ Disable: 1

jeven.s

.text

00001000:E59F4024 LDR R4, =A

00001004:E3A01019 MOV R1, #25

00001008:E3A02019 MOV R2, #25

0000100C:E92D0006 STMFD R13!, { R1, R2}

00001010:EB000001 BL LINK

00001014:E5840000 STR R0, [R4]

00001018:EF000011 SWI 0x11

0000101C:E8BD0030 LINK: LDMFD R13!, { R4, R5}

00001020:E0840005 ADD R0, R4, R5

00001024:E1A0F00E MOV PC, LR

00001028:00000000 A: .WORD 0

0000102C:00001028 .END

StackView

000053D8:81818181

000053DC:81818181

000053E0:81818181

000053E4:81818181

000053E8:81818181

000053EC:81818181

000053F0:81818181

000053F4:81818181

000053F8:00000019

000053FC:00000019

00005400:81818181

00005404:81818181

00005408:81818181

PARAMETER PASSING TO PROCEDURES USING STACK

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000000
R1	: 00000019
R2	: 00000019
R3	: 00000000
R4	: 00000019
R5	: 00000019
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00001014
R15 (pc)	: 00001020

CPSR Register	
Negative (N)	: 0
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0

jeven.s	
.text	
00001000:E59F4024	LDR R4, =A
00001004:E3A01019	MOV R1, #25
00001008:E3A02019	MOV R2, #25
0000100C:E92D0006	STMFD R13!, { R1, R2}
00001010:EB000001	BL LINK
00001014:E5840000	STR R0, [R4]
00001018:EF000011	SWI 0x11
0000101C:E8BD0030	LINK: LDMFD R13!, { R4, R5}
00001020:E0840005	ADD R0, R4, R5
00001024:E1A0F00E	MOV PC, LR
00001028:00000000	A: .WORD 0
0000102C:00001028	.END

StackView	
000053E0:81818181	
000053E4:81818181	
000053E8:81818181	
000053EC:81818181	
000053F0:81818181	
000053F4:81818181	
000053F8:00000019	
000053FC:00000019	
00005400:81818181	
00005404:81818181	
00005408:81818181	

PARAMETER PASSING TO PROCEDURES USING STACK

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	:00000032
R1	:00000019
R2	:00000019
R3	:00000000
R4	:00000019
R5	:00000019
R6	:00000000
R7	:00000000
R8	:00000000
R9	:00000000
R10 (s1)	:00000000
R11 (fp)	:00000000
R12 (ip)	:00000000
R13 (sp)	:00005400
R14 (lr)	:00001014
R15 (pc)	:00001014

CPSR Register	
Negative (N)	:0
Zero (Z)	:0
Carry (C)	:0
Overflow (V)	:0
IRQ Disable	:1

```
jeven.s
.text
00001000:E59F4024    LDR    R4, =A
00001004:E3A01019    MOV    R1, #25
00001008:E3A02019    MOV    R2, #25
0000100C:E92D0006    STMFD  R13!, { R1, R2}
00001010:EB000001    BL     LINK
00001014:E5840000    STR    R0, [R4]
00001018:EF000011    SWI    0x11

0000101C:E8BD0030    LINK:  LDMFD R13!, { R4, R5}
00001020:E0840005    ADD    R0, R4, R5
00001024:E1A0F00E    MOV    PC, LR

00001028:00000000    A:     .WORD 0

0000102C:00001028    .END

StackView
000053E0:81818181
000053E4:81818181
000053E8:81818181
000053EC:81818181
000053F0:81818181
000053F4:81818181
000053F8:00000019
000053FC:00000019
00005400:81818181
00005404:81818181
00005408:81818181
0000540C:81818181
00005410:81818181
```

NESTED PROCEDURE CALL

.DATA

A: .WORD 0

.TEXT ; **MAIN Procedure**

LDR R4, =A

MOV R1, #11

MOV R2, #10

MOV R3, #02

STMFD R13!, {R1,R2,R3}

BL ADDFun ; Call to ADD Procedure

STR R0, [R4]

SWI 0x11

ADDFun: LDMFD R13!, {R4, R5,R6} ; **ADD Procedure**

ADD R0, R4, R5

STMFD R13!, {R0,R6,LR}

BL MULFun

; Call to MUL Procedure

LDMFD R13!,{LR}

MOV PC, LR

; Return to Main Procedure

MULFun: LDMFD R13!, { R4, R5}

MUL R0, R4, R5

MOV PC, LR

; Return to ADD Procedure

NESTED PROCEDURE CALL

	Hexadecimal
	Unsigned Decimal
	Signed Decimal
R0	: 00000000
R1	: 0000000b
R2	: 0000000a
R3	: 00000002
R4	: 00001044
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001010

```

.TEXT
00001000:E59F4038  LDR R4,=A
00001004:E3A0100B  MOV R1,#11
00001008:E3A0200A  MOV R2,#10
0000100C:E3A03002  MOV R3,#2
00001010:E92D000E  STMFD R13!, {R1,R2,R3}
00001014:EB000001  BL ADDFun
00001018:E5840000  STR R0, [R4]
0000101C:EF000011  SWI 0x11

00001020:E8BD0070  ADDFun: LDMFD R13!, { R4, R5,R6}
00001024:E0840005  ADD R0, R4, R5
00001028:E92D4041  STMFD R13!, {R0,R6,LR}
0000102C:EB000000  BL MULFun
00001030:E1A0F00E  MOV PC, LR
00001034:E8BD4030  MULFun: LDMFD R13!, { R4, R5,LR}
00001038:E0000594  MUL R0, R4, R5
0000103C:E1A0F00E  MOV PC, LR

.DATA
00001044:  A:  .WORD  0
  
```

StackView  

000053AC:81818181
000053B0:81818181
000053B4:81818181
000053B8:81818181
000053BC:81818181
000053C0:81818181
000053C4:81818181
000053C8:81818181
000053CC:81818181
000053D0:81818181
000053D4:81818181
000053D8:81818181
000053DC:81818181
000053E0:81818181
000053E4:81818181
000053E8:81818181
000053EC:81818181
000053F0:81818181
000053F4:81818181
000053F8:81818181
000053FC:81818181
00005400:81818181
00005404:81818181
00005408:81818181
0000540C:81818181
00005410:81818181
00005414:81818181
00005418:81818181
0000541C:81818181
00005420:81818181
00005424:81818181
00005428:81818181
0000542C:81818181

NESTED PROCEDURE CALL

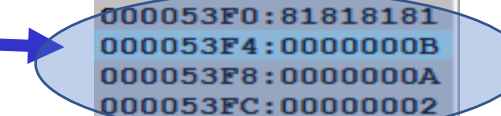
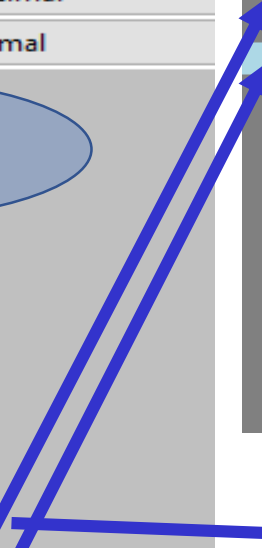
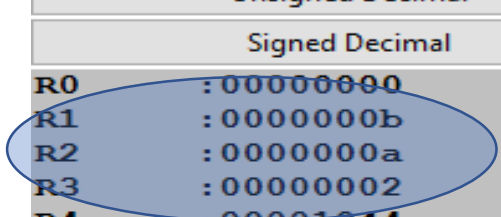
RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000000
R1	: 0000000b
R2	: 0000000a
R3	: 00000002
R4	: 00001044
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 000053f4
R14 (lr)	: 00001018
R15 (pc)	: 00001020

```

00001000:E59F4038    LDR R4,=A
00001004:E3A0100B    MOV R1,#11
00001008:E3A0200A    MOV R2,#10
0000100C:E3A03002    MOV R3,#2
00001010:E92D000E    STMFD R13!, {R1,R2,R3}
00001014:EB000001    BL ADDProc
00001018:E5840000    STR R0, [R4]
0000101C:EF000011    SWI 0x11
00001020:E8BD0070    ADDProc: LDMFD R13!, { R4, R5,R6}
00001024:E0840005            ADD R0, R4, R5
00001028:E92D4041            STMFD R13!, {R0,R6,LR}
0000102C:EB000000            BL MULProc
00001030:E1A0F00E            MOV PC, LR
00001034:E8BD4030    MULProc: LDMFD R13!, { R4, R5,LR}
00001038:E0000594            MUL R0, R4, R5
0000103C:E1A0F00E            MOV PC, LR

00001044:            .DATA
A:            .WORD 0
  
```

StackView	
000053A0	: 81818181
000053A4	: 81818181
000053A8	: 81818181
000053AC	: 81818181
000053B0	: 81818181
000053B4	: 81818181
000053B8	: 81818181
000053BC	: 81818181
000053C0	: 81818181
000053C4	: 81818181
000053C8	: 81818181
000053CC	: 81818181
000053D0	: 81818181
000053D4	: 81818181
000053D8	: 81818181
000053DC	: 81818181
000053E0	: 81818181
000053E4	: 81818181
000053E8	: 81818181
000053EC	: 81818181
000053F0	: 81818181
000053F4	: 0000000B
000053F8	: 0000000A
000053FC	: 00000002
00005400	: 81818181
00005404	: 81818181
00005408	: 81818181
0000540C	: 81818181
00005410	: 81818181



NESTED PROCEDURE CALL

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000015
R1	: 0000000b
R2	: 0000000a
R3	: 00000002
R4	: 0000000b
R5	: 0000000a
R6	: 00000002
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 000053f4
R14 (lr)	: 00001030
R15 (pc)	: 00001034

```

00001000:E59F4038    LDR R4,=A
00001004:E3A0100B    MOV R1,#11
00001008:E3A0200A    MOV R2,#10
0000100C:E3A03002    MOV R3,#2
00001010:E92D000E    STMFD R13!, {R1,R2,R3}
00001014:EB000001    BL ADDProc
00001018:E5840000    STR R0, [R4]
0000101C:EF000011    SWI 0x11
00001020:E8BD0070    ADDProc: LDMFD R13!, { R4, R5,R6}
00001024:E0840005        ADD R0, R4, R5
00001028:E92D4041    STMFD R13!, {R0,R6,LR}
0000102C:EB000000    BL MULProc
00001030:E1A0F00E    MOV PC, LR
00001034:E8BD4030    MULProc: LDMFD R13!, { R4, R5,LR}
00001038:E0000594        MUL R0, R4, R5
0000103C:E1A0F00E    MOV PC, LR

00001044:
        .DATA
A:      .WORD 0
  
```

StackView	
000053A0:81818181	
000053A4:81818181	
000053A8:81818181	
000053AC:81818181	
000053B0:81818181	
000053B4:81818181	
000053B8:81818181	
000053BC:81818181	
000053C0:81818181	
000053C4:81818181	
000053C8:81818181	
000053CC:81818181	
000053D0:81818181	
000053D4:81818181	
000053D8:81818181	
000053DC:81818181	
000053E0:81818181	
000053E4:81818181	
000053E8:81818181	
000053EC:81818181	
000053F0:81818181	
000053F4:00000015	
000053F8:00000002	
000053FC:00001018	
00005400:81818181	

NESTED PROCEDURE CALL

Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 0000002a
R1	: 0000000b
R2	: 0000000a
R3	: 00000002
R4	: 00000015
R5	: 00000002
R6	: 00000002
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00001018
R15 (pc)	: 0000103c

```

.TEXT
00001000:E59F4038    LDR R4,=A
00001004:E3A0100B    MOV R1,#11
00001008:E3A0200A    MOV R2,#10
0000100C:E3A03002    MOV R3,#2
00001010:E92D000E    STMFD R13!, {R1,R2,R3}
00001014:EB000001    BL ADDProc
00001018:E5840000    STR R0, [R4]
0000101C:EF000011    SWI 0x11
00001020:E8BD0070    ADDProc: LDMFD R13!, { R4, R5,R6}
00001024:E0840005                ADD R0, R4, R5
00001028:E92D4041                STMFD R13!, {R0,R6,LR}
0000102C:EB000000                BL MULProc
00001030:E1A0F00E                MOV PC, LR
00001034:E8BD4030    MULProc: LDMFD R13!, { R4, R5,LR}
00001038:E0000594                MUL R0, R4, R5
0000103C:E1A0F00E                MOV PC, LR

.DATA
00001044:                A:      .WORD  0

```

NESTED PROCEDURE CALL

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 0000002a
R1	: 0000000b
R2	: 0000000a
R3	: 00000002
R4	: 00000015
R5	: 00000002
R6	: 00000002
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00001018
R15 (pc)	: 00001018

```

.TEXT
00001000:E59F4038    LDR R4,=A
00001004:E3A0100B    MOV R1,#11
00001008:E3A0200A    MOV R2,#10
0000100C:E3A03002    MOV R3,#2
00001010:E92D000E    STMFD R13!, {R1,R2,R3}
00001014:EB000001    BL  ADDProc
00001018:E5840000    STR R0, [R4]
0000101C:EF000011    SWI 0x11
00001020:E8BD0070    ADDProc:  LDMFD R13!, { R4, R5,R6}
00001024:E0840005                ADD R0, R4, R5
00001028:E92D4041                STMFD R13!, {R0,R6,LR}
0000102C:EB000000                BL MULProc
00001030:E1A0F00E                MOV PC, LR
00001034:E8BD4030    MULProc:  LDMFD R13!, { R4, R5,LR}
00001038:E0000594                MUL R0, R4, R5
0000103C:E1A0F00E                MOV PC, LR

.DATA
00001044:          A:      .WORD  0

```

Microprocessor & Computer Architecture (μ pCA)



UE24CS251B

Software Interrupts

Session - 8.2

WHAT IS INTERRUPT/EXCEPTION?

- Main ()
- {
- :
- Doing something
- (e.g.
- browsing)
- :
- } ring

Can happen anytime
Depends on types of
interrupts



Phone
rings

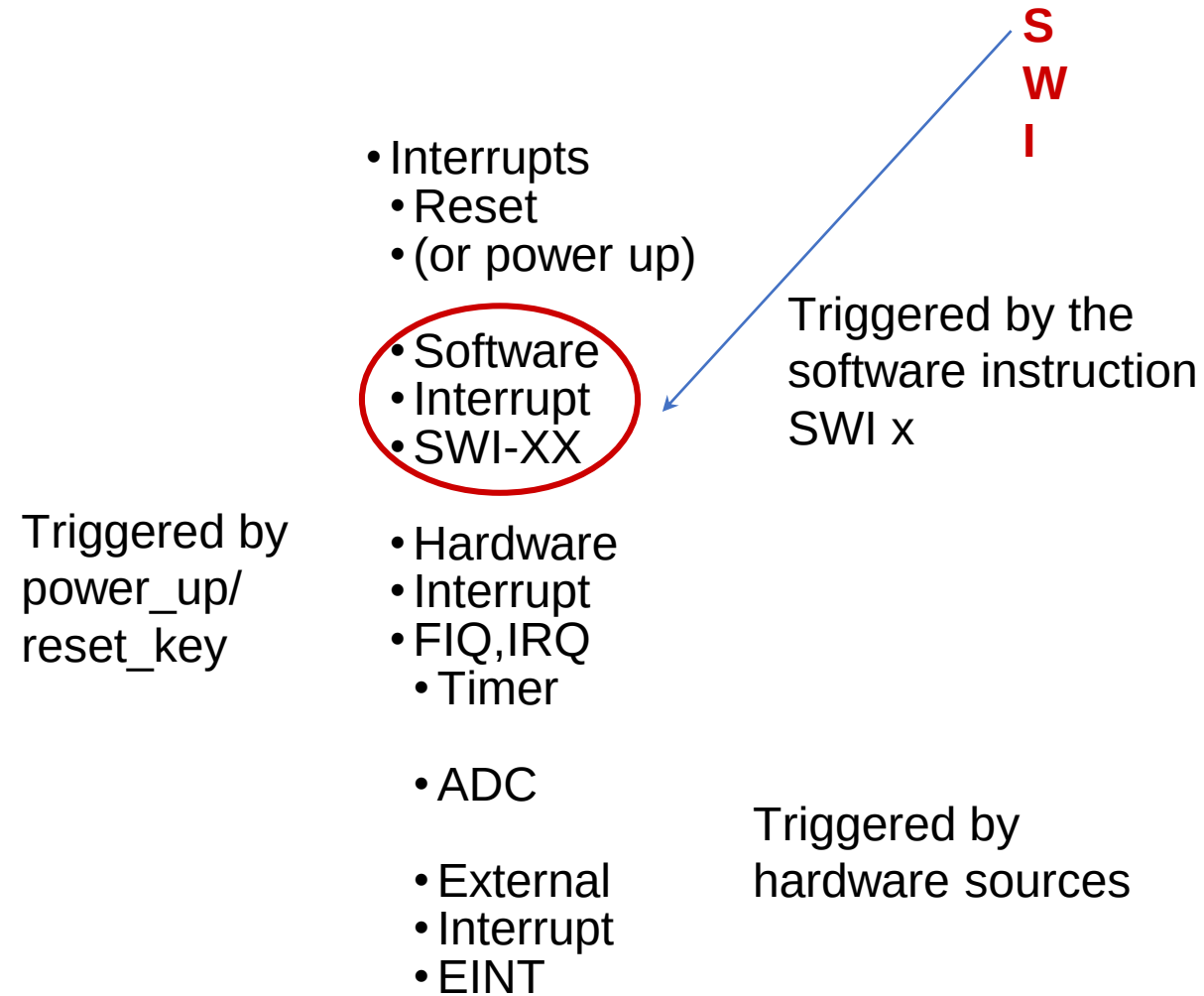
Phone
rings



```
_isr() // Interrupt service routine
{
    some tasks (e.g. answer
                telephone)
} //when finished,
  //goes back to main
```



IMPORTANT INTERRUPTS



INTERRUPTS AND EXCEPTION

- The terms are used differently by various manufacturers
- Traditionally exception means

The normal operation of a program is interrupted and the processor will execute another piece of software (exception handling) somewhere.

- Interrupt (hardware interrupt) is an exception caused by some hardware condition happening outside the processor (e.g. external hard interrupt, IRQ FIQ).
- Software interrupt (SWI) is an exception caused by an assembly software instruction (SWI 0x?? exception call instruction) written in the software code.
- Trap is an exception caused by a failure condition of the processor (e.g. abort “pre-fetch , data” , undefined instruction, divided_by_zero, or stack overflow etc)

IMPORTANT INTERRUPTS IN WORDS

- Reset, a special interrupt to start the system– happens at power up , or reset button depressed)
- Software interrupt SWI :
Similar to subroutine – happens when “SWI 0x??”
is written in the program
- Hardware interrupt
 - FIQ (fast interrupt) or IRQ (external interrupt), when
 - the external interrupt request pin is pulled low, or
 - an analog to digital conversion is completed, or
 - A timer/counter has made a regular request

HARDWARE AND SOFTWARE INTERRUPT

• Hardware interrupt

```
IRQ_Eint1(
)
{
::
}
```

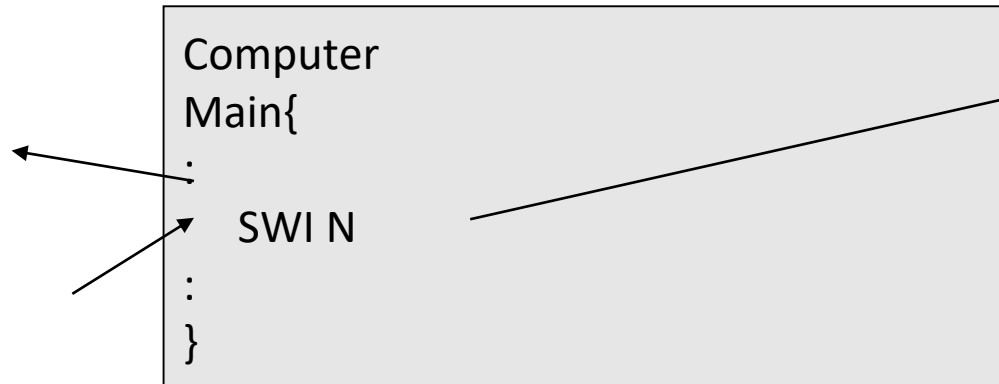


A falling edge at EINT3 will trigger the execution of the interrupt service routine `__irq IRQ_Eint1()`



• Software interrupt

```
N-th-sys-
routine()
{
::
}
```



An instruction "SWI N" in the program will trigger the execution of the N-th-sys-routine (system routine)

EXCEPTION (INTERRUPT) MODES

- ARM supports 7 types of exceptions and has a privileged processor mode for each type of exception.
- ARM Exception (interrupt) vectors

INTERRUPT VECTOR TABLE:

	Address	Exception	Mode in Entry
1	0x00000000	Reset	Supervisor
2	0x00000004	Undefined instruction	Undefined
3	0x00000008	Software Interrupt	Supervisor
4	0x0000000C	Abort (prefetch)	Abort
5	0x00000010	Abort (data)	Abort
x	0x00000014	Reserved	Reserved
6	0x00000018	IRQ (external interrupt)	IRQ
7	0x0000001C	FIQ (fast interrupt)	FIQ

SWI



SWI INTERRUPT PROCEDURES (enter the supervisor mode)

- **SWI (software interrupt)**
 - Caused by “**SWI 0x??**” in your program
 - Arm completes the current instruction.
 - **Goto SWI** exception address **0x08** (short form for 0x 0000 0008)
 - {Change to supervisor op. mode : CPSR (bit0-4)}
 - Return from interrupt
 - **MOVS pc, lr**
 - return to main

- Whenever a program requires **input or output**, for instance to send some **text to the display**, it is normal to **call** a supervisor routine.
 - **To read data from the keyboard or to display the contents on the console**
- The supervisor is a program which operates at a privileged level, which means that it can do things that a user-level program cannot do directly.
- The limitations on the capabilities of a user-level program vary from system to system, but in many systems the user cannot access hardware facilities directly.
- **The supervisor** provides trusted ways to access system resources which appear to the user-level program rather like **special subroutine** accesses.

SOFTWARE INTERRUPTS - 'SUPERVISOR CALL'

- The instruction set includes a special instruction, **SWI**, to call these functions, (SWI stands for '**Software Interrupt**', but is usually pronounced '**Supervisor Call**').
- **Note:** In **privileged modes** there is another register for each mode called **Saved Processor Status Register (SPSR)**.

The SPSR is used to **save the current CPSR** before changing modes.

- SWI interrupt occurs when the SWI instruction has been fetched and decoded successfully, and none of the other higher priority exceptions/interrupts have been flagged.
- Upon entry to the handler the CPSR will be set to SVC mode.
- **Note:** If a **SWI calls another SWI** (which is a common occurrence), then to avoid corruption, **the link register (LR & SPSR)** must be **stacked away** before branching to the nested SWI.

EXAMPLE 01

A procedure to compute the statement in high level language using ARM ALP.

if (R0==R1) R2++;

MOV R0, #10

MOV R1, #10

BL GREAT

SWI 0x11 ; terminate the program / logical end.

GREAT: CMP R0, R1

ADDEQ R2, R2, #1

MOV PC, LR

EXAMPLE 02

A program to display a string on the screen using ARM ALP.

System.out.print (“ Hello World”);

LDR R1, =A

LOOP: LDRB R0, [R1], #1

CMP R0, #0

SWINE 0x00 ; display a character on the screen.

BNE LOOP

SWI 0x11 ; terminate the program.

.DATA

A: .ASCIZ “HELLO WORLD”

EXAMPLE 03

A procedure to display a string on the screen using ARM ALP.

```
// System.out.print (" Hello World");
```

```
LDR  R1, =A
```

```
BL  strprint      ; call display routine
```

```
SWI 0x11          ; terminate the program.
```

```
strprint: LDRB R0, [R1], #1
```

```
CMP R0, #0
```

```
SWINE 0x00        ; display a character on the screen.
```

```
BNE strprint
```

```
MOV PC, LR
```

```
.DATA
```

```
A: .ASCIZ "HELLO WORLD"
```

EXAMPLE 04

A procedure to display a string on the screen using ARM ALP using routine 0x02.

.TEXT

LDR R0, =A

SWI 0x02 ; display a string on the screen

SWI 0x11 ; exit

.DATA

A: .ASCIZ "HELLO WORLD"

1. Which ARM instruction is used to call a subroutine, and what specific action does it perform regarding the return address?

- A) `MOV PC, LR` – It jumps to the subroutine and saves the current Program Counter in R0.
- B) `BL label` – It branches to the label and stores the address of the next instruction in the Link Register (R14).
- C) `SWI 0x11` – It triggers a software interrupt to jump to a subroutine.
- D) `BX LR` – It branches to the subroutine without saving the return address.

2. In a nested procedure call (where Subroutine A calls Subroutine B), what crucial step must be taken to prevent the loss of the return address?

- A) The Stack Pointer (SP) must be reset to zero.
- B) The Link Register (LR) must be pushed onto the stack before the second call is made.
- C) The Program Counter (PC) must be manually incremented.
- D) The CPSR register must be cleared.

1. Which ARM instruction is used to call a subroutine, and what specific action does it perform regarding the return address?

A) `MOV PC, LR` – It jumps to the subroutine and saves the current Program Counter in R0.

B) `BL label` – It branches to the label and stores the address of the next instruction in the Link Register (R14).

C) `SWI 0x11` – It triggers a software interrupt to jump to a subroutine.

D) `BX LR` – It branches to the subroutine without saving the return address.

2. In a nested procedure call (where Subroutine A calls Subroutine B), what crucial step must be taken to prevent the loss of the return address?

A) The Stack Pointer (SP) must be reset to zero.

B) The Link Register (LR) must be pushed onto the stack before the second call is made.

C) The Program Counter (PC) must be manually incremented.

D) The CPSR register must be cleared.

3. Which register is designated as the Stack Pointer in ARM architecture, and which instruction is typically used to pop data from the stack?*

- A) `R14` is the Stack Pointer; `LDR` is used to pop data.
- B) `R13` is the Stack Pointer; `LDMFD` is used to pop data.
- C) `R15` is the Stack Pointer; `STMFD` is used to pop data.
- D) `R12` is the Stack Pointer; `MOV` is used to pop data.

4. What is the fundamental difference between a Hardware Interrupt and a Software Interrupt (SWI)?

- A) Hardware interrupts are triggered by external hardware events (like a timer), while SWIs are triggered by executing a specific instruction in the code.
- B) Hardware interrupts always stop the processor, while SWIs run in the background.
- C) Hardware interrupts use the stack, while SWIs do not.
- D) SWIs are caused by errors (like division by zero), while Hardware interrupts are caused by user input.

3. Which register is designated as the Stack Pointer in ARM architecture, and which instruction is typically used to pop data from the stack?

- A) `R14` is the Stack Pointer; `LDR` is used to pop data.
- B) `R13` is the Stack Pointer; `LDMFD` is used to pop data.**
- C) `R15` is the Stack Pointer; `STMFD` is used to pop data.
- D) `R12` is the Stack Pointer; `MOV` is used to pop data.

4. What is the fundamental difference between a Hardware Interrupt and a Software Interrupt (SWI)?

- A) Hardware interrupts are triggered by external hardware events (like a timer), while SWIs are triggered by executing a specific instruction in the code.**
- B) Hardware interrupts always stop the processor, while SWIs run in the background.
- C) Hardware interrupts use the stack, while SWIs do not.
- D) SWIs are caused by errors (like division by zero), while Hardware interrupts are caused by user input.

5. When a Software Interrupt (SWI) occurs, into which mode does the processor switch, and to which vector address does it jump?

- A) It stays in User mode and jumps to `0x00000000`.
- B) It switches to Supervisor (SVC) mode and jumps to `0x00000008`.
- C) It switches to IRQ mode and jumps to `0x00000018`.
- D) It switches to FIQ mode and jumps to `0x0000001C`.

5. When a Software Interrupt (SWI) occurs, into which mode does the processor switch, and to which vector address does it jump?

- A) It stays in User mode and jumps to `0x00000000`.
- B) It switches to Supervisor (SVC) mode and jumps to `0x00000008`.**
- C) It switches to IRQ mode and jumps to `0x00000018`.
- D) It switches to FIQ mode and jumps to `0x0000001C`.



THANK YOU



Team MPCA

Department of Computer Science and
Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 9



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

PSR & SWAP Instructions

Move to Register from Status Register (MRS): Read from either CPSR or SPSR

Example:

MRS R0, CPSR

MRS R1, SPSR

Move to Status Register from Register (MSR):

_field

Example:

MSR CPSR_field, R0

MSR SPSR_field, R1

_c: Control Field (0:7)

_f: Flag Field(24:31)

_x: Extension (8:15)

_s:Status (16:23)

PSR Instructions

.text

MOVS R1,#-10

MRS R0,CPSR ; Take a copy of the CPSR.

AND R0,R0,#0000 ; Clear the mode bits.

MSR CPSR_F,R0 ; Write back the modified CPSR

.end

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000000
R1	: ffffffff6
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001004

CPSR Register	
Negative (N)	: 1
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

```
.text
```

```
MOVS R1,#-10
```

```
MRS R0,CPSR           ; Take a copy of the CPSR.
```

```
AND R0,R0,#0000        ; Clear the mode bits.
```

```
MSR CPSR_F,R0          ; Write back the modified CPSR
```

```
.end
```



RegistersView	
General Purpose Floating Point	
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 800000df
R1	: ffffffff6
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001008

CPSR Register	
Negative (N)	: 1
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

	0x800000df

PSR Instructions

```
.text
```

```
MOVS R1,#-10
```

```
MRS R0,CPSR           ; Take a copy of the CPSR.
```

```
AND R0,R0,#0000      ; Clear the mode bits.
```

```
MSR CPSR_F,R0         ; Write back the modified CPSR
```

```
.end
```



RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000000
R1	: ffffffff6
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 0000100c

CPSR Register	
Negative (N)	: 1
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

0x800000df	

PSR Instructions

```
.text
```

```
MOVS R1,#-10
```

```
MRS R0,CPSR      ; Take a copy of the CPSR.
```

```
AND R0,R0,#0000   ; Clear the mode bits.
```

```
MSR CPSR_F,R0    ; Write back the modified CPSR
```

```
.end
```

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 00000000
R1	: ffffffff6
R2	: 00000000
R3	: 00000000
R4	: 00000000
R5	: 00000000
R6	: 00000000
R7	: 00000000
R8	: 00000000
R9	: 00000000
R10 (s1)	: 00000000
R11 (fp)	: 00000000
R12 (ip)	: 00000000
R13 (sp)	: 00005400
R14 (lr)	: 00000000
R15 (pc)	: 00001010

CPSR Register	
Negative (N)	: 0
Zero (Z)	: 0
Carry (C)	: 0
Overflow (V)	: 0
IRQ Disable	: 1
FIQ Disable	: 1
Thumb (T)	: 0
CPU Mode	: System

0x000000df	

SWAP :A & R1

RegistersView	
General Purpose Floating Point	
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 0
R1	: 5
R2	: 0
R3	: 0
R4	: 0
R5	: 0
R6	: 0
R7	: 0
R8	: 0
R9	: 0
R10 (s1)	: 0
R11 (fp)	: 0
R12 (ip)	: 0
R13 (sp)	: 21504
R14 (lr)	: 0
R15 (pc)	: 4100

```
.text
00001000:E3A01005    MOV R1,#5
00001004:E59F2008    LDR R2,=a
00001008:E5923000    LDR R3,[R2]
0000100C:E5821000    STR R1,[R2]
00001010:E1A01003    MOV R1,R3
.data
00001018:          a:.word 10
```

MemoryView1

00001018 000A 0000 8181

SWAP : A & R1

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 0
R1	: 5
R2	: 4120
R3	: 10
R4	: 0
R5	: 0
R6	: 0
R7	: 0
R8	: 0
R9	: 0
R10 (s1)	: 0
R11 (fp)	: 0
R12 (ip)	: 0
R13 (sp)	: 21504
R14 (lr)	: 0
R15 (pc)	: 4112

```
.text
00001000:E3A01005    MOV R1,#5
00001004:E59F2008    LDR R2,=a
00001008:E5923000    LDR R3,[R2]
0000100C:E5821000    STR R1,[R2]
00001010:E1A01003    MOV R1,R3
.data
00001018:                a:.word 10
```

MemoryView1			
00001018			
00001018	0005	0000	81
0000103C	8181	8181	81

SWAP : A & R1

RegistersView	
General	Step Over Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 0
R1	: 10
R2	: 4120
R3	: 10
R4	: 0
R5	: 0
R6	: 0
R7	: 0
R8	: 0
R9	: 0
R10 (s1)	: 0
R11 (fp)	: 0
R12 (ip)	: 0
R13 (sp)	: 21504
R14 (lr)	: 0
R15 (pc)	: 4116

CPSR Register	

```
.text
00001000:E3A01005    MOV R1,#5
00001004:E59F2008    LDR R2,=a
00001008:E5923000    LDR R3,[R2]
0000100C:E5821000    STR R1,[R2]
00001010:E1A01003    MOV R1,R3
.data
00001018:          a:.word 10
```

MemoryView1	
00001018	
00001018	0005 0000 8

Swap Instruction

SWP <Swap Destination>, <Original>, [<address>]

Eg: SWP R0, R1, [R2]

.text

LDR R2, =a

MOV R1, #5

SWP **R1**, **R1**, [**R2**]

.data

a:.word 10

RegistersView	
General Purpose	Floating Point
Hexadecimal	
Unsigned Decimal	
Signed Decimal	
R0	: 0
R1	: 10
R2	: 4112
R3	: 0
R4	: 0
R5	: 0
R6	: 0
R7	: 0
R8	: 0
R9	: 0
R10 (s1)	: 0
R11 (fp)	: 0
R12 (ip)	: 0
R13 (sp)	: 21504
R14 (lr)	: 0
R15 (pc)	: 70656

```
.text
00001000:E59F2004 LDR R2,=a
00001004:E3A01005 MOV R1,#5
00001008:E1021091 SWP R1,R1,[R2]
.data
00001010: a:.word 10
```

After
Swapping

MemoryView1	
00001010	0005 0000 8181 8

1. Which instruction is used to read the contents of the Current Program Status Register (CPSR) or Saved Program Status Register (SPSR) into a general-purpose register?
A. MSR B. SWP C. MRS D. LDR

2. When using the MSR instruction to write to a status register, specific fields can be targeted. According to the slides, which bit range does the Flag Field (_f) cover?
A. Bits 0:7 B. Bits 8:15 C. Bits 16:23 D. bits24:31

1. Which instruction is used to read the contents of the Current Program Status Register (CPSR) or Saved Program Status Register (SPSR) into a general-purpose register?
- A. MSR B. SWP C. MRS D. LDR

Answer – C. MRS

2. When using the MSR instruction to write to a status register, specific fields can be targeted. According to the slides, which bit range does the Flag Field (_f) cover?
- A. Bits 0:7 B. Bits 8:15 C. Bits 16:23 D. Bits 24:31

Answer – D. Bits 24:31

3. Consider the following instruction : SWP R1, R1, [R2]. What operation does this instruction perform?
- A. It swaps the values inside registers R1 and R2.
 - B. It loads the word from memory at address R2 into R1 and stores the original value of R1 into memory at address R2.
 - C. It adds the value of R1 to the value at memory address R2.
 - D. It compares the value in R1 with the value at memory address R2
4. To safely modify the bits in the CPSR, What is the correct logical order of operations shown?
- A. MSR (Write) → AND (Modify) → MRS (Read)
 - B. MRS (Read) → AND (Modify) → MSR (Write)
 - C. LDR (Load) → STR (Store)
 - D. SWP (Swap) → MRS (Read)

3. Consider the following instruction : SWP R1, R1, [R2]. What operation does this instruction perform?

- A. It swaps the values inside registers R1 and R2.
- B. It loads the word from memory at address R2 into R1 and stores the original value of R1 into memory at address R2.
- C. It adds the value of R1 to the value at memory address R2.
- D. It compares the value in R1 with the value at memory address R2

Answer - B

4. To safely modify the bits in the CPSR, What is the correct logical order of operations shown?

- A. MSR (Write) → AND (Modify) → MRS (Read)
- B. MRS (Read) → AND (Modify) → MSR (Write)
- C. LDR (Load) → STR (Store)
- D. SWP (Swap) → MRS (Read)

Answer – B. MRS (Read) → AND (Modify) → MSR (Write)



THANK YOU



V R BADRI PRASAD

badriprasad@pes.edu

Department of Computer Science and Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 10

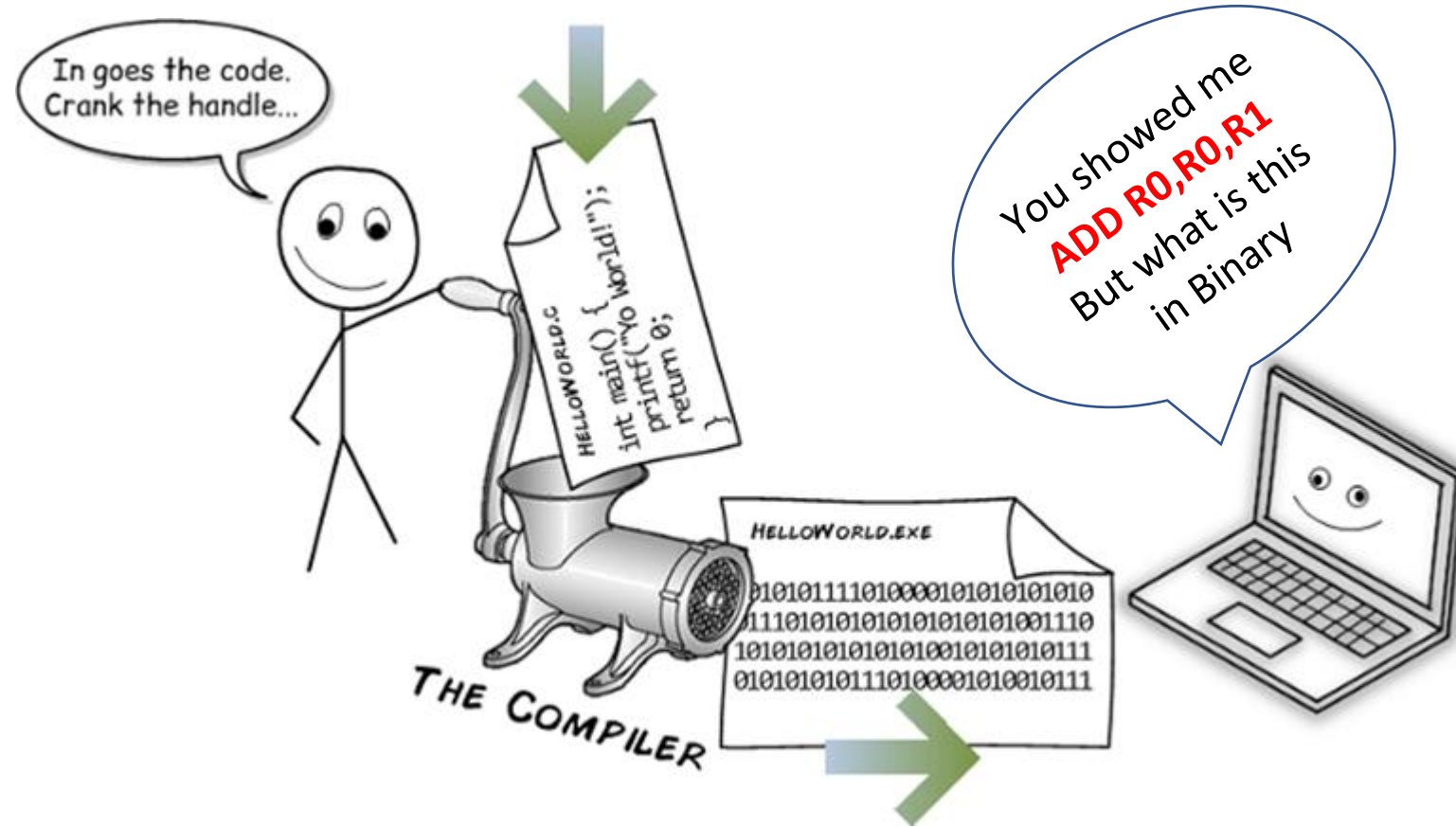


Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

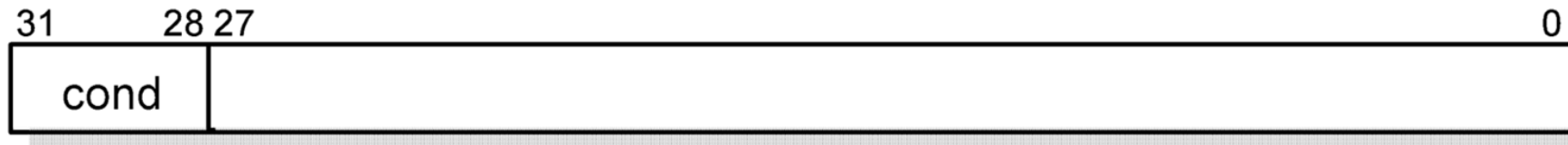
Instruction Encoding - 1

Instruction Encoding

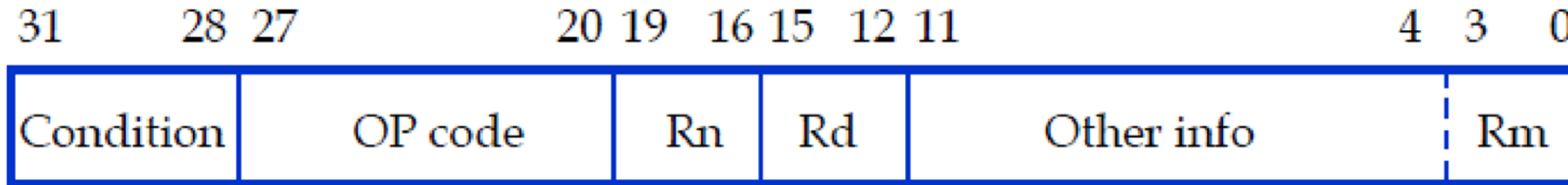


What is an ARM Instruction?

OPcode{condition}{S} Rd,Operand1,Operand2



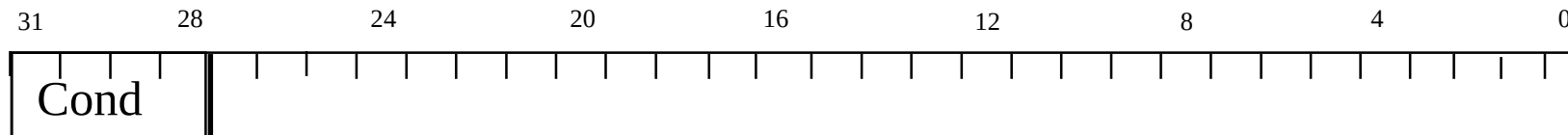
Example: Data Processing Instruction



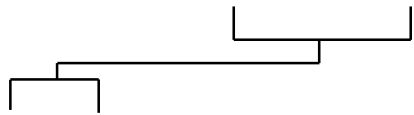
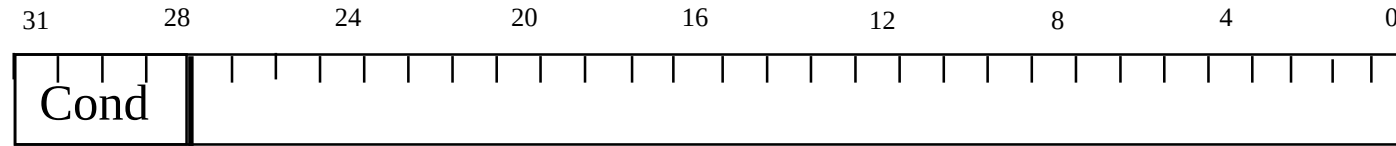
An instruction specifies a **Conditional Execution Code** (Condition), the **OP Code**, **Two or Three Registers** (Rn, Rd, and Rm), and some other information

- Fifteen different conditions may be used: these are listed in **Condition code summary**.
- The **sixteenth (1111) is reserved**, and must not be used.
- In the **absence** of a suffix, the condition field of most instructions is set to "**Always**" (suffix **AL**). This means the instruction will always be executed regardless of the CPSR condition codes.

The ARM condition code field



The Condition Field



0000 = EQ - Z set (equal)

0001 = NE - Z clear (not equal)

0010 = HS / CS - C set (unsigned higher or same)

0011 = LO / CC - C clear (unsigned lower)

0100 = MI - N set (negative)

0101 = PL - N clear (positive or zero)

0110 = VS - V set (overflow)

0111 = VC - V clear (no overflow)

1000 = HI - C set and Z clear (unsigned higher)

1001 = LS - C clear or Z (set unsigned lower or same)

1010 = GE - N set and V set, or N clear and V clear ($\geq$)

1011 = LT - N set and V clear, or N clear and V set ($>$)

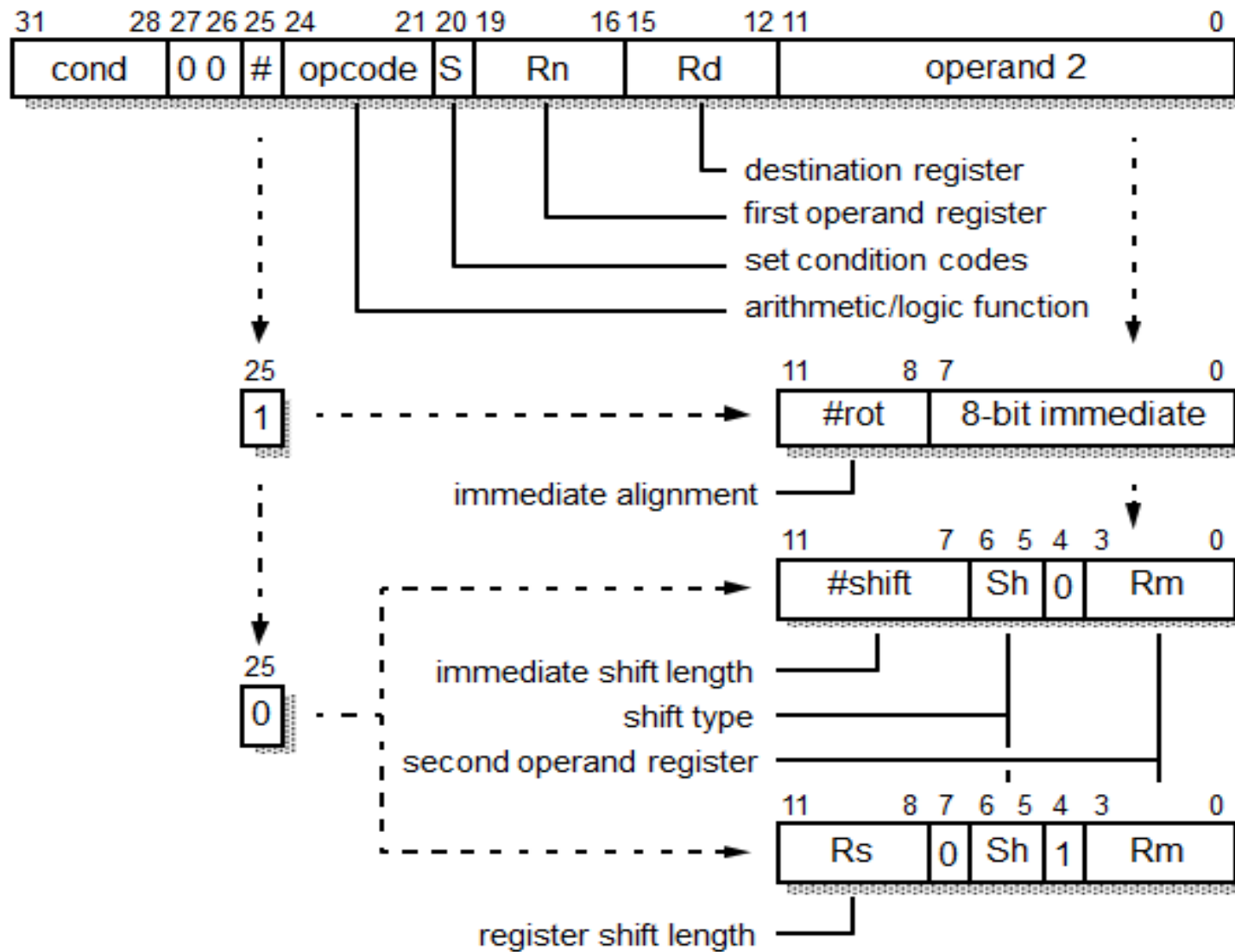
1100 = GT - Z clear, and either N set and V set, or N clear and V set ($>$)

1101 = LE - Z set, or N set and V clear, or N clear and V set ($\leq$, or $=$)

1110 = AL - always

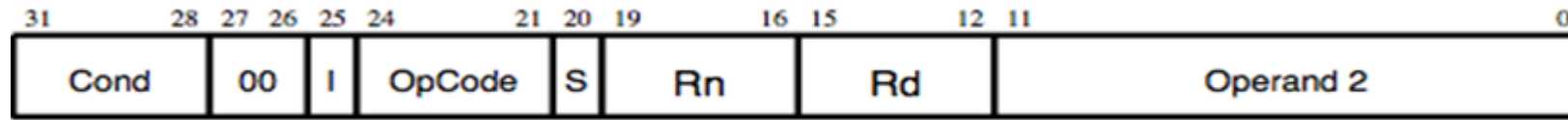
1111 = NV - reserved.

Data Processing Instruction



Opcode [24:21]	Mnemonic
0000	AND
0001	EOR
0010	SUB
0011	RSB
0100	ADD
0101	ADC
0110	SBC
0111	RSC
1000	TST
1001	TEQ
1010	CMP
1011	CMN
1100	ORR
1101	MOV
1110	BIC
1111	MVN

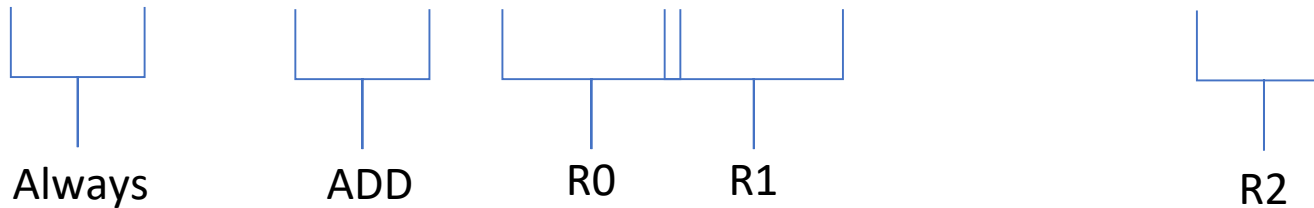
Data Processing Instruction - Example 01:



■ Ex:

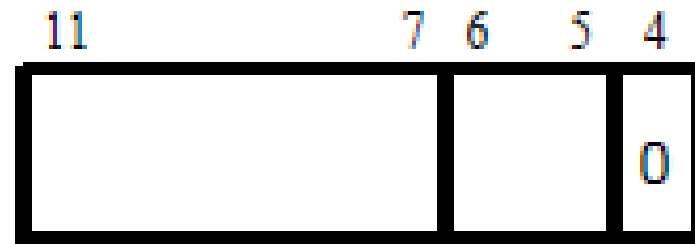
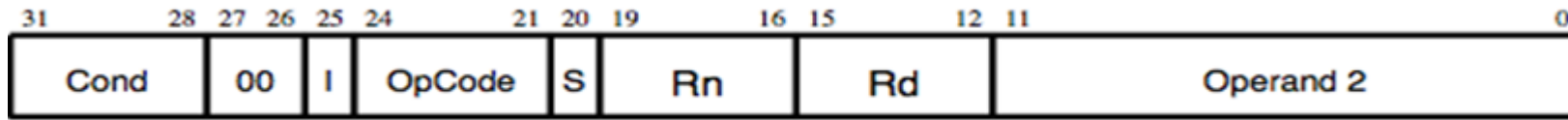
ADD R1, R0, R2

1110 000 0100 S Rn Rd 00000000 Rm
1110 0000 1000 0000 0001 0000 0000 0010



E0801002

Data Processing Instruction



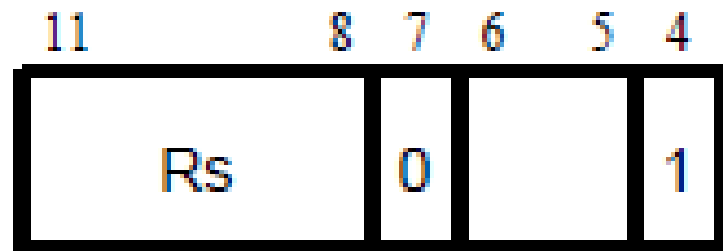
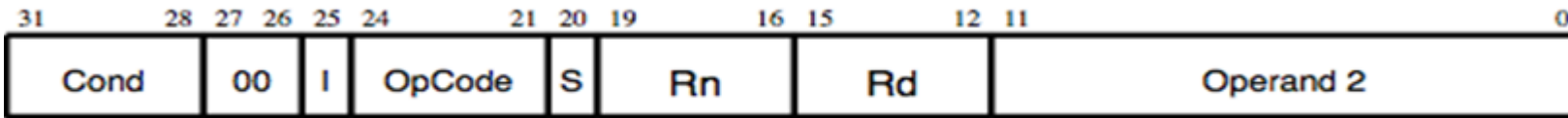
Shift type

- 00 = logical left
- 01 = logical right
- 10 = arithmetic right
- 11 = rotate right

Shift amount

5 bit unsigned integer

Data Processing Instruction



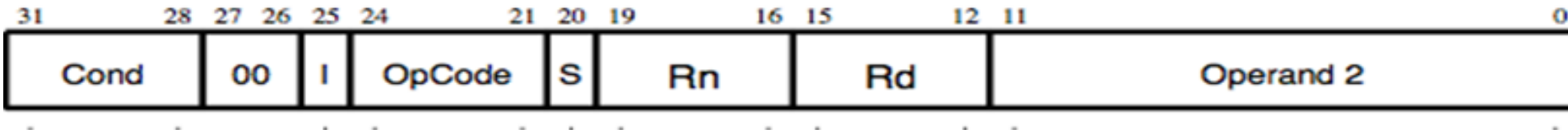
Shift type

- 00 = logical left
- 01 = logical right
- 10 = arithmetic right
- 11 = rotate right

Shift register

Shift amount specified in
bottom byte of Rs

Data Processing Instruction - Example 02:



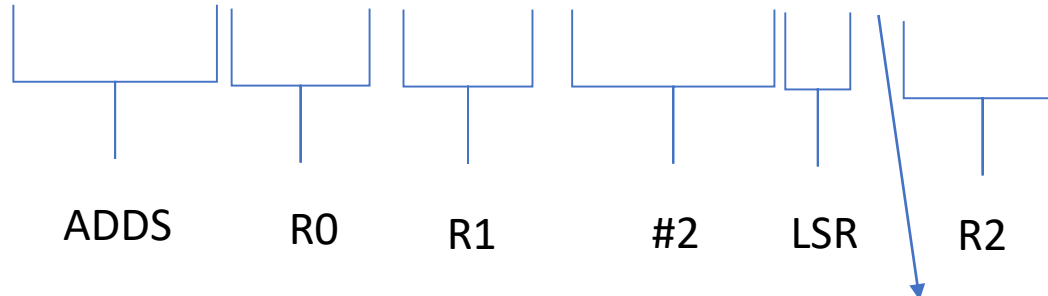
■ Ex:

ADDS R1, R0, R2 LSR #2

1110 000 0100 S Rn Rd Shift Rm

1110 000 0100 S Rn Rd 00010 01 0 Rm

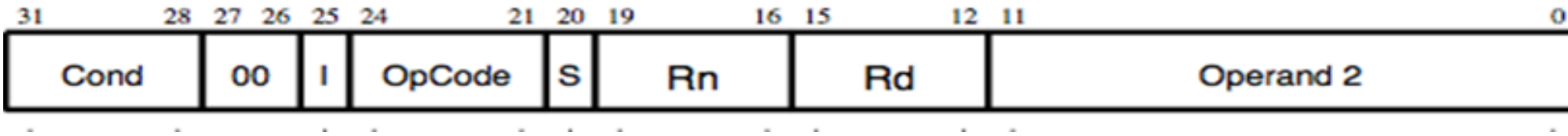
1110 0000 1001 0000 0001 0001 0010 0010



E0901122

Shift is specified as immediate

Data Processing Instruction Example 03:



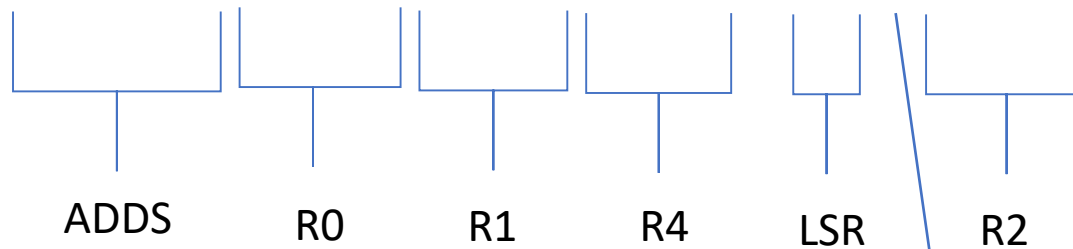
■ Ex:

ADDS R1, R0, R2 LSR R4

1110 000 0100 S Rn Rd Shift Rm

1110 000 0100 S Rn Rd 0100 0 01 1 Rm

1110 0000 1001 0000 0001 0100 0011 0010



Shift is specified in Register

[illegible]

Cond	0	0	1	Opcode				S	Rn		Rd		Operand 2											
Cond	0	0	0	0	0	0	A	S	Rd		Rn		Rs		1	0	0	1	Rm					
Cond	0	0	0	0	1	U	A	S	RdHi		RdLo		Rn		1	0	0	1	Rm					
Cond	0	0	0	1	0	B	0	0	Rn		Rd		0	0	0	0	1	0	0	1	Rm			
Cond	0	0	0	1	0	0	1	0	1	1	1	1	1	1	1	1	1	1	0	0	0	1	Rn	
Cond	0	0	0	P	U	0	W	L	Rn		Rd		0	0	0	0	1	S	H	1	Rm			
Cond	0	0	0	P	U	1	W	L	Rn		Rd		Offset			1	S	H	1	Offset				
Cond	0	1	1	P	U	B	W	L	Rn		Rd		Offset											
Cond	0	1	1														1							
Cond	1	0	0	P	U	S	W	L	Rn		Register List													
Cond	1	0	1	L	Offset																			

*Data Processing /
PSR Transfer*

Multiply

Multiply Long

Single Data Swap

Branch and Exchange

*Halfword Data Transfer:
register offset*

*Halfword Data Transfer:
immediate offset*

Single Data Transfer

Undefined

Block Data Transfer

Branch

MCQ's

1. If an instruction does not have a specific condition suffix (e.g., EQ, NE), it defaults to "Always" (AL).

What is the 4-bit binary code for this "Always" condition?

- A. 0000 B. 1111 C. 1110 D. 1000

2. In the standard 32-bit ARM data processing instruction format, specific bits are allocated for the Destination Register (Rd). Which bit range corresponds to Rd?

- A. Bits 24:21 B. Bits 19:16 C. Bits 15:12 D. Bits 11:0

3. When using Immediate Shift in Operand 2, a 2-bit field specifies the type of shift to be performed.

Which binary value represents a Logical Shift Right (LSR)?

- A. 00 B. 01 C. 10 D. 11

MCQ's

1. If an instruction does not have a specific condition suffix (e.g., EQ, NE), it defaults to "Always" (AL). What is the 4-bit binary code for this "Always" condition?

- A. 0000 B. 1111 C. 1110 D. 1000

Answer – C. 1110

2. In the standard 32-bit ARM data processing instruction format, specific bits are allocated for the Destination Register (Rd). Which bit range corresponds to Rd?

- A. Bits 24:21 B. Bits 19:16 C. Bits 15:12 D. Bits 11:0

Answer – Bits 15:12

3. When using Immediate Shift in Operand 2, a 2-bit field specifies the type of shift to be performed. Which binary value represents a Logical Shift Right (LSR)?

- A. 00 B. 01 C. 10 D. 11

Answer – B. 01

MCQ's

4. In the instruction encoding format, bit 20 is labeled as 'S'. What is the function of this bit?

- A. It indicates that the instruction is a Store operation.
- B. It specifies the size of the operand (byte or word).
- C. It determines if the instruction updates the Condition Codes (CPSR).
- D. It selects the Source register.

5. Based on the table of Opcodes (Bits 24:21) in the slides, which 4-bit binary sequence represents the MOV (Move) instruction?

- A. 0100
- B. 1010
- C. 1101
- D. 1110

4. In the instruction encoding format, bit 20 is labeled as 'S'. What is the function of this bit?

- A. It indicates that the instruction is a Store operation.
- B. It specifies the size of the operand (byte or word).
- C. It determines if the instruction updates the Condition Codes (CPSR).
- D. It selects the Source register.

Answer - C. It determines if the instruction updates the Condition Codes (CPSR).

5. Based on the table of Opcodes (Bits 24:21) presented in the slides, which 4-bit binary sequence represents the MOV (Move) instruction?

- A. 0100
- B. 1010
- C. 1101
- D. 1110

Answer – C. 1101



THANK YOU



V R BADRI PRASAD

badriprasad@pes.edu

Department of Computer Science and Engineering



Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Session - 11

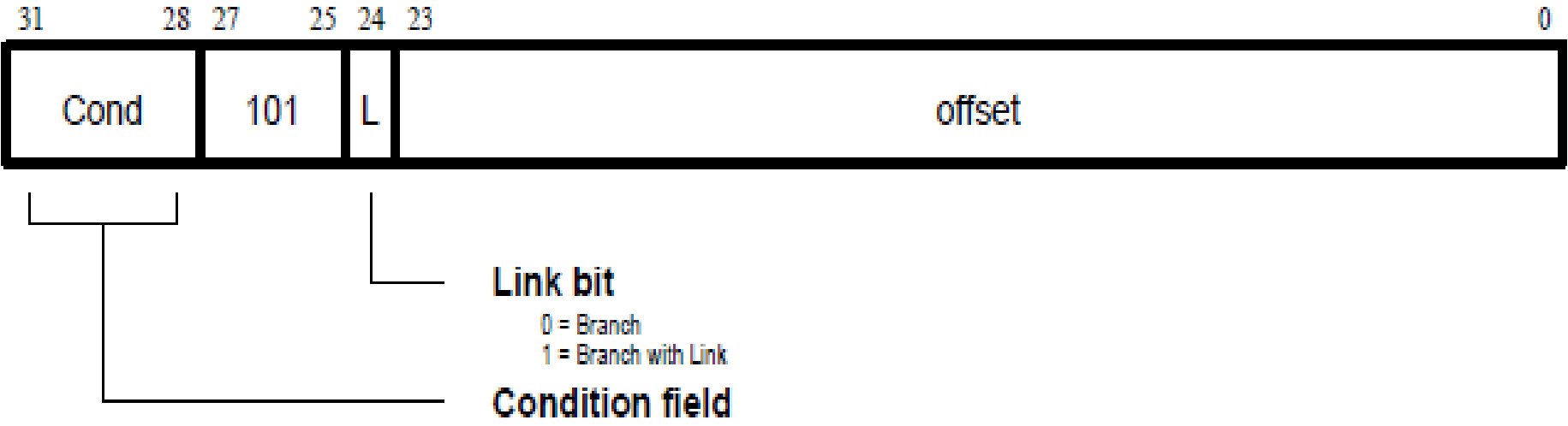


Microprocessor & Computer Architecture (μ pCA)

UE24CS251B

Instruction Encoding – 2

Branch Instruction



Bits	Field	Description
31–28	cond	Condition code (e.g., EQ, NE, AL) – determines if the branch executes.
27–25	101	Identifies branch-type instruction.
24	L	Link bit: 0 = branch only, 1 = branch with link (stores return address in LR).
23–0	imm24	Signed 24-bit immediate offset (word-aligned).

Branch Target Address= (PC+8)+(offset<<2)

PC=PC+8-the Program Counter (PC) does not hold the address of the currently executing instruction. Instead, due to instruction pipelining, it points ahead by two instructions.

offset is shifted left by 2 (since ARM instructions are 4-byte aligned)

$$\text{Offset} = \frac{\text{Target Address} - (\text{PC} + 8)}{4}$$

PES
UNIVERSITY
ONLINE

```

.text
00001000:E3A01003    main:    mov r1, #3
00001004:EB000003          bl foo
00001008:E0802001          add r2, r0, r1
0000100C:E0802001          add r2, r0, r1
00001010:E2800001    add r0, r0, #1
00001014:EF000011          swi 0x011
00001018:                foo:
00001018:E3A00002          mov r0, #2
0000101C:E12FFF1E          bx lr
.end

```

- $$\text{Offset} = \frac{\text{Target Address} - (\text{PC} + 8)}{4}$$

$$\text{Offset} = \frac{0x1018 - 0x100C}{4} = \frac{0xC}{4} = 3$$

[illegible]

PES
UNIVERSITY
ONLINE

example_encoding2.s

- **Target Address = 0x1000 (address of foo)**
- **PC at Execution = 0x100C + 8 = 0x1014 (because of the ARM pipeline)**

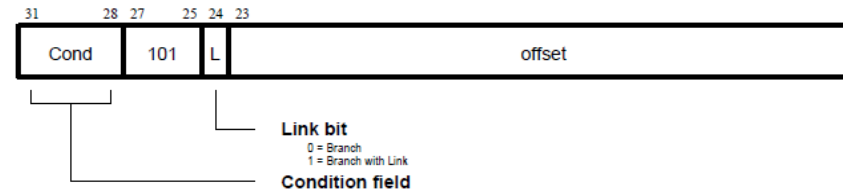
- **Offset**= $0X1000-0X1014 / 4 = -0X14/4$
 $= -20 \text{ decimal}/4$
 $= -5$

Backward branch -
take two's complement

[illegible]

Microprocessor & Computer Architecture (μpCA)

```
00001000:E3A02004  MOV r2,#4
00001004:E3A06005  MOV r6,#5
00001008:E3A07007  mov r7,#7
0000100C:EA000002  B forward
00001010:E2821004  ADD r1, r2, #4
00001014:E2860002  ADD r0, r6, #2
00001018:E2873004  ADD r3, r7, #4
0000101C:          forward:
0000101C:E2421004  SUB r1, r2, #4
00001020:          backward:
00001020:E2821004  ADD r1, r2, #4
00001024:E2421004  SUB r1, r2, #4
00001028:E0864007  ADD r4, r6, r7
0000102C:EAFFFFFB  B backward
00001030:E0411002  SUB r1,r1,r2
00001034:E0811002  ADD r1,r1,r2
```



PES
UNIVERSITY
ONLINE

Diagram illustrating the Branch with Link (btl) instruction format. The instruction is 32 bits long, divided into four fields:

- Cond** (5 bits): Condition field.
- 101** (3 bits): Fixed value.
- L** (1 bit): Link bit.
- offset** (27 bits): Branch offset.

Legend:

- Link bit**: 0 = Branch, 1 = Branch with Link.
- Condition field**: The Cond field.

$$\text{Offset} = \frac{0x0000101C - 0x00001014}{4}$$
[illegible]

PES
UNIVERSITY
ONLINE

31 28 27 25 24 23

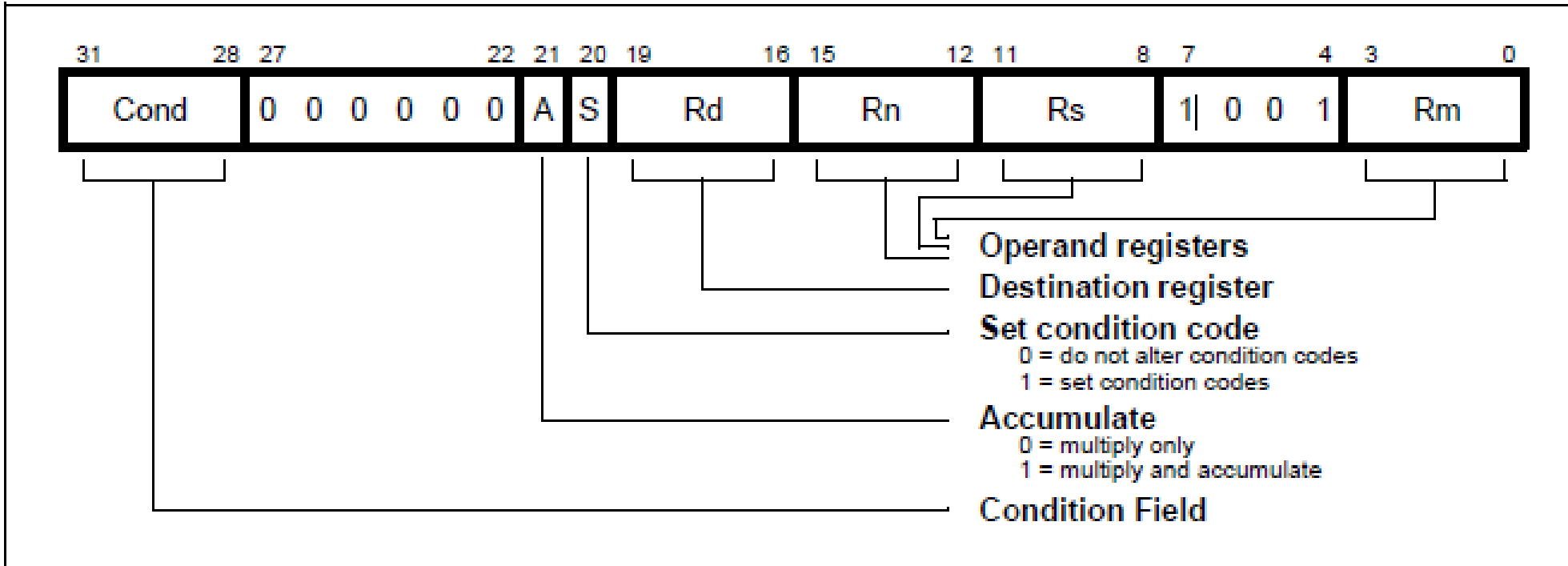
Cond	101	L	offset
------	-----	---	--------

Link bit
0 = Branch
1 = Branch with Link

Condition field

$$\begin{aligned}\text{Offset} &= \frac{0x00001020 - 0x00001034}{4} \\ \text{Offset} &= \frac{-0x14}{4} = -5\end{aligned}$$
[illegible]

Multiplication Instructions

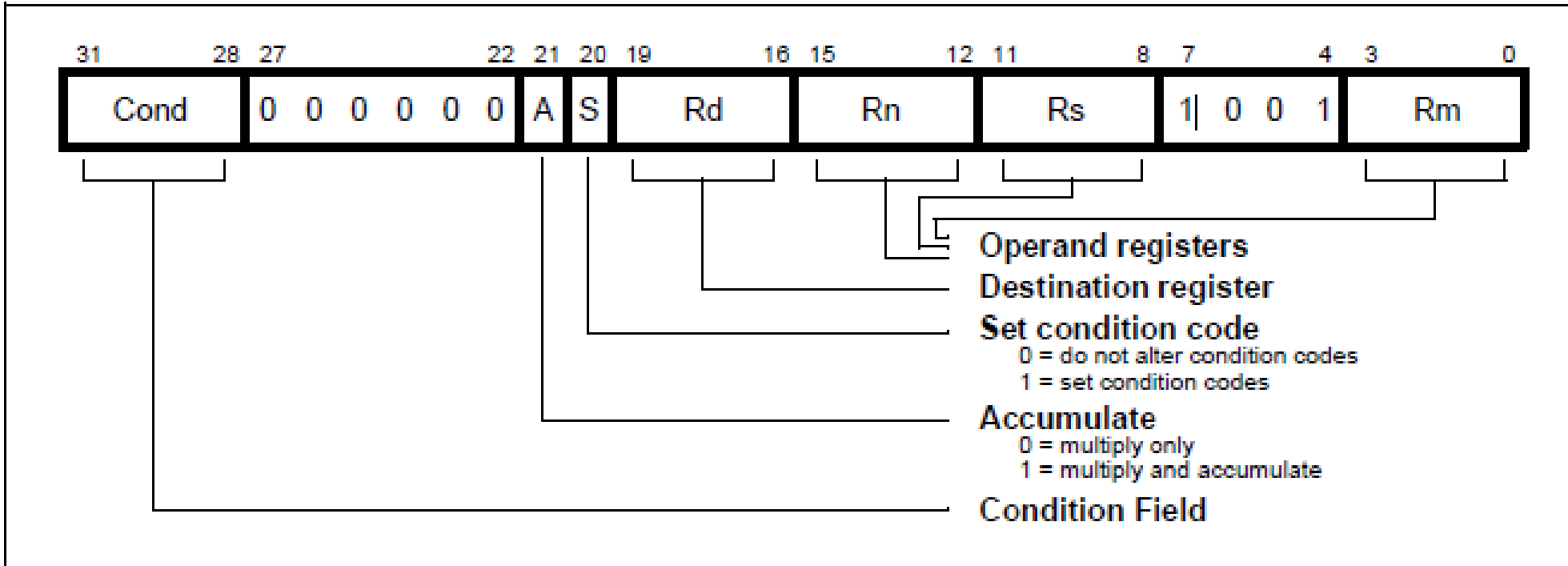


Ex 1: MUL R0, R1, R2 (MUL Rd,Rm,Rs)

E0000291

3 1	3 0	2 9	2 8	2 7	2 6	2 5	2 4	2 3	2 2	2 1	2 0	1 9	1 8	1 7	1 6	1 5	1 4	1 3	1 2	1 1	1 0	9	8	7	6	5	4	3	2	1	0
										A	S	Rd				Rn				Rs								Rm			
1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	1	0	0	1

Multiplication Instructions

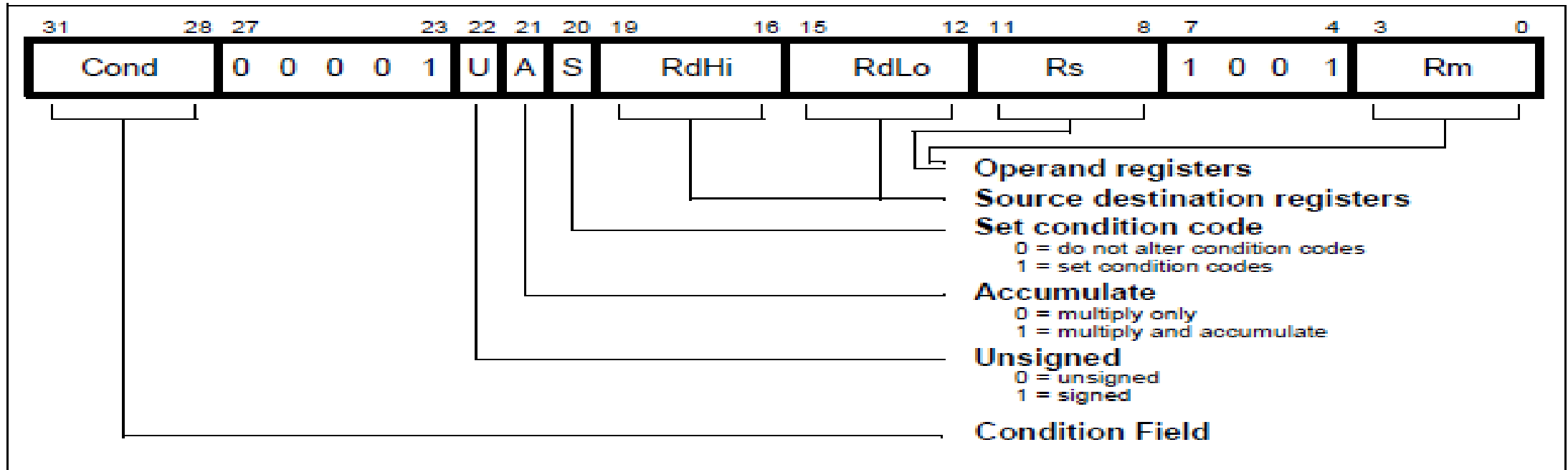


Ex 2: MLA R0, R1, R2 , R3 (MLA Rd,Rm,Rs,Rn)

E0203291

3 1	3 0	2 9	2 8	2 7	2 6	2 5	2 4	2 3	2 2	2 1	2 0	1 9	1 8	1 7	1 6	1 5	1 4	1 3	1 2	1 1	1 0	9	8	7	6	5	4	3	2	1	0	
Cond										A	S	Rd				Rn				Rs								Rm				
1	1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	1	1	0	0	1	0	1	0	0	0	1	0	0	0	1

Multiplication Instructions

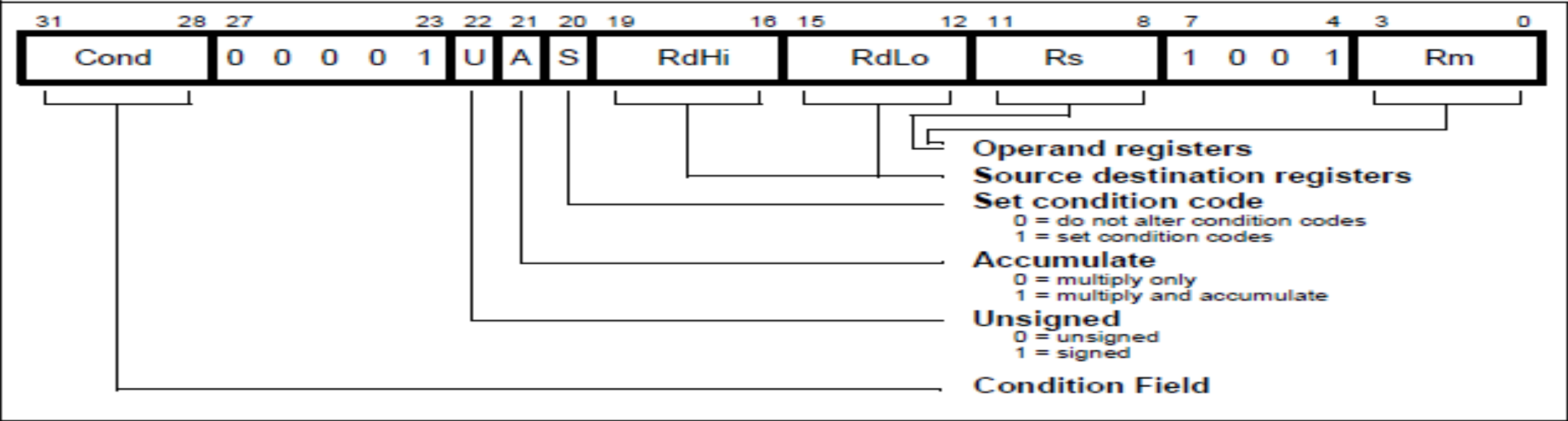


Ex 3: SMULL R0, R1, R2, R3 (SMULL RdLo,RdHi,Rm,Rs)

E0C10392

3 1	3 0	2 9	2 8	2 7	2 6	2 5	2 4	2 3	2 2	2 1	2 0	1 9	1 8	1 7	1 6	1 5	1 4	1 3	1 2	1 1	1 0	9	8	7	6	5	4	3	2	1	0
Cond									U	A	S	RdHi				RdLo				Rs								Rm			
1	1	1	0	0	0	0	0	1	1	0	0	0	0	0	1	0	0	0	0	0	0	1	1	1	0	0	1	0	0	1	0

Multiplication Instructions

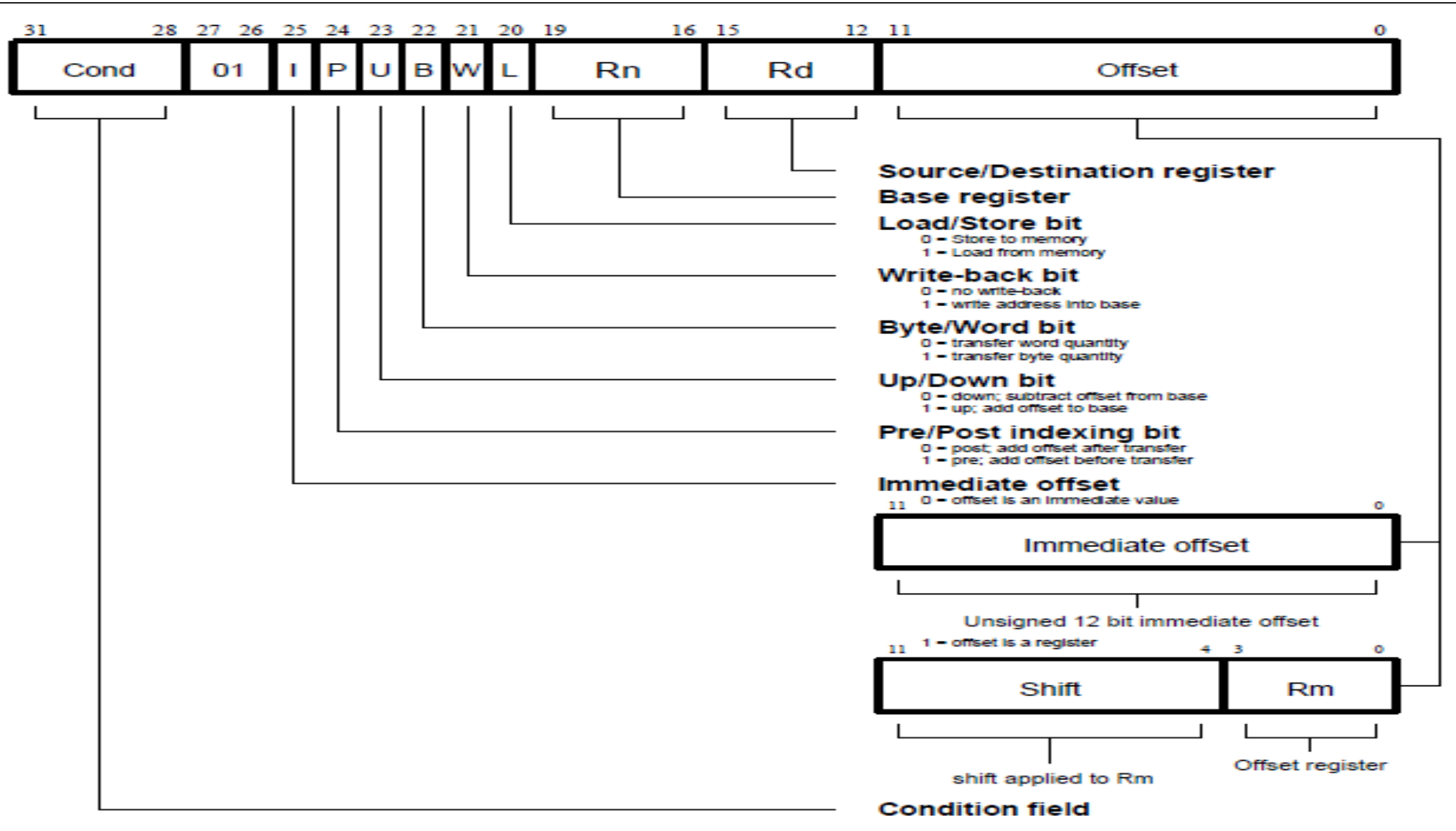


Ex 4: UMLAL R0, R1, R2 , R3 (UMLAL RdLo,RdHi,Rm,Rs)

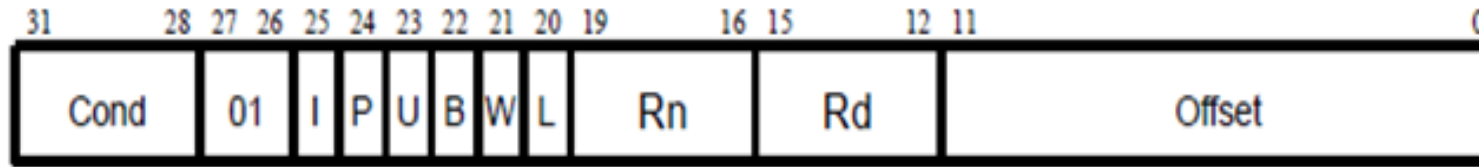
E0A10392

3 1	3 0	2 9	2 8	2 7	2 6	2 5	2 4	2 3	2 2	2 1	2 0	1 9	1 8	1 7	1 6	1 5	1 4	1 3	1 2	1 1	1 0	9	8	7	6	5	4	3	2	1	0
Cond									U	A	S	RdHi				RdLo				Rs								Rm			
1	1	1	0	0	0	0	0	1	0	1	0	0	0	0	1	0	0	0	0	0	0	1	1	1	0	0	1	0	0	1	0

Data Transfer Instruction



Data Transfer Instruction:STR



■ Ex:

STR R0, [R1]

1110 01 0 1 1 0 0 0 0001 0000 0000 0000 0000

1110 01 I P U B W L 0001 0000 0000 0000 0000

Binary Code: E5810000

Data Transfer Instruction:LDR

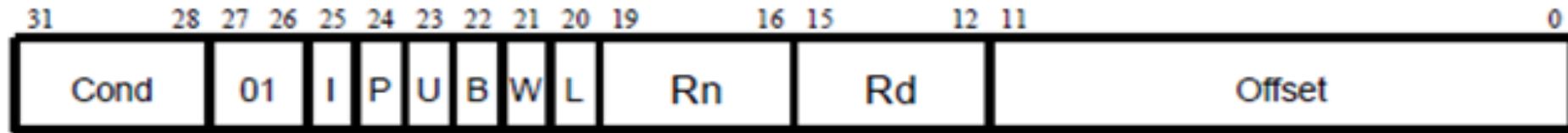
■ Ex:

LDR R0, [R1], R2

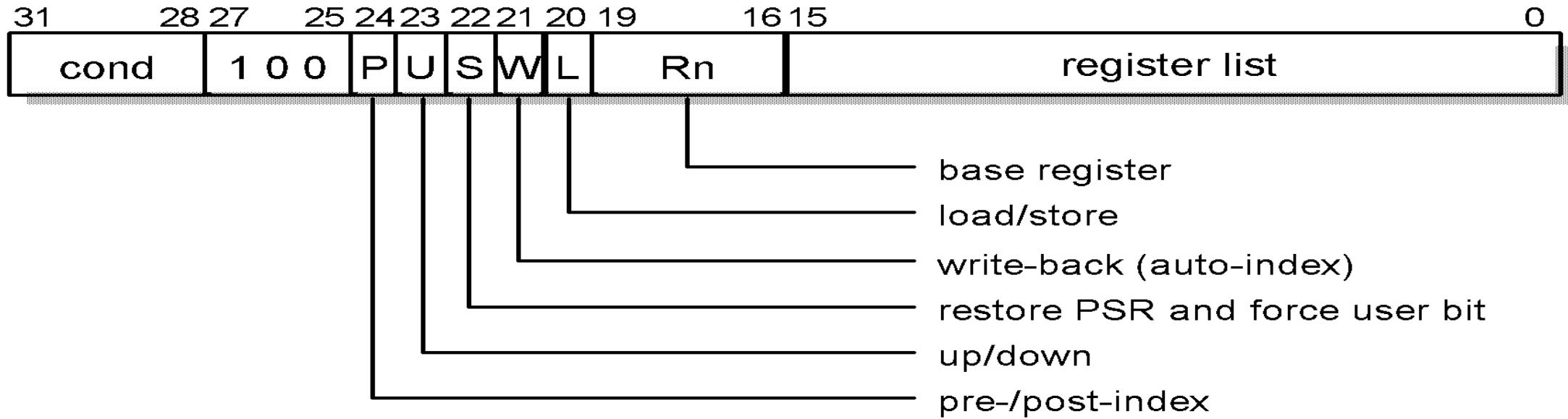
1110 01 I P U B W L 0001 0000 0000 0000 0010

1110 01 1 0 1 0 0 1 0001 0000 0000 0000 0010

E6910002



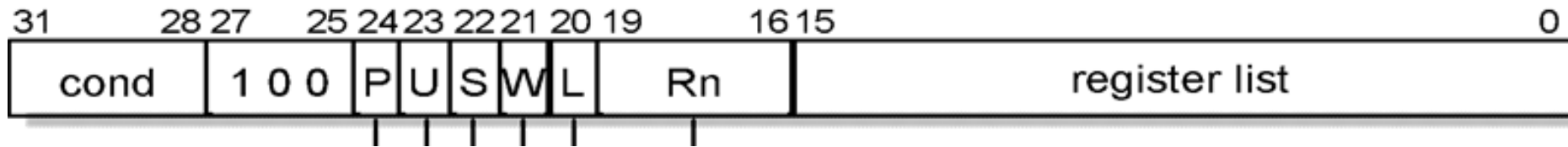
Block Transfer Instructions



Ex 1: LDMIA R13! , { R0, R5 - R8, R11}

Ex 2: STMIB R13! , { R8, R4- R6, R12}

Block Transfer Instruction : LDM



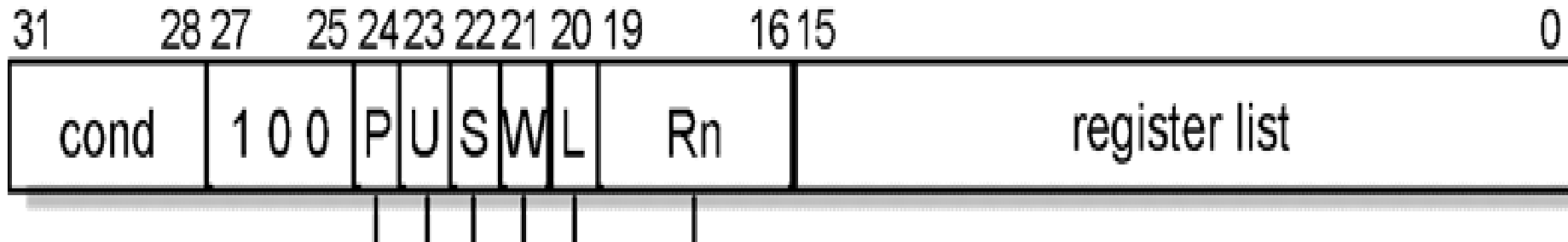
LDMIA R13! , { R0, R5 - R8, R11}

1110 100 **P** **U** **S** **W** **L** 1101 **0000** **R11** **00****R8****R7****R6****R5** **0000****R0**

1110 100 **0** **1** **0** **1** **1** 1101 **0000** **1** **00****1** **1****1****1** **0000****1**

E8BD09E1

Block Transfer Instruction : STM



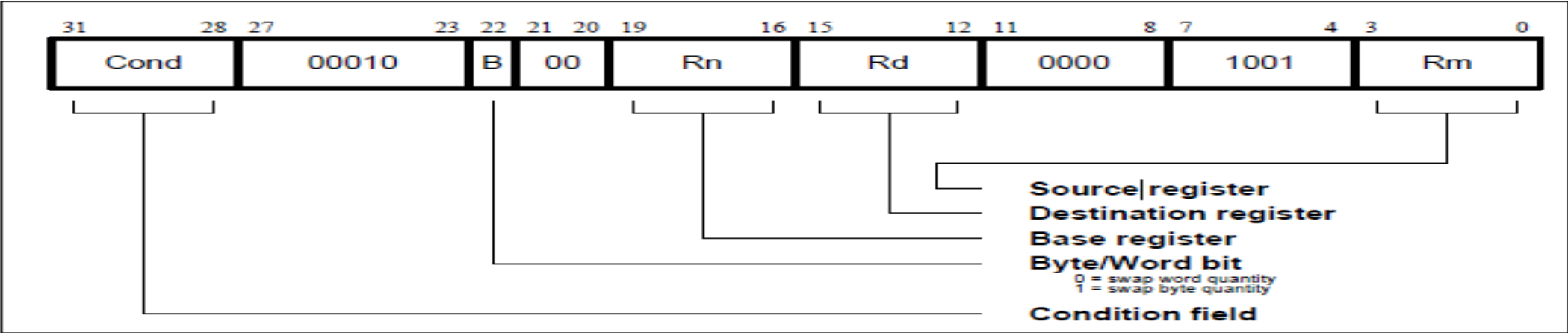
STMIB R13! , { R8, R4- R6, R12}

1110 100 **1** 1 **0** **1** **0** 1101 000**1**0 00**1**0**11**0000

1110 100 **P** **U** **S** **W** **L** 1101 000**R12**0 00**R8** 0**R6R5R4** 0000

E9AD1170

SWAP:



SWP{B}{cond} Rd, Rm, [Rn]

|31-28|27-24|23|22|21|20|19-16|15-12|11-8|7-4 |3-0 |
| cond| 0001 | 0| B | 0| 0| Rn | Rd |0000|1001| Rm |

Eg.: SWP R1, R2, [R3]

1110 0001 0000 0011 0001 0000 1001 0010 E1031092

3	3	2	2	2	2	2	2	2	2	2	2	1	1	1	1	1	1	1	1	1	0	9	8	7	6	5	4	3	2	1	0
1	0	9	8	7	6	5	4	3	2	1	0	9	8	7	6	5	4	3	2	1	0										
Cond												B				Rn				Rd								Rm			
1	1	1	0	0	0	0	1	0	0	0	0	0	0	1	1	0	0	0	1	0	0	0	0	0	0	1	0	0	1	0	0

1. In ARM Branch instruction format, what is the function of the Link bit (L) located at bit position 24?
 - A. It determines if the offset is positive or negative.
 - B. It distinguishes between a standard Branch (B) and a Branch with Link (BL).
 - C. It indicates if the instruction allows conditional execution.
 - D. It sets the processor to Thumb mode.

2. Which specific bit in the Multiplication instruction encoding is set to '1' to convert a standard Multiply (MUL) operation into a Multiply-Accumulate (MLA) operation?
 - A. The S bit (Set Condition Code)
 - B. The U bit (Unsigned)
 - C. The A bit (Accumulate)
 - D. The W bit (Write-back)

1. In ARM Branch instruction format, what is the function of the Link bit (L) located at bit position 24?
- A. It determines if the offset is positive or negative.
 - B. It distinguishes between a standard Branch (B) and a Branch with Link (BL).
 - C. It indicates if the instruction allows conditional execution.
 - D. It sets the processor to Thumb mode.

Answer – B. It distinguishes between a standard Branch (B) and a Branch with Link (BL).

2. Which specific bit in the Multiplication instruction encoding is set to '1' to convert a standard Multiply (MUL) operation into a Multiply-Accumulate (MLA) operation?
- A. The S bit (Set Condition Code)
 - B. The U bit (Unsigned)
 - C. The A bit (Accumulate)
 - D. The W bit (Write-back)

Answer – C. The A bit (Accumulate)

3. In the Single Data Transfer instruction format (used for LDR/STR), which bit indicates whether the data being transferred is a Byte or a Word quantity?

- A. The L bit
- B. The B bit
- C. The W bit
- D. The P bit

4. Which ARM instruction is identified by the specific encoding pattern where bits 27–23 are 00010 and bits 7–4 are 1001?

- A. SWAP (SWP)
- B. Branch Exchange (BX)
- C. Store Multiple (STM)
- D. Multiply Long (SMULL)

3. In the Single Data Transfer instruction format (used for LDR/STR), which bit indicates whether the data being transferred is a Byte or a Word quantity?

- A. The L bit
- B. The B bit
- C. The W bit
- D. The P bit

Answer – B. The B bit

4. Which ARM instruction is identified by the specific encoding pattern where bits 27–23 are 00010 and bits 7–4 are 1001?

- A. SWAP (SWP)
- B. Branch Exchange (BX)
- C. Store Multiple (STM)
- D. Multiply Long (SMULL)

Answer – A. SWAP (SWP)



THANK YOU



V R BADRI PRASAD

badriprasad@pes.edu

Department of Computer Science and Engineering